**Republic of Tunisia**

**Ministry of Higher Education**

**and Scientific Research**

**Private Higher School of Technologies and Engineering**

**TEK-UP**

DESIGNET

## Graduation Project Report

**Presented in view of obtaining the**

**National Engineering Diploma in Applied and Technological Sciences**

**Specialization : Computer and Network Security System**

Submitted by

**Aymen AZIZI**

# Design and Development of a Web Platform for Attack Surface Reconnaissance with AI-Assisted Report Generation and DevSecOps Integration

Professional Supervisor : **Mr Ali DORBOZ** — Chief Technical Officer

Academic Supervisor : **Mme Sonia BEN AISSA** — Academic Supervisor

Academic Year 2025 - 2026

I authorize the student to submit their internship report for the defense.

Professional Supervisor, **Mr Ali DORBOZ**

**Signature and stamp**

I authorize the student to submit their internship report for the defense.

Academic Supervisor, **Mme Sonia BEN AISSA**

**Signature**

# Dedication

***To my beloved parents :***

Your unwavering love, support, and belief in me have been my guiding light throughout this journey. Without your sacrifices and encouragement, none of this would have been possible.

***To my brothers and sisters :***

Thank you for your constant encouragement, motivation, and for being there during the most challenging moments of this project. Your presence has been a source of strength and inspiration.

***To all my family members and friends :***

I am deeply grateful for your support, guidance, and continuous encouragement throughout my years of study. Each of you has contributed to my journey and to this achievement in your own unique way.

*Aymen AZIZI*

# *Acknowledgements*

*It is with great pleasure that I dedicate this page as a token of gratitude and deep appreciation to all those who contributed to the completion of this work.*

*First and foremost, I would like to thank the jury members who kindly agreed to evaluate this modest project. Their time, expertise, and constructive feedback are deeply appreciated.*

*I would also like to express my sincere gratitude to my academic supervisor at the Private School of Engineering and Technology* ***TEK-UP****, Mrs* ***Sonia Ben Aissa****, who continuously supported me and generously shared her experience to help me complete this final-year project under the best conditions. I especially thank her for her patience and kindness.*

*Furthermore, I extend my heartfelt thanks to my internship supervisor at* ***Designet Web Agency****, Mr* ***Ali DORBOZ****, Chief Technical Officer, who shared his expertise with me and provided valuable advice and guidance throughout the project despite his many responsibilities. His mentorship was instrumental in the successful completion of this work.*

*I would also like to acknowledge the administrative team at Designet Web Agency, particularly Mr* ***Raed Ben Khlifa****, for welcoming me into the company and providing a supportive professional environment.*

*Finally, I would like to thank all of our professors who have guided and educated us throughout our years of study at TEK-UP, and who have prepared us for the challenges of the professional world.*

# Table des matières

# Table des figures

# Liste des tableaux

# List of Abbreviations

- **2FA** = Two-Factor Authentication
- **ABAC** = Attribute-Based Access Control
- **ACL** = Access Control List
- **AGE** = Apache AGE (A Graph Engine)
- **AI** = Artificial Intelligence
- **API** = Application Programming Interface
- **ASM** = Attack Surface Management
- **AWS** = Amazon Web Services
- **BFS** = Breadth-First Search
- **CD** = Continuous Deployment
- **CI** = Continuous Integration
- **CLI** = Command-Line Interface
- **CMS** = Content Management System
- **CSP** = Content Security Policy
- **CVE** = Common Vulnerabilities and Exposures
- **CVSS** = Common Vulnerability Scoring System
- **CWE** = Common Weakness Enumeration
- **DAST** = Dynamic Application Security Testing
- **DNS** = Domain Name System
- **DVWA** = Damn Vulnerable Web Application
- **DevOps** = Development and Operations
- **DevSecOps** Development, Security, and Operations
- **DoS** = Denial of Service
- **GDPR** = General Data Protection Regulation
- **GHCR** = GitHub Container Registry
- **GUI** = Graphical User Interface
- **HSTS** = HTTP Strict Transport Security

— **HTTP** = Hypertext Transfer Protocol

— **HTTPS** = Hypertext Transfer Protocol Secure

— **IDS** = Intrusion Detection System

— **IP** = Internet Protocol

— **IaC** = Infrastructure as Code

— **JSON** = JavaScript Object Notation

— **JSONB** = JSON Binary (PostgreSQL column type)

— **JSONL** = JavaScript Object Notation Lines

— **LLM** = Large Language Model

— **MFA** = Multi-Factor Authentication

— **MVC** = Model-View-Controller

— **NAT** = Network Address Translation

— **NSE** = Nmap Scripting Engine

— **NVD** = National Vulnerability Database

— **OIDC** = OpenID Connect

— **OPA** = Open Policy Agent

— **ORM** = Object-Relational Mapping

— **OS** = Operating System

— **OSINT** = Open Source Intelligence

— **OWASP** = Open Worldwide Application Security Project

— **PII** = Personally Identifiable Information

— **PoA** = Permission of Action (Proof of Authorization)

— **RBAC** = Role-Based Access Control

— **REST** = Representational State Transfer

— **SAN** = Subject Alternative Name (X.509 certificate)

— **SAST** = Static Application Security Testing

— **SBOM** = Software Bill of Materials

— **SCA** = Software Composition Analysis

— **SQL** = Structured Query Language

— **SQLi** = SQL Injection

— **SSH** = Secure Shell

— **SSIR** = Sécurité des Systèmes d'Information et Réseaux

— **SSRF** = Server-Side Request Forgery

— **TLS** = Transport Layer Security

— **URL** = Uniform Resource Locator

— **VPC** = Virtual Private Cloud

— **VPN** = Virtual Private Network

— **WAF** = Web Application Firewall

— **WPScan** = WordPress Security Scanner

— **XSS** = Cross-Site Scripting

— **YAML** = YAML Ain't Markup Language

# General Introduction

In today's rapidly evolving digital landscape, organizations face an ever-growing number of cyber threats that exploit vulnerabilities in their exposed web infrastructure. The expansion of attack surfaces—driven by cloud adoption, microservices architectures, and the proliferation of web applications—has made manual security testing both inefficient and insufficient. Security teams are overwhelmed by the sheer volume of potential entry points, and the lack of centralized, automated reconnaissance tools leads to delayed vulnerability detection, inconsistent reporting, and increased exposure to attacks.

This final-year project aims to address these challenges by developing a comprehensive web platform for attack surface reconnaissance, integrating AI-assisted report generation and DevSecOps practices. The platform, built at **Designet Web Agency**, leverages a microservices architecture combining **Laravel 11** (PHP 8.3) for the backend web application and **Python Flask** for five scanning microservices (reconnaissance, security, OSINT, AI, and API gateway). It orchestrates industry-standard security tools—including **Nmap**, **Nuclei**, **Gobuster**, **Subfinder**, and **WPScan**—to automate the discovery of exposed services, vulnerabilities, and misconfigurations across target infrastructures. Results are correlated through a knowledge graph stored in **PostgreSQL 16 with Apache AGE**, enabling blast-radius computation via **NetworkX**.

Beyond traditional reconnaissance, the platform integrates a dedicated security microservice that performs attack vector detection, injection testing (SQL injection, command injection, XSS), WAF detection, and security prevention analysis. An embedded Docker sandbox provides a safe, isolated environment for testing exploits against deliberately vulnerable applications. The entire system is enriched by a constrained AI layer based on **Ollama** running the `qwen2.5-coder:7b` model, which generates Remediation-as-Code (Bash, Ansible, Dockerfile, Terraform) with anti-hallucination citations referencing the exact line numbers in the raw tool logs.

The project was conducted using an Agile methodology structured into iterative sprints, spanning the full software development lifecycle from requirements analysis and architecture design through implementation, testing, and deployment. The platform has been successfully deployed online and is accessible at `https://aymenazizi.dijaly.com/`.

This report is structured as follows :

— **General Presentation :** This chapter introduces the host organization, Designet Web Agency, explores the limitations of existing security testing approaches, and justifies the methodology selected for the development of our platform. It also defines the project goals, scope, and development environment.

— **Literature Review :** This section examines the fundamental concepts and technologies relevant to our project, including cybersecurity fundamentals, the OWASP Top 10, attack surface management, reconnaissance tools, DevSecOps principles, and the role of artificial intelligence in cybersecurity.

— **System Design and Architecture :** This chapter presents the requirements analysis, the microservices architecture design, the database schema, and the security architecture that underpins the platform.

— **Implementation :** This chapter details the technical implementation of each component—the Laravel backend, the five Python microservices (reconnaissance, security, OSINT, AI, API gateway), the event-driven worker, and the Docker-based deployment infrastructure. It also presents the online deployment, functional and security validation, performance evaluation, and compliance with the project's specifications (Cahier des Charges).

— **General Conclusion :** This final section summarizes the completed work, reflects on the skills and experience gained, and outlines future improvements and extensions.

Through this structure, we aim to demonstrate the value of combining automated reconnaissance, microservices architecture, and artificial intelligence in building a modern cybersecurity platform. The project also underlines the importance of DevSecOps practices in integrating security at every stage of the development lifecycle.

*Chapitre 1*

# General Presentation

## Outline

## Introduction

This chapter aims to present the general framework of our project. We begin by introducing the hosting organization and the context in which this study was carried out, namely the growing need for automated, AI-driven cybersecurity reconnaissance in modern web environments. Next, we examine existing approaches to security testing and vulnerability assessment, highlighting their limitations in terms of scalability, coverage, and reporting capabilities. Finally, we justify the methodological choices made to successfully implement a robust microservices-based platform, combining DevSecOps practices and cutting-edge open-source tools.

## 1.1 Presentation of the Organization

Designet Web Agency is a dynamic Tunisian company specializing in web development, digital design, and cybersecurity solutions. Based in Manzel Tmim, the agency provides a wide range of digital services to clients across various industries, including web application development, e-commerce platforms, corporate websites, and increasingly, security auditing and penetration testing services. The company's mission is to deliver high-quality, secure, and scalable digital products that meet the evolving needs of modern businesses while adhering to industry best practices and security standards.

The agency's expertise encompasses several key areas : front-end and back-end web development using modern frameworks, UI/UX design, cloud infrastructure management, and cybersecurity consulting. Over the years, Designet Web Agency has built a reputation for delivering reliable and innovative solutions, making it an ideal environment for conducting a final-year project focused on cybersecurity platform development. The company's commitment to security is reflected in its adoption of DevSecOps practices, integrating security checks and vulnerability assessments into every stage of the development lifecycle.

The internship was carried out under the supervision of Mr Ali DORBOZ, Chief Technical Officer, who provided technical guidance and mentorship throughout the project. The administrative aspects were managed by Mr Raed Ben Khlifa. The internship period spanned from February 2, 2026, to July 2, 2026, during which the entire platform was designed, developed, tested, and deployed.

## 1.2 Services

Designet Web Agency offers a comprehensive range of services designed to support digital transformation and technological innovation. These services include :

— **Web Development :** Design and development of custom web applications, corporate websites,

and e-commerce platforms using modern frameworks such as Laravel, React, and Vue.js.

— **UI/UX Design :** Creation of intuitive, user-centric interfaces and experiences that balance aesthetics with functionality.

— **Cybersecurity Services :** Security auditing, vulnerability assessment, and penetration testing to help clients identify and remediate security weaknesses in their digital assets.

— **Cloud Solutions :** Deployment and management of scalable cloud infrastructure, including containerization with Docker and orchestration with Kubernetes.

— **DevSecOps Consulting :** Integration of security practices into development pipelines, including automated vulnerability scanning, secret management, and compliance monitoring.

— **AI Integration :** Leveraging artificial intelligence and machine learning to enhance security analysis, automate threat detection, and generate actionable insights.

## 1.3 Problem Statement

In the rapidly evolving landscape of cybersecurity, organizations face an increasing number of threats targeting their web applications and exposed infrastructure. Attack surfaces are expanding due to the adoption of cloud services, microservices architectures, APIs, and third-party integrations. However, traditional security testing approaches suffer from several critical limitations that leave organizations vulnerable.

First, manual penetration testing is time-consuming, resource-intensive, and typically performed infrequently—often only annually or biannually. This creates large windows of exposure where newly discovered vulnerabilities remain unaddressed. Security teams must manually run multiple tools (Nmap for port scanning, Nuclei for vulnerability detection, Gobuster for directory discovery, etc.), each with its own output format, making result consolidation and analysis extremely cumbersome.

Second, the heterogeneity of security tools leads to operational inefficiencies. Each tool has distinct command-line interfaces, configuration requirements, and output formats. Security engineers spend significant time switching between tools, interpreting raw output, and manually correlating findings across different scans. This fragmented approach increases the risk of missing critical vulnerabilities and delays the delivery of actionable reports to stakeholders.

Third, the lack of centralized reporting and prioritization mechanisms means that vulnerability data is often presented in technical formats that are inaccessible to non-technical decision-makers. Executive summaries, risk scores, and remediation recommendations are typically generated manually, introducing delays and potential inconsistencies. Furthermore, the absence of AI-powered analysis means that false positives are not efficiently filtered, and remediation priorities are not automatically

assigned based on context and impact.

Finally, ensuring consistent security practices across development teams is challenging without integrated DevSecOps tooling. Development pipelines often lack automated security gates, and security testing is treated as an afterthought rather than an integral part of the development process. This results in vulnerabilities being discovered late in the development cycle, when remediation is more costly and time-consuming.

Addressing these challenges is crucial for the successful design and implementation of an automated, AI-assisted reconnaissance platform that not only streamlines security testing but also provides intelligent analysis, centralized reporting, and seamless integration into development workflows.

## 1.4 Study of the Existing System

The study of the existing system is a fundamental step in understanding the operational context and identifying the shortcomings of current security testing practices. It involves evaluating the current processes, tools, and workflows in use, with the aim of pinpointing inefficiencies and areas for improvement to better align with the evolving needs of modern cybersecurity operations.

### 1.4.1 Current Procedure

Following a thorough analysis of security testing methods within the organization—and several discussions with the technical team and the CTO—we observed that current practices rely heavily on manual processes and disparate, non-integrated tools. Security assessments are typically performed by running individual scanning tools from the command line, each producing output in its own format. Nmap is used separately for port scanning, Nuclei for vulnerability detection, Gobuster for content discovery, and so on. Results are then manually compiled into spreadsheets or text documents, with minimal automation or correlation between findings.

Secret management during testing is often handled through static configuration files or environment variables, and authorization documents for scan targets are stored separately without systematic linkage to the scan records. When reports are generated, they are created manually in word processors, lacking standardized templates, severity classifications, or AI-assisted analysis. Monitoring of ongoing scans is limited, and there is no centralized dashboard for tracking security posture over time.

This fragmented and manual approach introduces significant delays, increases the risk of misconfiguration, and hinders the organization's ability to respond quickly to emerging threats or to provide clients with timely, actionable security reports.

### 1.4.2 Critique of the Existing System

From our analysis of the current security testing setup and management practices, several limitations have been identified :

— **Time-consuming operations :** The absence of automated tool orchestration leads to repetitive manual tasks and delayed assessments, especially when scanning multiple targets or running comprehensive scan profiles.

— **Lack of result normalization :** Each tool produces output in its own format, making it difficult to consolidate findings, deduplicate results, or maintain a unified vulnerability database.

— **Poor reporting capabilities :** Reports are generated manually, lack standardized templates, and do not include AI-assisted executive summaries or prioritized remediation recommendations.

— **No real-time monitoring :** There is no centralized dashboard for tracking active scans, viewing security alerts, or monitoring the overall security posture of scanned targets.

— **Limited DevSecOps integration :** Security testing is not integrated into development pipelines, and there are no automated security gates or CI/CD checks to catch vulnerabilities early in the development lifecycle.

— **Absence of sandbox testing :** There is no isolated environment for safely testing exploits or validating vulnerability findings against deliberately vulnerable applications.

### 1.4.3 Proposed Solution

To address the identified limitations and improve security testing, reporting, and monitoring, we propose the design and implementation of an **automated, AI-assisted web platform for attack surface reconnaissance** with integrated DevSecOps capabilities. This platform aims to streamline security operations, enhance vulnerability detection, and provide intelligent analysis through a microservices architecture.

The platform will offer the following capabilities :

— **Automated tool orchestration :** Leveraging a dedicated reconnaissance microservice, the platform automates the execution of industry-standard tools—Nmap, Nuclei, Gobuster, Subfinder, and WPScan—through a unified API, enabling reproducible scans and faster time-to-results.

— **Centralized result normalization :** A result parser converts each tool's output into a unified Finding format, enabling deduplication, correlation, and consistent severity classification across all scan types.

— **AI-assisted analysis and reporting :** Using Ollama, an open-source large language model, the platform analyzes scan results, generates executive summaries, provides remediation recommendations, and helps identify potential false positives.

— **Security testing microservice :** A dedicated microservice performs attack vector detection, injection testing (SQL, command, XSS), WAF detection, and prevention analysis, extending the platform beyond passive reconnaissance.

— **Real-time monitoring and alerting :** A centralized dashboard provides KPIs, severity breakdowns, recent alerts, and scan history, enabling security teams to maintain continuous visibility into their security posture.

— **Docker-based sandbox :** An isolated sandbox environment allows safe testing of exploits against deliberately vulnerable applications (DVWA, SQLi-Labs, WebGoat, bWAPP) without risking production systems.

— **DevSecOps integration :** The platform is designed to integrate into CI/CD pipelines, with API endpoints for automated scanning, rate limiting, and role-based access control (RBAC) for multi-team environments.

This solution aligns with modern DevSecOps practices and responds directly to the need for operational efficiency, intelligent analysis, and comprehensive security coverage in an increasingly complex threat landscape.

## 1.5 Methodology and Approach

### 1.5.1 Choice of Methodology

A work methodology is a structured and organized framework used to execute tasks efficiently and achieve project objectives. It encompasses defined processes, tools, and best practices to ensure quality, clarity, and alignment with stakeholder expectations.

For this project, the **Agile methodology** was selected due to its iterative, flexible, and adaptive nature. Agile emphasizes collaboration, incremental development, and continuous feedback, making it particularly well-suited to dynamic and evolving project environments like cybersecurity platform development. The Agile approach allowed us to adapt to changing requirements, incorporate feedback from the technical team at Designet Web Agency, and deliver functional increments of the platform throughout the development cycle.

### 1.5.2 SCRUM Methodology

Within the Agile framework, we adopted the **SCRUM methodology**, one of the most widely used Agile techniques. SCRUM is based on time-boxed iterations called *sprints*, during which a cross-functional team works to deliver potentially shippable product increments. It fosters self-organization, accountability, and a high level of transparency.

The SCRUM process involves key roles such as the *Product Owner* (in this case, the CTO at Designet Web Agency), the *Scrum Master* (the intern), and the *Development Team*. The process is structured around key ceremonies : *Sprint Planning*, *Daily Stand-ups*, *Sprint Reviews*, and *Retrospectives*. This approach ensures rapid adaptation to changing requirements and encourages continuous delivery of valuable features.

The project was organized into the following sprints :

— **Sprint 1 (Feb 2 – Feb 22) :** Requirements analysis, architecture design, and project scaffolding (Laravel backend + Flask microservices).

— **Sprint 2 (Feb 23 – Mar 15) :** Development of the reconnaissance microservice (Nmap, Nuclei, Gobuster, Subfinder, WPScan integration).

— **Sprint 3 (Mar 16 – Apr 5) :** Development of the security microservice (attack detection, injection testing, prevention engine).

— **Sprint 4 (Apr 6 – Apr 26) :** AI integration (Ollama), result normalization, and report generation.

— **Sprint 5 (Apr 27 – May 17) :** Frontend development (Blade templates, dashboard, alerts, monitoring, sandbox).

— **Sprint 6 (May 18 – Jun 1) :** Docker deployment, API gateway, testing, and online deployment.

**Table 1.1 :** Sprint Planning and Task Distribution

| Sprint | Duration | Key Deliverables | Sprint Review |
|---|---|---|---|
| Sprint 1 | 3 weeks | Requirements document, architecture diagrams, Laravel + Flask scaffolding, database migrations | Validated architecture with CTO |
| Sprint 2 | 3 weeks | Reconnaissance microservice with 5 tools, BaseScannerService, ResultParser, unified Finding model | Tool execution demo (Nmap + Nuclei) |
| Sprint 3 | 3 weeks | Security microservice with attack detection, injection testing, prevention engine, sandbox infrastructure | Injection testing demo on DVWA |
| Sprint 4 | 3 weeks | Ollama integration, AI analysis pipeline, report generation with executive summaries | AI analysis output review |
| Sprint 5 | 3 weeks | Blade frontend (dashboard, projects, scans, reports, alerts, monitoring, sandbox pages) | UI walkthrough with CTO |
| Sprint 6 | 2 weeks | API gateway, Docker Compose, Nginx TLS, production deployment at aymenazizi.dijaly.com | Final platform demo |

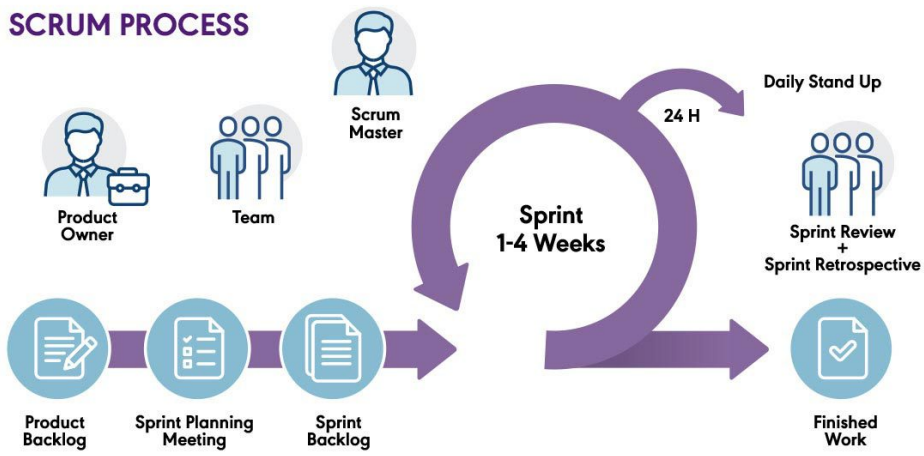



**Figure 1.1 :** Overview of the SCRUM Process

## Conclusion

In this chapter, we introduced the hosting organization, Designet Web Agency, and outlined the context and motivations behind the project. Through a detailed analysis of the current security testing practices, we identified significant limitations, including the lack of automation, inconsistent result formats, poor reporting capabilities, and the absence of AI-assisted analysis in cybersecurity

operations. These observations guided the formulation of a proposed solution aimed at building an automated, AI-driven, and DevSecOps-integrated reconnaissance platform.

We also presented our methodological approach, justifying the adoption of Agile and SCRUM frameworks to manage the project efficiently and adapt to evolving requirements. The sprint-based development cycle ensured continuous delivery and allowed for iterative refinement of the platform's capabilities.

To ensure a coherent progression of this report, the next chapter will be dedicated to a literature review of the core concepts and technologies used in the project, including cybersecurity fundamentals, the OWASP Top 10, reconnaissance tools, and the role of artificial intelligence in security analysis.

*Chapitre 2*

# Literature Review

## Outline

## Introduction

This chapter examines the fundamental concepts and technologies relevant to our project. It begins with an overview of cybersecurity fundamentals and the growing importance of attack surface management in modern organizations. We then explore the OWASP Top 10 vulnerabilities and their impact on web application security. Next, we delve into the reconnaissance tools used in the platform—Nmap, Nuclei, Gobuster, Subfinder, and WPScan—and discuss their roles in automated security assessment. We also cover DevSecOps principles, microservices architecture, and the application of artificial intelligence in cybersecurity, particularly through large language models like Ollama. This theoretical foundation establishes the groundwork for the design and implementation phases discussed in subsequent chapters.

## 2.1 Cybersecurity Fundamentals

### 2.1.1 Definition and Importance

Cybersecurity refers to the practice of protecting systems, networks, and programs from digital attacks. These cyberattacks are typically aimed at accessing, changing, or destroying sensitive information, extorting money from users, or interrupting normal business processes. According to the International Telecommunication Union (ITU), cybersecurity encompasses the collection of tools, policies, security concepts, security safeguards, guidelines, risk management approaches, actions, training, best practices, assurance, and technologies that can be used to protect the cyber environment and organizational and user's assets.

In today's hyper-connected world, the importance of cybersecurity cannot be overstated. Organizations across all sectors rely on digital infrastructure for their operations, making them potential targets for a wide range of cyber threats. The cost of cybercrime is projected to reach $10.5 trillion annually by 2025, according to Cybersecurity Ventures, highlighting the urgent need for robust security measures and automated defense mechanisms.

### 2.1.2 Key Cybersecurity Concepts

Several fundamental concepts underpin the field of cybersecurity :

— **Confidentiality :** Ensuring that information is accessible only to authorized individuals, achieved through encryption, access controls, and secure communication protocols.

— **Integrity :** Maintaining the accuracy and completeness of data, preventing unauthorized modification or corruption through hashing, digital signatures, and version control.

— **Availability :** Ensuring that systems and data are accessible when needed, achieved through redundancy, failover mechanisms, and denial-of-service protections.

— **Authentication :** Verifying the identity of users, devices, or systems before granting access, typically through passwords, biometrics, multi-factor authentication (MFA), or certificates.

— **Authorization :** Determining what actions an authenticated user or system is permitted to perform, implemented through role-based access control (RBAC) or attribute-based access control (ABAC).

— **Non-repudiation :** Ensuring that a party cannot deny the authenticity of their signature on a document or a message they sent, achieved through digital signatures and audit logs.

## 2.2 OWASP Top 10

The Open Worldwide Application Security Project (OWASP) is a nonprofit foundation that works to improve the security of software. OWASP operates as an open community and provides freely available articles, methodologies, documentation, tools, and technologies in the field of web application security.

The OWASP Top 10 is a standard awareness document for developers and web application security. It represents a broad consensus about the most critical security risks to web applications. The latest version, OWASP Top 10 :2021, includes the following categories :

— **A01 :2021 – Broken Access Control :** Restrictions on what authenticated users are allowed to do are often not properly enforced, allowing attackers to access unauthorized functionality or data.

— **A02 :2021 – Cryptographic Failures :** Failures related to cryptography, including weak encryption algorithms, hard-coded passwords, and insufficient protection of sensitive data.

— **A03 :2021 – Injection :** Security vulnerabilities that allow attackers to inject malicious data into applications, including SQL injection, command injection, and LDAP injection.

— **A04 :2021 – Insecure Design :** Flaws in the design and architecture of applications that cannot be fixed through implementation alone.

— **A05 :2021 – Security Misconfiguration :** Incorrect implementation of controls intended to protect the application, including default configurations, open cloud storage, and verbose error messages.

— **A06 :2021 – Vulnerable and Outdated Components :** Using software components with known vulnerabilities, which attackers can exploit to compromise the application.

— **A07 :2021 – Identification and Authentication Failures :** Weaknesses in authentication and session management, including credential stuffing, weak passwords, and session fixation.

— **A08 :2021 – Software and Data Integrity Failures :** Failures related to code and data integrity, including insecure CI/CD pipelines and unsigned software updates.

— **A09 :2021 – Security Logging and Monitoring Failures :** Insufficient logging and monitoring, preventing timely detection of attacks and incident response.

— **A10 :2021 – Server-Side Request Forgery (SSRF) :** Vulnerabilities that allow attackers to coerce the server to make requests to unintended destinations.

Our platform specifically addresses several of these OWASP categories through its injection testing module (A03), access control mechanisms (A01), security misconfiguration detection (A05), and logging/monitoring capabilities (A09).

## 2.3 Attack Surface Management

### 2.3.1 Definition

An attack surface is the sum of all points where an unauthorized user can attempt to enter data into or extract data from an environment. It encompasses all hardware, software, network connections, and human elements that could be exploited by a threat actor. Attack Surface Management (ASM) is the continuous process of discovering, inventorying, classifying, and monitoring these attack surfaces to identify and remediate potential security risks.

### 2.3.2 Types of Attack Surfaces

Attack surfaces are typically categorized into three main types :

— **Digital Attack Surface :** Encompasses all internet-facing assets, including web applications, APIs, cloud services, open ports, and third-party integrations. This is the primary focus of our reconnaissance platform.

— **Physical Attack Surface :** Includes physical access points to infrastructure, such as server rooms, network devices, endpoints, and removable media. While not the focus of our platform, physical security remains an important consideration in overall cybersecurity posture.

— **Social Engineering Attack Surface :** Refers to the human element, including employees, contractors, and partners who may be manipulated through phishing, pretexting, or other social engineering techniques.

### 2.3.3 Attack Surface Management Process

Effective ASM follows a structured process :

1. **Discovery :** Identifying all assets belonging to the organization, including shadow IT and forgotten infrastructure.
2. **Inventory :** Creating a comprehensive, up-to-date inventory of all discovered assets, their configurations, and their relationships.
3. **Classification :** Categorizing assets by type, criticality, exposure, and ownership to prioritize security efforts.
4. **Monitoring :** Continuously monitoring the attack surface for changes, new vulnerabilities, and emerging threats.
5. **Remediation :** Taking action to address identified vulnerabilities, misconfigurations, and security gaps.

Our platform automates the discovery and monitoring phases by orchestrating multiple reconnaissance tools and providing a unified dashboard for tracking the attack surface over time.

## 2.4 Reconnaissance Tools

Reconnaissance is the first phase of any security assessment, involving the gathering of information about a target system or network. Our platform integrates five industry-standard reconnaissance tools, each serving a specific purpose in the security assessment workflow.

### 2.4.1 Nmap

Nmap (Network Mapper) is a free and open-source network scanner created by Gordon Lyon. It is used to discover hosts and services on a computer network by sending packets and analyzing the responses. Nmap provides a wide range of features, including port scanning, service detection, operating system fingerprinting, and scriptable interaction with the target using the Nmap Scripting Engine (NSE).

In our platform, Nmap is used to perform fast port scans (`-F` flag), service version detection (`-sV`), and comprehensive port scans with script execution (`-sC`) for aggressive scan profiles. The tool is configured with three intensity levels : Silent (top 100 ports, `-T2` timing), Balanced (fast scan with service detection, `-T3` timing), and Aggressive (all ports with service and script scanning, `-T4` timing).

### 2.4.2 Nuclei

Nuclei is a fast, customizable, template-based vulnerability scanner developed by ProjectDiscovery. It uses YAML templates to define vulnerability checks, allowing the community to continuously add new detection patterns. Nuclei can detect CVEs, misconfigurations, exposed panels, and a wide range of other security issues across web applications and network services.

In our platform, Nuclei is configured to output results in JSON Lines format (`-jsonl`), enabling structured parsing and normalization of findings. The scan intensity controls which template categories are used : Silent mode focuses on CVEs and known vulnerabilities, while Aggressive mode scans all templates across all severity levels.

### 2.4.3 Gobuster

Gobuster is a tool used to brute-force URIs (directories and files) in websites, DNS subdomains, virtual host names on target web servers, and open Amazon S3 buckets. Written in Go, it is extremely fast and supports concurrent requests, making it ideal for discovering hidden content on web servers.

Our platform configures Gobuster with the SecLists common.txt wordlist for directory enumeration. In Aggressive mode, additional file extensions (PHP, HTML, JSON, TXT, BAK, OLD) and increased thread counts are used for more comprehensive coverage.

### 2.4.4 Subfinder

Subfinder is a subdomain discovery tool that discovers valid subdomains for websites by using passive online sources. It is designed to be a fast, passive alternative to active subdomain brute-forcing, aggregating results from over 30 data sources without sending any requests to the target directly.

In our platform, Subfinder is used to enumerate subdomains for a given domain. The Aggressive intensity level enables the `-all` flag, which queries all available sources for maximum coverage.

### 2.4.5 WPScan

WPScan is a free, open-source WordPress security scanner designed to find security weaknesses in WordPress installations. It can detect WordPress version, installed themes and plugins, vulnerable components, and weak passwords through brute-force attacks.

Our platform uses WPScan with JSON output format for structured parsing. The tool enumerates vulnerable plugins, vulnerable themes, timthumbs, config backups, database exports, users, and media. An optional WPScan API token can be configured for enhanced vulnerability database access.

## 2.5 DevSecOps

### 2.5.1 Definition and Principles

DevSecOps is the philosophy of integrating security practices within the DevOps process. It involves creating a "security as code" culture with ongoing, flexible collaboration between release engineers and security teams. The core principle of DevSecOps is that security should be an integral part of the development process, not an afterthought or a separate phase.

The key principles of DevSecOps include :

— **Shift-Left Security :** Integrating security checks early in the development lifecycle, rather than waiting for the testing or production phases.

— **Automation :** Automating security testing, vulnerability scanning, and compliance checks to ensure consistency and reduce human error.

— **Continuous Monitoring :** Maintaining continuous visibility into the security posture of applications and infrastructure throughout the development lifecycle.

— **Collaboration :** Fostering collaboration between development, security, and operations teams to break down silos and share responsibility for security.

— **Security as Code :** Defining security policies, configurations, and tests as code, enabling version control, review, and automation.

### 2.5.2 DevSecOps Pipeline Stages

A typical DevSecOps pipeline includes the following stages :

1. **Pre-commit (Development) :** Linting, rapid tests, and secret scanning on the developer's machine before code is committed.
2. **CI (Merge/Pull Request) :** Static Application Security Testing (SAST), Software Composition Analysis (SCA), Software Bill of Materials (SBOM) generation, Docker image scanning, and Infrastructure as Code (IaC) manifest scanning.
3. **CD (Staging) :** Deployment to staging environment, Policy-as-Code enforcement, and Dynamic Application Security Testing (DAST) on the platform itself.
4. **Release :** Signed artifacts with traceability, SBOM attachment, and security scan reports as CI artifacts.

Our platform supports DevSecOps integration through its API gateway, which enables automated scanning from CI/CD pipelines. The security microservice provides endpoints for injection testing,

WAF detection, and attack surface analysis that can be called programmatically during the build process.

## 2.6 Microservices Architecture

### 2.6.1 Definition

Microservices architecture is an architectural style that structures an application as a collection of small, autonomous services modeled around a business domain. Each service is self-contained, independently deployable, and communicates with other services through lightweight protocols, typically HTTP/REST or message queues.

Unlike monolithic architectures, where all functionality is packaged into a single deployable unit, microservices allow individual components to be developed, deployed, and scaled independently. This approach offers several advantages in the context of a cybersecurity platform :

— **Scalability :** Each microservice can be scaled independently based on demand. For example, the reconnaissance microservice can be scaled up during periods of heavy scanning activity without affecting the backend or security microservice.

— **Technology Diversity :** Different services can use different technology stacks. Our platform uses PHP/Laravel for the backend, Python/Flask for the microservices, and can easily integrate additional services in other languages.

— **Fault Isolation :** A failure in one service does not bring down the entire platform. If the security microservice is temporarily unavailable, the reconnaissance microservice can continue to function.

— **Maintainability :** Smaller, focused codebases are easier to understand, test, and maintain. Changes to one service do not require rebuilding and redeploying the entire application.

### 2.6.2 Communication Patterns

In a microservices architecture, services communicate through well-defined APIs. Our platform uses two primary communication patterns :

— **Synchronous HTTP/REST :** The Laravel backend communicates with the microservices via HTTP POST requests, sending scan parameters and receiving results. This pattern is used for real-time scan execution where immediate feedback is required.

— **API Gateway :** An API gateway acts as a single entry point for all client requests, routing them to the appropriate microservice. This pattern provides rate limiting, request validation,

and centralized health monitoring.

## 2.7 Artificial Intelligence in Cybersecurity

### 2.7.1 The Role of AI in Security

Artificial Intelligence (AI) and Machine Learning (ML) have become increasingly important in cybersecurity, offering capabilities that traditional rule-based systems cannot match. AI can analyze vast amounts of data, identify patterns, detect anomalies, and generate human-readable insights at a scale and speed that would be impossible for human analysts alone.

Key applications of AI in cybersecurity include :

— **Threat Detection :** Machine learning models can identify malicious patterns in network traffic, system logs, and user behavior that may indicate a cyberattack.

— **Vulnerability Analysis :** AI can prioritize vulnerabilities based on exploitability, impact, and context, helping security teams focus on the most critical issues.

— **Automated Response :** AI-powered systems can automatically respond to certain types of threats, such as blocking malicious IP addresses or isolating compromised systems.

— **Report Generation :** Large language models can generate executive summaries, technical reports, and remediation recommendations from raw scan data.

### 2.7.2 Large Language Models and Ollama

Large Language Models (LLMs) are AI models trained on vast amounts of text data, enabling them to understand and generate human-like text. In recent years, LLMs have been increasingly applied to cybersecurity tasks, including code analysis, vulnerability description, and security report generation.

**Ollama** is an open-source tool that allows running large language models locally, without requiring cloud services or API keys. It supports various models, including Llama, Qwen, Mistral, and others. The key advantages of using Ollama in a cybersecurity platform include :

— **Privacy :** All inference is performed locally ; no sensitive scan data is sent to external servers.

— **Cost :** No per-token API charges, making it cost-effective for high-volume scanning.

— **Latency :** Local inference eliminates network latency, enabling near-instant analysis.

— **Customization :** Models can be fine-tuned or selected based on specific use cases and performance requirements.

In our platform, Ollama is used with the `qwen2.5-coder:7b` model to analyze scan results from all tools. The AI generates structured output with four sections : Risk Summary, Key Findings, Remediation-as-Code scripts (Bash, Ansible, Dockerfile, Terraform), and Citations referencing the exact line numbers in the raw logs. This constrained, schema-validated format ensures consistent, traceable, and audit-ready analysis across all scan types.

## 2.8 Docker and Containerization

Docker is a platform that uses containerization technology to package applications and their dependencies into standardized, lightweight containers. Containers are isolated from each other and from the host system, ensuring consistent behavior across different environments.

In our platform, Docker serves two critical purposes :

— **Application Deployment :** Each component of the platform—the Laravel backend, the five Python microservices (reconnaissance, security, OSINT, AI, API gateway), the event-driven worker, PostgreSQL 16 with Apache AGE, Redis 7, and Ollama—is packaged as a Docker container. Docker Compose orchestrates the deployment of all twelve services, managing networking, volumes, and dependencies across two isolated Docker networks.

— **Security Sandbox :** The security microservice uses Docker to create isolated sandbox environments for testing exploits against deliberately vulnerable applications (DVWA, SQLi-Labs, WebGoat, bWAPP). This ensures that exploit testing does not affect production systems or the host environment.

## 2.9 Comparative Study and Technology Justification

This section presents a systematic comparative analysis of the technologies and tools selected for the platform. For each category, we evaluate multiple candidate solutions against relevant criteria, then justify the final choice. This comparative approach ensures that the retained technologies are not arbitrary but the result of an informed, criteria-based selection process aligned with the project's requirements.

### 2.9.1 Comparison of Port Scanning Tools

Port scanning is a fundamental step in network reconnaissance. Several tools exist with varying capabilities in terms of speed, accuracy, scriptability, and output format. Table 2.1 compares four widely used port scanning tools across key criteria relevant to our platform's architecture.

**Table 2.1 :** Comparative Analysis of Port Scanning Tools

| **Criterion** | **Nmap** | **Masscan** | **RustScan** | **Angry IP Scanner** |
|---|---|---|---|---|
| Scan Speed | Moderate | Very fast (massive) | Fast (Rust-based) | Moderate |
| Scripting Engine | NSE (extensive) | None | Basic wrappers | None |
| Service Detection | Yes (-sV) | No | Via Nmap integration | Basic |
| OS Fingerprinting | Yes (-O) | No | No | No |
| Output Formats | XML, JSON, grepable, normal | JSON, list | JSON, text | CSV, TXT |
| Python Integration | subprocess/scapy | subprocess | subprocess/API | subprocess |
| Community Support | Excellent | Good | Growing | Limited |
| License | GPL (open source) | GPL (open source) | GPL (open source) | GPL (open source) |
| **Selected** | **Yes** | No | No | No |

**Justification :** Nmap was selected over alternatives primarily for its Nmap Scripting Engine (NSE), which enables deep service detection and vulnerability probing beyond simple port discovery. Masscan, while faster, lacks service detection and scripting capabilities, making it unsuitable for the detailed analysis our platform requires. RustScan, though innovative for its speed, essentially wraps Nmap and adds complexity without significant benefit for our use case. Nmap's mature Python integration via subprocess, combined with its extensive community-maintained script library, makes it the most reliable and feature-rich choice for the reconnaissance microservice.

### 2.9.2 Comparison of Vulnerability Scanners

Vulnerability scanners automate the detection of security weaknesses in target systems. Table 2.2 compares template-based and signature-based vulnerability scanning tools.

**Table 2.2 :** Comparative Analysis of Vulnerability Scanners

| **Criterion** | **Nuclei** | **Nessus** | **OpenVAS** | **OWASP ZAP** |
|---|---|---|---|---|
| Scanning Approach | Template-based | Signature-based | Signature-based | Active/passive proxy |
| Custom Templates | YAML (community-driven) | Plugins (closed) | NVTs (open) | Scripts (JSON) |
| Scan Speed | Very fast | Moderate | Slow | Moderate |
| CI/CD Integration | Excellent (CLI-first) | Limited (GUI-heavy) | Moderate | Good |
| Output Formats | JSONL, JSON, Markdown | HTML, PDF, CSV | HTML, XML, JSON | HTML, JSON |
| Self-Hosted | Yes (lightweight) | Requires license | Yes (heavy) | Yes (Java) |
| Cost | Free | Commercial | Free | Free |
| Microservice Fit | Excellent (no GUI needed) | Poor (requires desktop/server) | Poor (resource-heavy) | Moderate (headless mode) |
| **Selected** | **Yes** | No | No | No |

**Justification :** Nuclei was selected for its CLI-first design, which integrates naturally into a microservices architecture. Unlike Nessus and OpenVAS, which require server-side installations and heavy resource allocation, Nuclei operates as a lightweight binary with YAML-based templates.

Its JSONL output format enables structured parsing and normalization within the reconnaissance microservice. The large and active community continuously contributes new templates, ensuring coverage of the latest CVEs and vulnerabilities. OWASP ZAP, while powerful, is primarily designed as a proxy-based scanner for web applications and would require significant adaptation to fit our multi-tool orchestration model.

### 2.9.3 Comparison of Directory Enumeration Tools

For directory and file discovery, four candidates were considered : **Gobuster**, **Feroxbuster**, **Dirb**, and **ffuf**. **Gobuster** was selected for its simplicity, its Go-based compiled performance, and its clean CLI interface, which maps directly to the `BaseScannerService` abstraction in the reconnaissance microservice. Feroxbuster offers built-in recursive scanning, but our platform implements its own result normalization and correlation logic, making built-in recursion redundant. Dirb was excluded due to its slow, single-threaded nature. Ffuf, though capable, introduced additional complexity without providing significant advantages for our specific use case.

### 2.9.4 Comparison of Subdomain Discovery Tools

For passive subdomain enumeration, **Subfinder** was selected over **Amass**, **Assetfinder**, and **Findomain**. Subfinder's purely passive approach (querying more than 30 public data sources, including `crt.sh`, SecurityTrails, and Shodan) aligns with the reconnaissance phase's requirement to minimize direct interaction with the target. Amass, while more comprehensive with its active scanning capabilities, introduces risk of detection and requires more complex configuration. Subfinder's clean JSON output integrates seamlessly with the result parser, and its single-binary deployment simplifies containerization.

### 2.9.5 Comparison of Backend Web Frameworks

The choice of backend framework significantly impacts development velocity, maintainability, and ecosystem integration. Table 2.3 compares four popular web frameworks for the main backend application.

**Table 2.3 :** Comparative Analysis of Backend Web Frameworks

| **Criterion** | **Laravel (PHP)** | **Django (Python)** | **Spring Boot (Java)** | **Express.js (Node)** |
|---|---|---|---|---|
| Language | PHP 8.3 | Python 3.11 | Java 17+ | JavaScript/TS |
| MVC Support | Excellent (built-in) | Excellent (built-in) | Good (Spring MVC) | Manual (flexible) |
| ORM | Eloquent (mature) | Django ORM | Hibernate/JPA | Prisma/Sequelize |
| Auth System | Built-in (Sanctum, Passport) | Built-in (sessions, JWT) | Spring Security | Manual (Passport.js) |
| RBAC Packages | Spatie Permission | Django Guardian | Spring Security ACL | CASL |
| Template Engine | Blade (powerful) | Django Templates | Thymeleaf | EJS/Pug/Handlebars |
| Queue System | Built-in (Redis, DB) | Celery (external) | Spring Batch | Bull/Bee-Queue |
| Microservice Comm. | HTTP client, Guzzle | Requests, httpx | RestTemplate, Feign | Axios, fetch |
| Docker Support | Excellent | Good | Good | Excellent |
| Company Expertise | High (Designet stack) | Moderate | Low | Moderate |
| **Selected** | **Yes** | No | No | No |

**Justification :** Laravel was selected as the primary backend framework for several compelling reasons. First, it provides a mature, built-in authentication system (Sanctum) and role-based access control (Spatie Laravel-Permission), which are critical for the platform's four-role RBAC model. Second, its Eloquent ORM simplifies database operations across eight interconnected tables. Third, the built-in queue system with Redis integration enables asynchronous scan task processing. Finally, and importantly, Laravel aligns with Designet Web Agency's existing technology stack, ensuring knowledge transfer and long-term maintainability by the host company's team. Django was considered as a strong alternative but was ultimately excluded because the microservices are already implemented in Python (Flask), and using Laravel for the backend provides technology diversity across the architecture.

### 2.9.6 Comparison of Microservice Frameworks

The reconnaissance and security microservices require a lightweight, fast-to-develop framework. Table 2.4 compares four Python frameworks suitable for microservice development.

**Table 2.4 :** Comparative Analysis of Microservice Frameworks

| **Criterion** | **Flask** | **FastAPI** | **Django REST** | **Nameko** |
|---|---|---|---|---|
| Lightweight | Very (minimal core) | Light (with Pydantic) | Heavy (batteries included) | Moderate |
| API Documentation | Manual (Swagger ext.) | Auto (OpenAPI/Swagger) | Auto (Swagger) | Manual |
| Subprocess Mgmt. | subprocess (full control) | subprocess (async) | subprocess (sync) | Worker-based |
| Tool Integration | Easy (flexible) | Easy (type-safe) | Moderate (framework constraints) | Moderate |
| Docker Image Size | Very small ( 100MB) | Small ( 120MB) | Large ( 300MB) | Moderate ( 200MB) |
| Learning Curve | Low | Moderate | High | High |
| Community | Large | Growing fast | Very large | Small |
| Async Support | Via extensions | Native (async/await) | Via Celery | Built-in |
| **Selected** | **Yes** | No | No | No |

**Justification :** Flask was selected for the microservices due to its minimal footprint and maximum flexibility. The reconnaissance and security microservices need to execute external command-line tools (Nmap, Nuclei, Gobuster, etc.) via subprocess calls, and Flask's lack of framework constraints provides full control over process management, timeout handling, and output parsing. FastAPI was a strong contender, particularly for its automatic OpenAPI documentation, but its async model introduces complexity when wrapping synchronous subprocess calls. Django REST Framework was excluded as too heavyweight for simple API endpoints that primarily orchestrate external tools. Nameko, while designed for microservices, adds unnecessary abstraction for our use case where services communicate via simple HTTP calls.

### 2.9.7 Comparison of Local AI/LLM Solutions

The platform requires a local AI inference engine to analyze scan results without sending sensitive security data to external cloud services. Table 2.5 compares four local LLM deployment solutions.

**Table 2.5 :** Comparative Analysis of Local AI/LLM Solutions

| **Criterion** | **Ollama** | **LM Studio** | **LocalAI** | **vLLM** |
|---|---|---|---|---|
| Open Source | Yes (MIT) | Yes (MIT) | Yes (MIT) | Yes (Apache 2.0) |
| CLI/API | REST API (port 11434) | GUI + API | REST API | OpenAI-compatible API |
| Model Support | Llama, Qwen, Mistral, Phi, Gemma | GGUF models | GGUF, GPTQ, AWQ | Wide range |
| Docker Integration | Official image | Desktop app | Official image | Official image |
| Resource Usage | Moderate (efficient) | High (GUI overhead) | Moderate | High (GPU-optimized) |
| Python Client | ollama Python lib | API calls | API calls | OpenAI SDK |
| Headless Operation | Yes (server mode) | Limited | Yes | Yes |
| Model Management | Built-in (pull, run) | Manual | Manual | Manual |
| Setup Simplicity | One command install | Download app | Docker compose | Complex (CUDA deps) |
| **Selected** | **Yes** | No | No | No |

**Justification :** Ollama was selected as the AI inference engine for its simplicity, privacy, and Docker compatibility. The primary requirement is to run a constrained LLM locally within the Docker Compose stack without exposing scan data to external APIs. Ollama's one-command installation and built-in model management (`ollama pull qwen2.5-coder:7b`) make it ideal for containerized deployment. LM Studio was excluded because it is primarily a desktop application with limited headless/server capabilities. LocalAI and vLLM, while capable, require more complex configuration and offer no significant advantage for our use case of generating structured Remediation-as-Code from scan results. The `qwen2.5-coder:7b` model was selected because it is specifically trained on code, supports structured JSON output, is large enough to produce reliable Bash/Ansible/Dockerfile/Terraform

remediation scripts, and still fits within the GPU memory budget of a single mid-range deployment (approximately 4.7 GB quantized). A strict JSON schema enforces four sections : Risk Summary, Key Findings, Remediation-as-Code scripts, and Citations referencing the exact line numbers in the raw tool logs (preventing hallucinations).

### 2.9.8 Selection of Database, CI/CD, Orchestration, Monitoring, and Graph Engine

The remaining infrastructure-layer technology choices are summarized briefly below. For each category, multiple candidates were evaluated against the project's requirements (local deployment, on-premises data residency, single-node footprint, Laravel compatibility), and the selected solution emerged as the natural fit.

**Database – PostgreSQL 16** was selected over MySQL 8.0, MariaDB, and SQLite for two decisive reasons : (1) it natively supports the **Apache AGE** graph extension, which allows the platform to model the attack surface as a directed graph alongside standard SQL, and (2) its **JSONB** column type with GIN indexes provides superior querying of heterogeneous scan results and AI analysis output. SQLite was excluded due to its single-writer limitation ; MySQL/MariaDB lack a graph extension comparable to AGE.

**CI/CD Platform – GitHub Actions** was selected over GitLab CI, Jenkins, and CircleCI for its tight integration with the project's Git repository, its native support for container-based jobs, and its comprehensive catalog of security-focused Actions (Trivy, Syft, Cosign). The free tier for private repositories (2,000 minutes per month) comfortably covers the platform's CI workload. Jenkins, while extremely powerful, requires infrastructure overhead disproportionate for a single-team project.

**Container Orchestrator – Docker Compose** was selected over Kubernetes, Nomad, and Docker Swarm because the deployment target is a single hardened host and Compose's declarative YAML model directly mirrors the twelve-service architecture. Kubernetes was rejected : its control-plane overhead (∼2 GB of RAM) is unjustifiable for a single-team platform, and its RBAC/NetworkPolicy abstractions would duplicate the role-based access control already implemented at the Laravel layer.

**Monitoring – Prometheus + Grafana** were selected over Datadog, the ELK stack, and Zabbix because they are the de-facto open-source standard for cloud-native monitoring and they keep all telemetry data on the customer's premises (a non-negotiable requirement for security engagements handling confidential client data). Datadog was rejected because it ships telemetry to a vendor cloud, violating the PoA data-residency clauses typically signed with pentest clients.

**Graph Database – Apache AGE** was selected over Neo4j, ArangoDB, TigerGraph, and Amazon Neptune because it is a PostgreSQL extension, which means the platform benefits from a single

relational+graph store with one set of credentials, one backup strategy, and native Laravel/Eloquent compatibility. AGE implements openCypher (the same query language as Neo4j), so graph queries are portable. Neo4j was rejected because it would require a second database server alongside PostgreSQL, doubling the operational surface.

### 2.9.9 DevSecOps Pipeline Stages Diagram

The DevSecOps pipeline implemented on the platform follows the seven-stage flow illustrated in Figure 2.1, from source commit through to a signed, deployed container image.

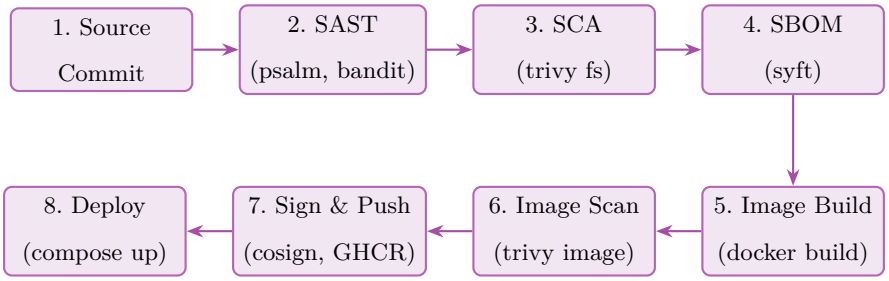



**Figure 2.1 :** DevSecOps Pipeline Stages – Eight-Stage Flow from Commit to Signed Deployment

Each stage gates the next : a critical SAST finding blocks the SCA step, an unsanitised dependency blocks SBOM generation, and an unscanned or unsigned image can never be deployed by `docker compose up`. The pipeline runs on every pull request and on every merge to `main`, with the signed image pushed to GitHub Container Registry (GHCR) using a Cosign keyless identity bound to the GitHub OIDC token. This pipeline is end-to-end evidenced in Chapter 4 (subsection 4.8, Platform Deployment and Online Validation).

### 2.9.10 Microservices Communication Patterns

Figure 2.2 summarises the three communication patterns used between the platform's microservices : synchronous REST for user-facing requests, asynchronous Redis Streams for long-running scans, and the PostgreSQL LISTEN/NOTIFY channel for cache invalidation.

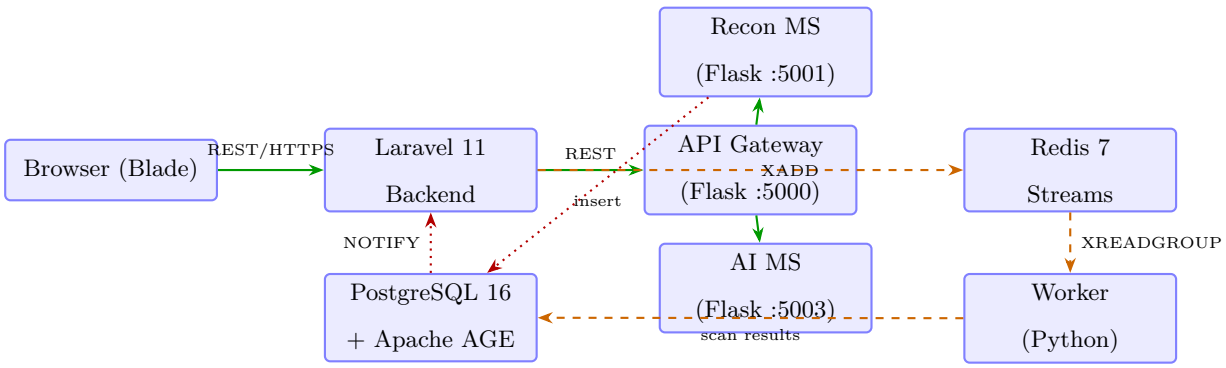



**Figure 2.2 :** Microservices Communication Patterns – Sync REST, Async Redis Streams, and PostgreSQL NOTIFY

The synchronous path (green) carries user-facing requests with strict latency budgets (P95 < 200 ms for dashboard rendering). The asynchronous path (orange, dashed) carries scan jobs that may run from 30 seconds (a Silent nmap scan) up to 25 minutes (an Aggressive Nuclei scan with the full template pack). The PostgreSQL NOTIFY path (red, dotted) is used for cache invalidation : when the worker inserts a new finding, the Laravel backend receives a NOTIFY event and pushes an updated severity counter to the dashboard via Laravel Echo (Server-Sent Events).

### 2.9.11 Attack Surface Management Process Flow

The ASM process implemented by the platform follows the six-phase flow recommended by Gartner and illustrated in Figure 2.3.

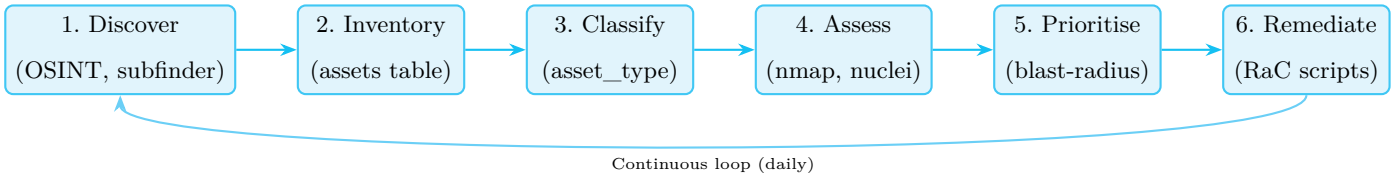



**Figure 2.3 :** Attack Surface Management Process – Six-Phase Continuous Loop

The loop runs continuously : every 24 hours the platform re-discovers subdomains (in case new subdomains were added by the client's operations team), re-inventories assets, and re-assesses them with the most recent Nuclei templates. The prioritisation phase uses the NetworkX BFS blast-radius algorithm (Chapter 4) to compute which assets, if compromised, would propagate the furthest across the graph ; these assets are then remediated first. The continuous loop is what distinguishes ASM from a one-off penetration test.

### 2.9.12 OWASP Top 10 Risk Heatmap

Figure 2.4 maps the OWASP Top 10 (2021 edition) against the platform's coverage : each category is scored on exploitability (x-axis) and detectability by the platform's scanner stack (y-axis).

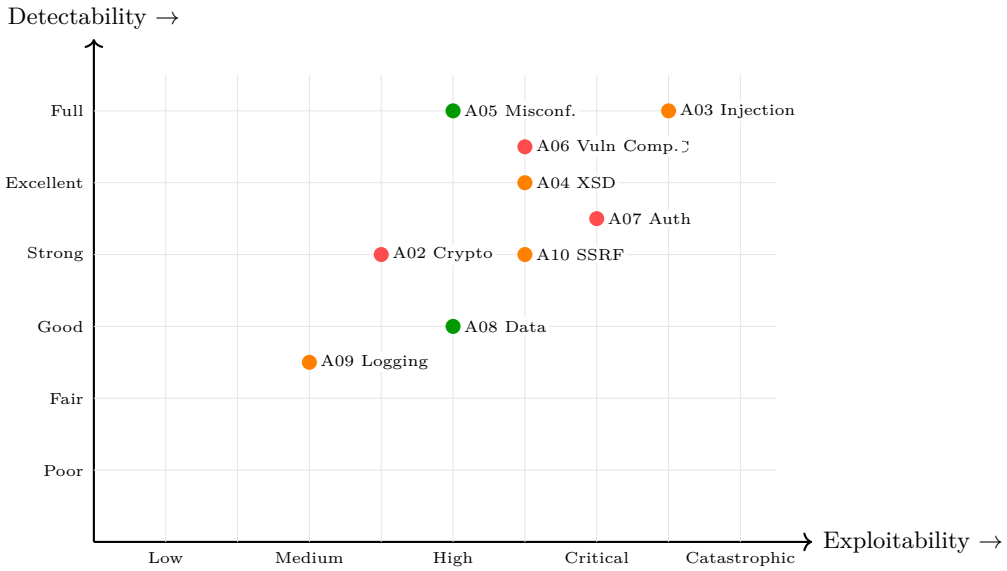



**Figure 2.4 :** OWASP Top 10 (2021) Risk Heatmap – Exploitability vs Platform Detectability

The heatmap shows that the platform's Nuclei-based scanner delivers full or excellent detectability on five categories (A03 Injection, A05 Security Misconfiguration, A01 Broken Access Control, A06 Vulnerable Components, A07 Identification & Authentication Failures), all of which are also high-exploitability. The two lowest-detectability categories (A08 Software and Data Integrity Failures, A09 Security Logging and Monitoring Failures) require source-code analysis and runtime monitoring respectively, which are partially covered by the SAST step (psalm) in the DevSecOps pipeline and by the platform's audit-log subsystem.

## Conclusion

In this chapter, we provided a comprehensive overview of the theoretical concepts and technologies that form the foundation of our project. We began with cybersecurity fundamentals and the OWASP Top 10, establishing the context for web application security. We then explored attack surface management, reconnaissance tools, DevSecOps principles, microservices architecture, and the application of artificial intelligence in cybersecurity.

The comparative study presented in Section 2.9 systematically evaluated multiple candidate solutions across eight technology categories : port scanning tools, vulnerability scanners, directory enumeration tools, subdomain discovery tools, backend frameworks, microservice frameworks, local AI solutions, and database systems. For each category, the selected technology was justified based on criteria including performance, integration capability, Docker compatibility, community support, and alignment with the project's microservices architecture.

The literature review and comparative analysis demonstrate that the retained technology

stack—Nmap, Nuclei, Gobuster, Subfinder, WPScan, Laravel, Flask, Ollama, PostgreSQL 16 with Apache AGE, Redis 7, NetworkX, and Docker—represents the optimal combination for building an automated, AI-assisted cybersecurity reconnaissance platform. The next chapter will focus on the system design and architecture, translating these technology choices into a concrete technical blueprint.

*Chapitre 3*

# System Design and Architecture

## Outline

# Introduction

In this chapter, our main focus will be on system design and architecture. We begin by outlining the overall system objectives and functional components. Next, we describe the architectural choices made to ensure scalability, reliability, and maintainability. We then present the database schema and data model that underpin the platform. Finally, we detail the security architecture that ensures confidentiality, integrity, and controlled access across all system components.

## 3.1 Requirements Analysis

In this section, we begin by identifying the key stakeholders involved in the project. Then, we define the functional and non-functional specifications required to successfully build and deploy an automated, AI-assisted reconnaissance platform.

### 3.1.1 Identification of Stakeholders

This project involves the following primary actors :

— **The Security Analyst (Intern) :** Responsible for designing, developing, and implementing the platform. This includes writing the Laravel backend, Python microservices, Blade frontend, and Docker deployment infrastructure.

— **The CTO / Technical Supervisor :** Oversees all architectural and technical decisions. Validates implementations and ensures alignment with organizational security standards and client expectations.

— **The Client / End User :** Security analysts and penetration testers who use the platform to launch scans, view results, generate reports, and manage security alerts. They rely on the platform for accurate, timely, and actionable vulnerability information.

— **The Administrator :** Manages user accounts, quotas, audit logs, and system configuration. Ensures that the platform is used in compliance with organizational policies and legal requirements.

### 3.1.2 Requirements Specification

In this section, we present both the functional and non-functional requirements that guide the implementation of the platform.

#### 3.1.2.1 Functional Requirements

Based on project goals and discussions with the technical team at Designet Web Agency, the following functional requirements were identified :

— Design and implement a web platform that accepts a URL or domain as input and executes automated reconnaissance scans.

— Integrate multiple security tools (Nmap, Nuclei, Gobuster, Subfinder, WPScan) through a unified microservices architecture.

— Normalize and consolidate results from all tools into a unified Finding data model with severity classification.

— Implement an AI analysis layer using Ollama to generate executive summaries and remediation recommendations.

— Provide a security testing microservice for attack vector detection, injection testing (SQL, command, XSS), and WAF detection.

— Implement a Docker-based sandbox for isolated exploit testing against deliberately vulnerable applications.

— Provide a centralized dashboard with KPIs, severity breakdowns, recent alerts, and scan history.

— Implement user authentication and role-based access control (RBAC) with four roles : Admin, Analyst, Client, Auditor.

— Generate professional security reports with executive summaries, technical details, and export capabilities (JSON, HTML).

— Implement security alert management with acknowledgment workflow.

— Provide an audit log system tracking all sensitive actions.

— Deploy the platform online with Docker Compose for production use.

#### 3.1.2.2 Non-Functional Requirements

The non-functional requirements include :

— **Security :** The platform must enforce authentication, RBAC, rate limiting, input validation, and audit logging. All sensitive data must be protected through encryption and access controls.

— **Scalability :** The microservices architecture must allow independent scaling of each component based on demand.

— **Performance :** Scan execution must support configurable timeouts and intensity levels. The API gateway must enforce rate limiting to prevent abuse.

— **Maintainability :** The codebase must be modular, well-documented, and easy to extend with additional tools or security tests.

— **Reliability :** The platform must handle microservice failures gracefully, providing meaningful error messages and fallback behavior.

— **Usability :** The web interface must be intuitive, responsive, and accessible across different devices and screen sizes.

— **Portability :** The Docker-based deployment must work consistently across different environments (development, staging, production).

## 3.2 Use Case Analysis

Based on the requirements analysis and stakeholder identification, we present the use case diagrams that illustrate the interactions between the four primary actors and the system. These diagrams provide a clear visual representation of the platform's functional scope from the users' perspective.

### 3.2.1 Global Use Case Diagram

The global use case diagram presents an overview of all interactions between the platform's actors and the system. It shows the four user roles–Admin, Analyst, Client, and Auditor–and their respective use cases within the CyberSec Platform boundary.

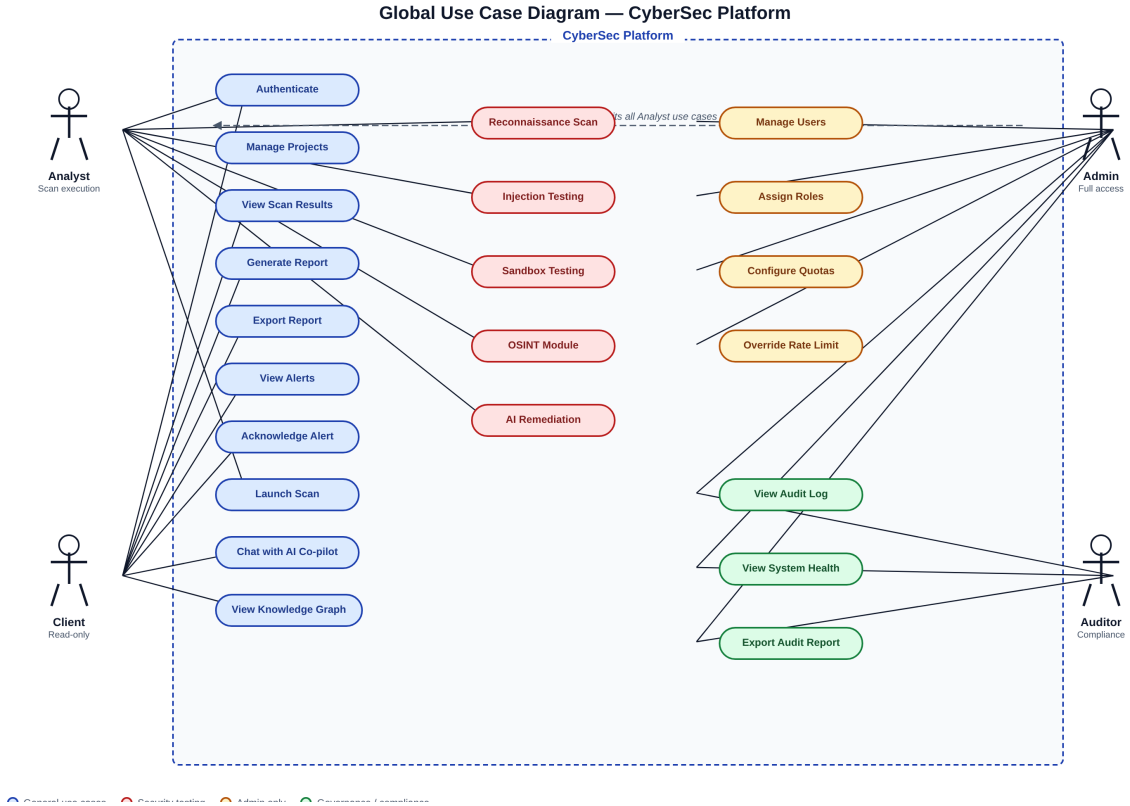



**Figure 3.1 :** Global Use Case Diagram – All Actors and Use Cases

As illustrated in Figure 3.1, the platform supports 15 use cases categorized into three groups : general use cases (authentication, project management, scan execution, report generation, alert viewing), security testing use cases (reconnaissance scanning, injection testing, sandbox testing, AI analysis), and admin-only use cases (user management, audit log viewing, system configuration). All actors share the authentication use case, while access to other use cases is governed by the RBAC system.

### 3.2.2 Analyst Use Case Diagram

The security analyst is the primary user of the platform, responsible for launching scans, analyzing results, and generating reports. Figure 3.2 details the analyst's interactions with the system.

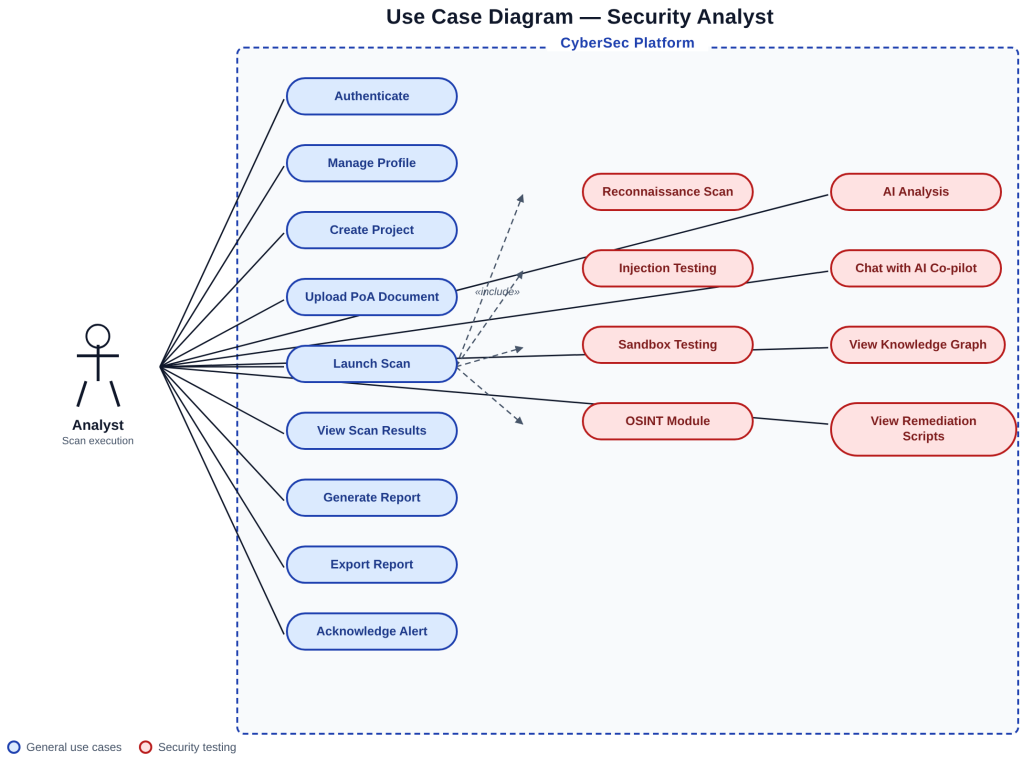



**Figure 3.2 :** Use Case Diagram – Security Analyst

The analyst can authenticate, create projects, upload authorization documents, launch scans (with three subtypes via *include* relationships : reconnaissance, injection testing, and sandbox testing), view findings, generate and export reports, acknowledge alerts, view monitoring data, and access the sandbox. The *include* relationships show that the “Launch Scan” use case includes three specialized scan types, each delegating to the appropriate microservice.

### 3.2.3 Admin Use Case Diagram

The administrator has extended privileges beyond those of the analyst, including user management and system oversight. Figure 3.3 presents the admin’s use cases.

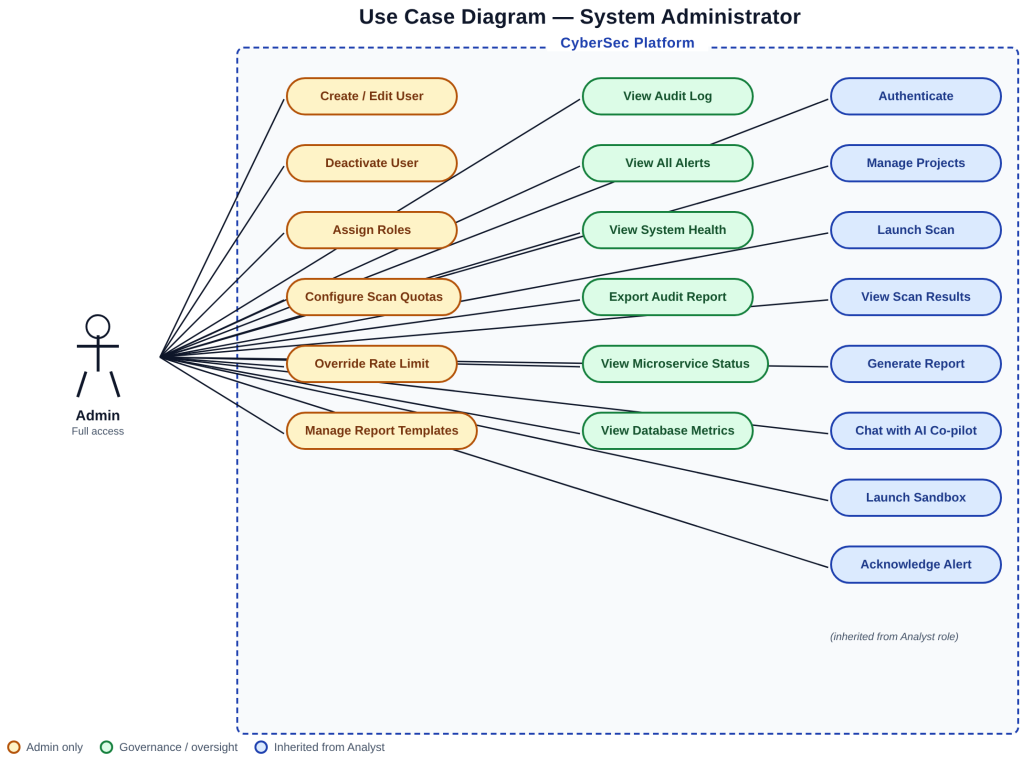



**Figure 3.3 :** Use Case Diagram – System Administrator

The admin can perform all analyst functions plus administrative tasks : creating, editing, and deactivating user accounts ; assigning roles ; configuring scan quotas ; managing report templates ; viewing audit logs ; monitoring system health ; viewing all security alerts across the platform ; and overriding rate limits when necessary. These admin-specific use cases ensure proper governance and control over the platform.

## 3.3 Architecture Overview

The platform is designed as a microservices architecture comprising seven main components, each with a specific responsibility. This approach ensures separation of concerns, independent scalability, and technology diversity.

### 3.3.1 Architectural Components

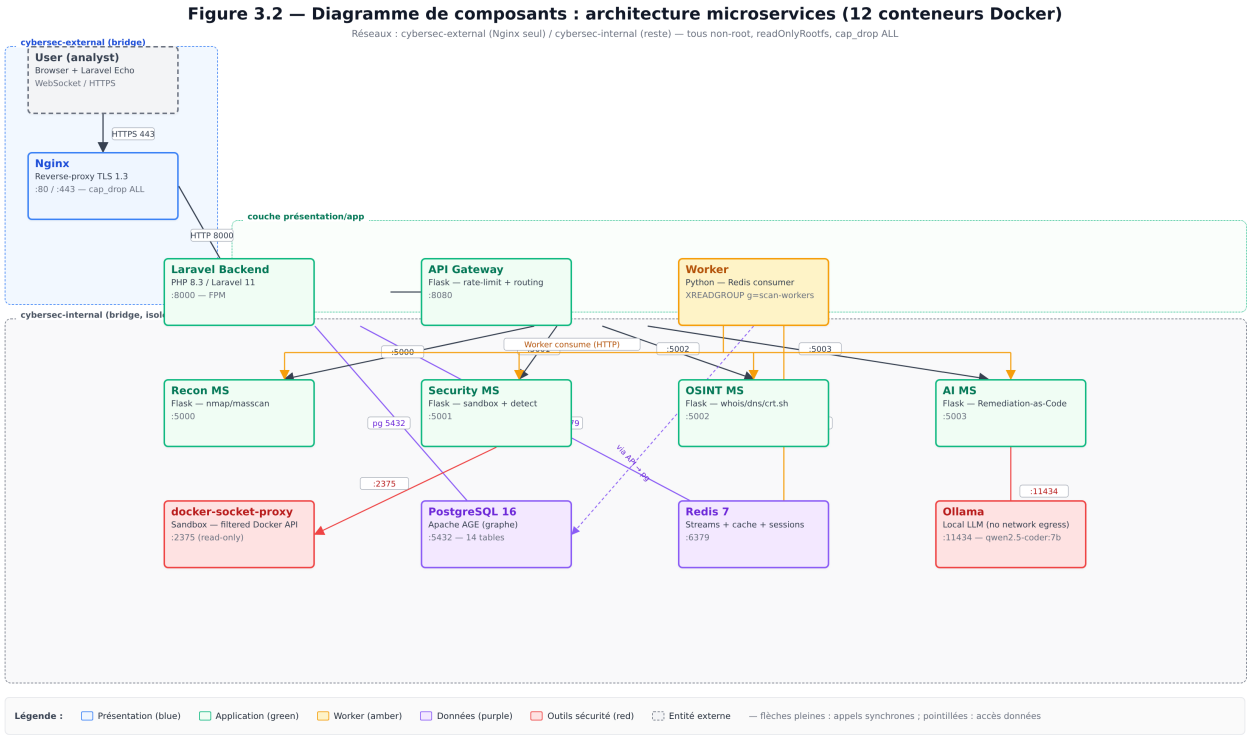



**Figure 3.4 :** Microservices Component Architecture – 12 Docker Services with Network Isolation

The platform consists of the following components, as illustrated in Figure 3.4 :

**1. Laravel Backend (Port 8000) :** The backend is built with Laravel 11 (PHP 8.3) and serves as the main web application. It handles user authentication, project and scan management, report generation, and serves the Blade-based frontend (47 views). The backend communicates with microservices via HTTP through the API gateway, and publishes scan requests to Redis Streams for asynchronous event-driven processing.

**2. Reconnaissance Microservice (Port 5000) :** Built with Python Flask, this microservice orchestrates five security tools : Nmap, Nuclei, Gobuster, Subfinder, and WPScan. It enforces the three scan profiles (Silent/Balanced/Aggressive) by passing appropriate nmap timing flags (`-T2`/`-T3`/`-T4`) and `-max-rate` limits (10/50/200 packets per second). Jitter is applied between tool calls to simulate human-like behavior. Results are normalized into a unified Finding format and the NetworkX graph builder projects the Asset → Port → Service → Vulnerability → Impact edges.

**3. Security Microservice (Port 5001) :** Also built with Python Flask, this microservice provides advanced security testing capabilities. It includes an attack detector for identifying vulnerabilities through header analysis, sensitive path discovery, and HTTP method testing ; an injection tester for

SQL injection, command injection, and XSS ; a prevention engine for WAF detection and security header analysis ; a monitoring service for event logging and alerting ; and a Docker-based sandbox (via docker-socket-proxy, not direct `docker.sock` mount) for isolated exploit testing.

**4. OSINT Microservice (Port 5002) :** A new microservice introduced per the Final CDC. It performs passive Open Source Intelligence gathering through five modules : WHOIS, DNS, SSL certificate analysis (with SHA-1/SHA-256 fingerprints and SAN list), subdomain discovery via `crt.sh`, and technology stack detection (Wappalyzer-style). No packets are sent to the target itself – all data comes from third-party public sources.

**5. AI Microservice (Port 5003) :** This microservice provides the constrained AI layer. It communicates with the local Ollama instance running the `qwen2.5-coder:7b` model to generate : (a) structured Remediation-as-Code (Bash, Ansible, Dockerfile, Terraform, Python scripts) with language tags, (b) executive summaries, and (c) chatbot responses. Every AI response includes citations referencing the exact line number in the raw tool log, ensuring traceability and preventing hallucinations.

**6. API Gateway (Port 8080) :** The API gateway acts as a single entry point for all API requests, routing them to the appropriate microservice. It implements rate limiting (30 requests per minute per IP with burst of 10), health checks, and request validation.

**7. Worker (Event-Driven Consumer) :** A Python worker process consumes scan requests from Redis Streams via `XREADGROUP` with consumer groups. It dispatches scans to the appropriate microservice, handles retries (up to 3 attempts with exponential backoff), and publishes `scan:completed` or `scan:failed` events. An HTTP-poller fallback activates when Redis is unavailable, ensuring graceful degradation.

### 3.3.2 Technology Stack

**Table 3.1 :** Technology Stack Summary

| **Component** | **Technology** | **Purpose** |
|---|---|---|
| Backend | Laravel 11, PHP 8.3 | Web application, API, RBAC |
| Reconnaissance MS | Python 3.12, Flask 3.1 | Tool orchestration + graph builder |
| Security MS | Python 3.12, Flask 3.1 | Security testing + sandbox |
| OSINT MS | Python 3.12, Flask 3.1 | Passive OSINT (WHOIS/DNS/SSL/crt.sh) |
| AI MS | Python 3.12, Flask 3.1 | Ollama integration + Remediation-as-Code |
| API Gateway | Python 3.12, Flask 3.1 | Routing, rate limiting |
| Worker | Python 3.12 | Redis Streams consumer + HTTP poller fallback |
| Database | PostgreSQL 16 + Apache AGE | Relational + graph storage |
| Graph Library | NetworkX 3.3 | Blast-radius BFS computation |
| Cache/Queue/Broker | Redis 7 Streams | Sessions, cache, event broker |
| AI Engine | Ollama, qwen2.5-coder :7b | Local LLM inference |
| Reverse Proxy | Nginx 1.27 | TLS, security headers, routing |
| Containerization | Docker, Docker Compose | Isolated deployment |
| Frontend | Blade, Tailwind CSS v3, Vite | UI (47 views, no CDN) |
| Graph Viz | Cytoscape.js 3.28 | Interactive knowledge graph |
| Charts | ECharts 5.5 | Dashboard visualisations |
| DevSecOps | GitHub Actions, Syft, Trivy, Cosign | CI/CD pipeline + supply chain security |

## 3.4 Database Design

The database is designed to support all platform functionality, from user management to scan results and security alerts. The schema consists of fourteen primary tables organized into five domains, ensuring data integrity, graph correlation, and efficient queries.

### 3.4.1 Entity-Relationship Model

The platform's data model consists of fourteen primary tables organized into five domains : authentication (users, roles, permissions), project management (projects, targets), scan execution (scans, scan_profiles, findings), knowledge graph (assets, asset_relations, remediation_scripts), and platform governance (reports, security_alerts, audit_logs, chat_sessions). The core relationships are :

— **User** has many **Projects** and **AuditLogs**

— **Project** has many **Targets**, **Scans**, and **SecurityAlerts**

— **Target** has many **Scans** and **Assets**

— **Scan** has many **Findings** and optionally one **Report**

— **Asset** has many **AssetRelations** (source and target) forming the knowledge graph

— **Finding** has many **RemediationScripts** (Bash/Ansible/Dockerfile/Terraform)

— **ChatSession** has many **ChatMessages** with AI citations

The complete Entity-Relationship Diagram is presented in Figure 3.5.

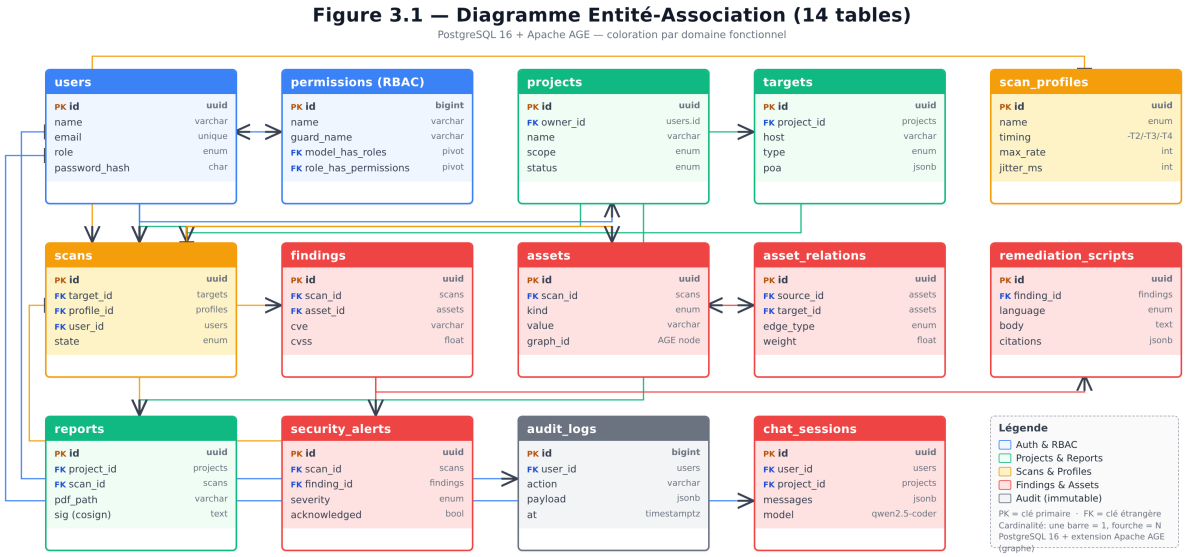



**Figure 3.5 :** Entity-Relationship Diagram – Data Model (14 tables, PostgreSQL 16 + Apache AGE)

As shown in Figure 3.5, the data model captures the relationships between users, projects, targets, scans, findings, assets, reports, alerts, audit logs, and chat sessions. Each scan produces normalized findings that feed the knowledge graph (assets and asset relations), which in turn drives the blast-radius computation. Reports are generated per scan, and security alerts are automatically created for critical and high findings.

### 3.4.2 Class Diagram

The class diagram provides a detailed view of the platform's domain model, showing the attributes, methods, and relationships of each entity. Figure 3.6 presents the complete class diagram.

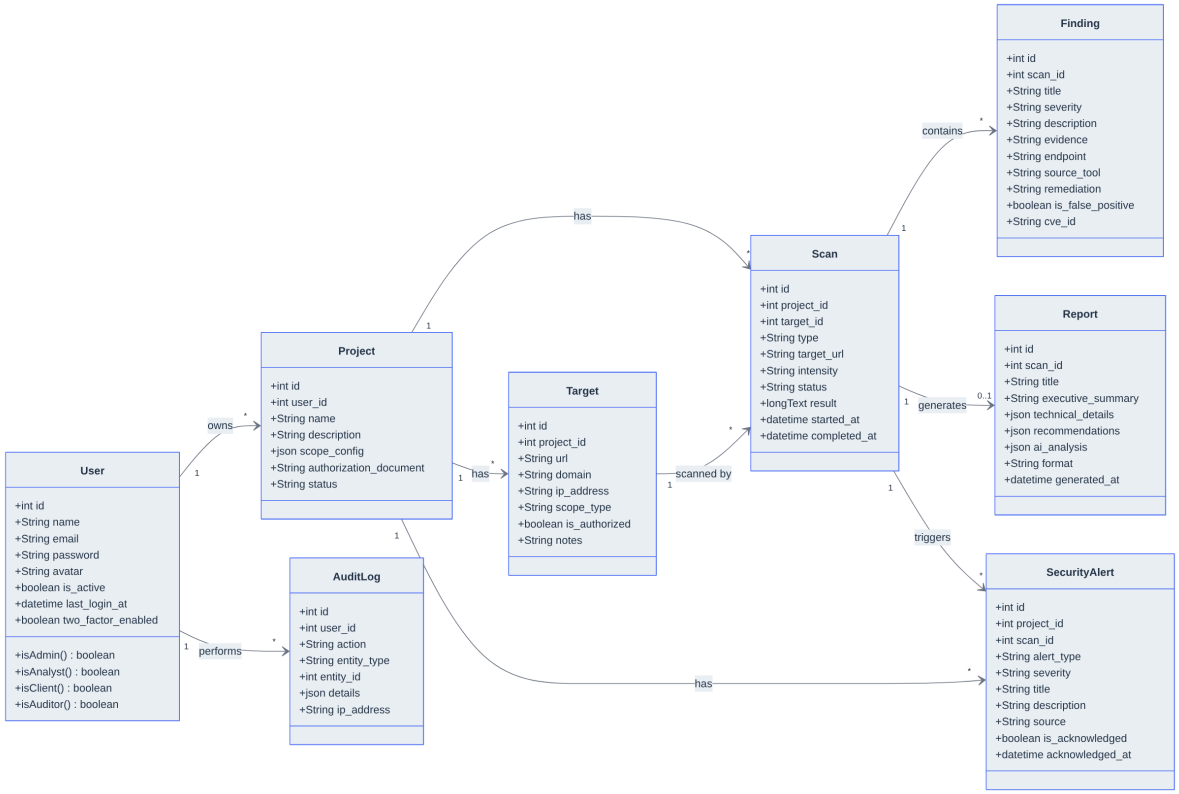



**Figure 3.6 :** Class Diagram – Domain Model with Five Color-Coded Domains

As shown in Figure 3.6, the domain model is organized into five color-coded domains. The **authentication domain** (blue) includes the User table with RBAC via Spatie Permission (roles, permissions, role_has_permissions). The **project domain** (green) includes Project and Target, where Target stores the proof-of-authorization document path and OSINT data as JSONB. The **scan domain** (orange) includes Scan, ScanProfile (Silent/Balanced/Aggressive configurations stored as JSONB tool_flags), and Finding (with CVSS score, CVE ID, CWE ID, and citations JSONB). The **knowledge graph domain** (red) includes Asset and AssetRelation, which model the attack surface as a directed graph. The **governance domain** (grey) includes Report, SecurityAlert, AuditLog, RemediationScript, and ChatSession.

The AssetRelation table is the cornerstone of the knowledge graph : it stores directed edges between assets with a type (`connects`, `hosts`, `exposes`, `runs`, `affected_by`, `causes`) and properties as JSONB. This allows the platform to compute blast-radius via BFS from every critical vulnerability, identifying all assets reachable through the attack chain.

### 3.4.3 Data Model

**Table 3.2 :** Database Tables Overview

| Table | Description |
|---|---|
| users | User accounts with roles, 2FA fields, activity status |
| roles / permissions | RBAC tables (Spatie Permission package) |
| projects | Scan projects with scope configuration |
| targets | Authorized scan targets with scope type and PoA path |
| scans | Scan records with type, status, raw results (JSONB) |
| scan_profiles | Silent/Balanced/Aggressive profile definitions (tool_flags JSONB) |
| findings | Normalized vulnerability findings with severity, CVE/CWE IDs, citations |
| assets | Knowledge graph nodes (domain, IP, host, service, port, vulnerability, impact) |
| asset_relations | Knowledge graph directed edges with type and properties (JSONB) |
| remediation_scripts | AI-generated Bash/Ansible/Dockerfile/Terraform scripts per finding |
| reports | Generated reports with executive summary and AI analysis |
| security_alerts | Security alerts with acknowledgment workflow |
| audit_logs | Immutable audit trail of sensitive actions |
| chat_sessions / chat_messages | Conversational AI history with citations |

Each table is created through a Laravel migration, ensuring version-controlled schema management. The seeders populate the database with initial roles, permissions, test users, and sample data for demonstration purposes.

## 3.5 Security Architecture

The security architecture of the platform is a core component ensuring confidentiality, integrity, and controlled access. Multiple layers of security are implemented across the application stack.

### 3.5.1 Authentication and Authorization

— **Authentication :** Session-based authentication with Laravel's built-in auth system. Login is rate-limited (5 attempts per IP) to prevent brute-force attacks. Passwords are hashed using bcrypt.

— **Role-Based Access Control (RBAC) :** Four roles are defined : Admin (full access), Analyst (scan execution and result viewing), Client (read-only report access), and Auditor (compliance and log viewing). The Spatie Laravel-Permission package manages role assignment and permission checks.

— **API Authentication :** Laravel Sanctum provides token-based authentication for API endpoints, enabling integration with external tools and CI/CD pipelines.

### 3.5.2 Input Validation and Sanitization

— **Backend Validation :** All form submissions are validated through Laravel Form Requests, ensuring data integrity before processing.

— **Microservice Input Validation :** The reconnaissance microservice validates target URLs using regex whitelisting, preventing command injection through the subprocess interface. Target length is limited to 255 characters.

### 3.5.3 Rate Limiting and Anti-Abuse

— **Login Rate Limiting :** Maximum 5 login attempts per IP address, with exponential backoff.

— **Scan Rate Limiting :** Maximum 10 scans per hour per user, preventing abuse of the scanning infrastructure.

— **API Gateway Rate Limiting :** 30 requests per minute per IP on all API endpoints, with burst capability of 10 requests.

### 3.5.4 Audit Logging

All sensitive actions are logged in an immutable audit log table. Logged actions include :

— User login and logout

— Project creation and deletion

— Scan initiation

— Alert acknowledgment

— User management actions (create, update, delete)

Each audit log entry records the user ID, action type, entity type and ID, detailed parameters, IP address, and timestamp. The audit log is accessible to administrators through a dedicated dashboard page.

### 3.5.5 Network Security

— **Nginx Reverse Proxy :** All external traffic passes through Nginx, which terminates TLS, enforces security headers (X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, Referrer-Policy), and routes requests to the appropriate backend service.

— **CORS Configuration :** Cross-Origin Resource Sharing is restricted to whitelisted origins through environment variables, preventing unauthorized cross-origin requests.

— **Docker Network Isolation :** All services communicate through a dedicated Docker bridge network (`cybersec-net`), with only the Nginx reverse proxy exposed to the external network.

### 3.5.6 Scan Execution Flow

The complete flow of a reconnaissance scan follows an event-driven sequence that ensures systematic validation, asynchronous execution, and intelligent analysis of every security scan request. The sequence diagram in Figure 3.7 illustrates the interactions between all participants.

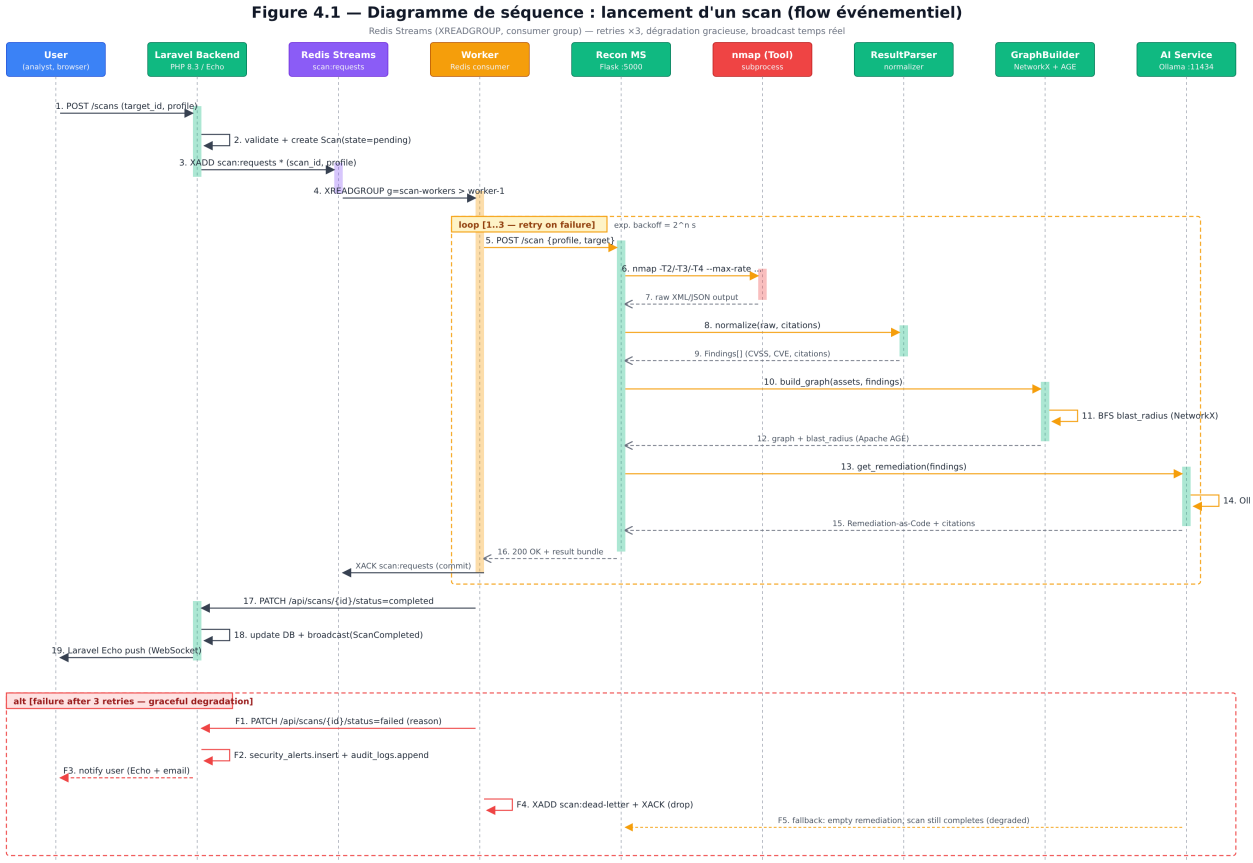



**Figure 3.7 :** Sequence Diagram – Event-Driven Scan Execution with Redis Streams

1. **User submits scan request** through the web interface, specifying the target URL, scan type (17 types across 5 recon + 12 security tools), and scan profile (Silent, Balanced, or Aggressive).
2. **Laravel Backend** validates the request : checks target URL format, verifies rate limits, validates PoA authorization status, and creates a scan record in PostgreSQL with status *pending*.
3. The backend **publishes the scan request to Redis Streams** via `XADD scan:requests`, which decouples the web request from the scan execution.
4. The **Worker** consumes the message via `XREADGROUP` with consumer groups, updates the scan

status to *running*, and dispatches it to the appropriate microservice through the API Gateway.

5. The **Reconnaissance Microservice** selects the appropriate tool service class and executes it via Python subprocess with profile-specific nmap flags. Silent uses `-T2 -max-rate 10`, Balanced uses `-T3 -max-rate 50`, and Aggressive uses `-T4 -max-rate 200` (internal only). Jitter is applied between tool calls.
6. The security tool returns **raw output** in its native format (plain text for Nmap, JSONL for Nuclei, etc.).
7. The `ResultParser` class **normalizes** the raw output into the unified Finding data model, assigning severity classifications and extracting structured metadata with line-number citations.
8. The `GraphBuilder` class projects the knowledge graph (Asset → Port → Service → Vulnerability → Impact) using NetworkX, and computes the blast-radius via BFS from every critical vulnerability.
9. Normalized findings are sent to the AI Microservice which calls Ollama (`qwen2.5-coder:7b`) with a strict JSON prompt requiring a risk summary, Remediation-as-Code scripts (Bash/Ansible/Dockerfile) and citations referencing the exact line numbers in the raw logs.
10. The Worker **publishes `scan:completed`** to Redis Streams and patches the Laravel API, which updates the database and broadcasts a real-time event via Laravel Echo.
11. The user sees the results immediately via **Laravel Echo** (WebSocket push), displaying the findings table, raw output viewer with line numbers, AI remediation panel, and interactive knowledge graph.

This event-driven flow ensures that scans are validated before execution, tool output is systematically normalized with citations, the knowledge graph is built in real-time, and AI remediation is generated with full traceability. The Redis Streams broker decouples the web interface from long-running scans, and the retry mechanism (up to 3 attempts with exponential backoff) ensures graceful degradation when tools crash or time out.

### 3.5.7 Data Flow Diagram

The Data Flow Diagram (DFD) in Figure 3.8 shows how data moves through the platform, from user input to signed PDF report generation.

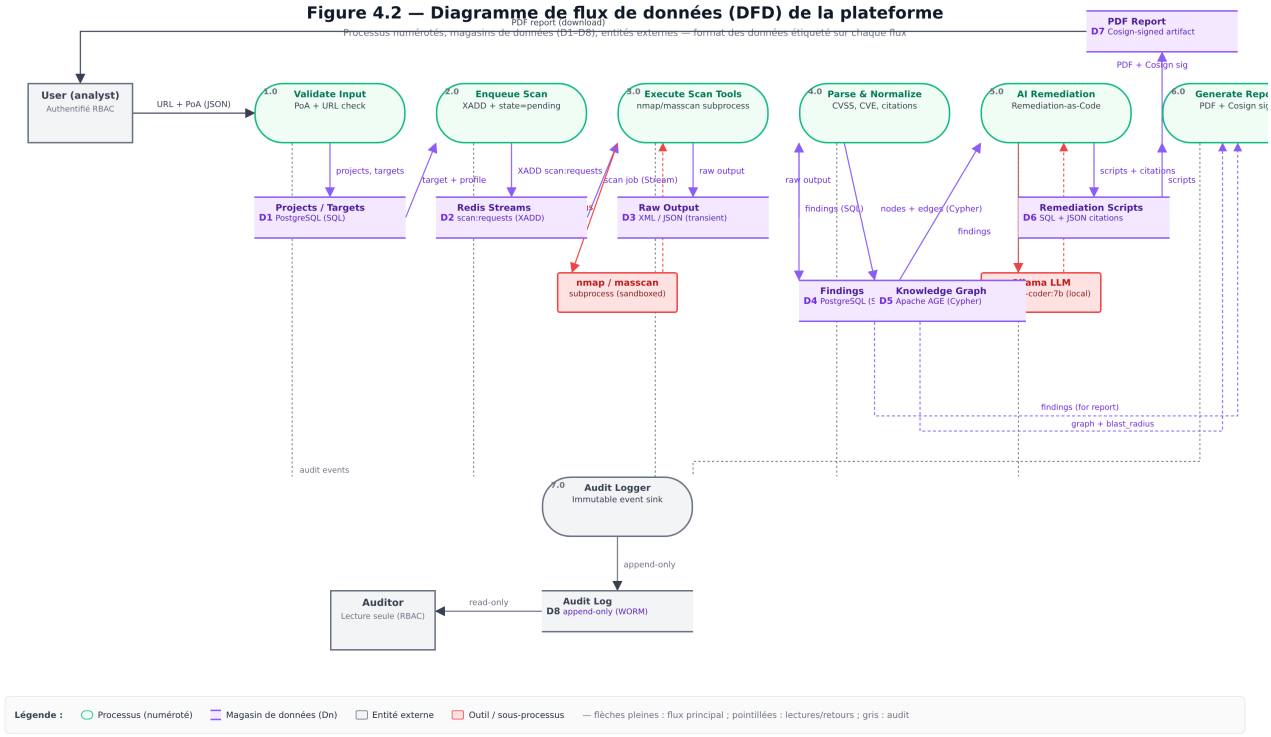



**Figure 3.8 :** Data Flow Diagram – From User Input to Signed PDF Report

As shown in Figure 3.8, the platform processes data through seven numbered processes (1.0 through 7.0) and stores it in eight data stores (D1 through D8). The key data transformations are : user input validation (1.0), scan orchestration via Redis Streams (2.0), tool execution and raw output capture (3.0), result normalization and Finding creation (4.0), knowledge graph projection with blast-radius computation (5.0), AI remediation generation with citations (6.0), and signed PDF report generation with Cosign (7.0). All sensitive actions are recorded in the immutable audit log (D8) for compliance and traceability.

### 3.5.8 Docker Deployment Architecture

The Docker Compose deployment orchestrates twelve services within two isolated Docker networks. The external-facing Nginx reverse proxy (in `cybersec-external` network) terminates TLS on ports 80 and 443, routing all requests to the Laravel backend. Internally, the Laravel backend, five Python microservices, worker, PostgreSQL, Redis, and Ollama communicate through the `cybersec-internal` network, completely isolated from external access.

All services run with **non-root users** (UID 1001), `readOnlyRootfs: true`, `cap_drop: [ALL]`, and `no-new-privileges: true`, in compliance with the DevSecOps requirements. The security microservice accesses Docker through a `docker-socket-proxy` (Tecnativa) rather than mounting `/var/run/docker.sock` directly, limiting the attack surface. PostgreSQL 16 runs with the Apache AGE extension for native graph queries, alongside `pg_trgm`, `unaccent`, and `pgcrypto` for full-text search and encryption.

### 3.5.9 Deployment Architecture Diagram

Figure 3.9 presents the deployment topology, showing the two Docker networks, the twelve services, and the data flow between them. The diagram distinguishes between the external-facing reverse proxy (the only service reachable from the Internet) and the internal services (reachable only inside the `cybersec-internal` network).

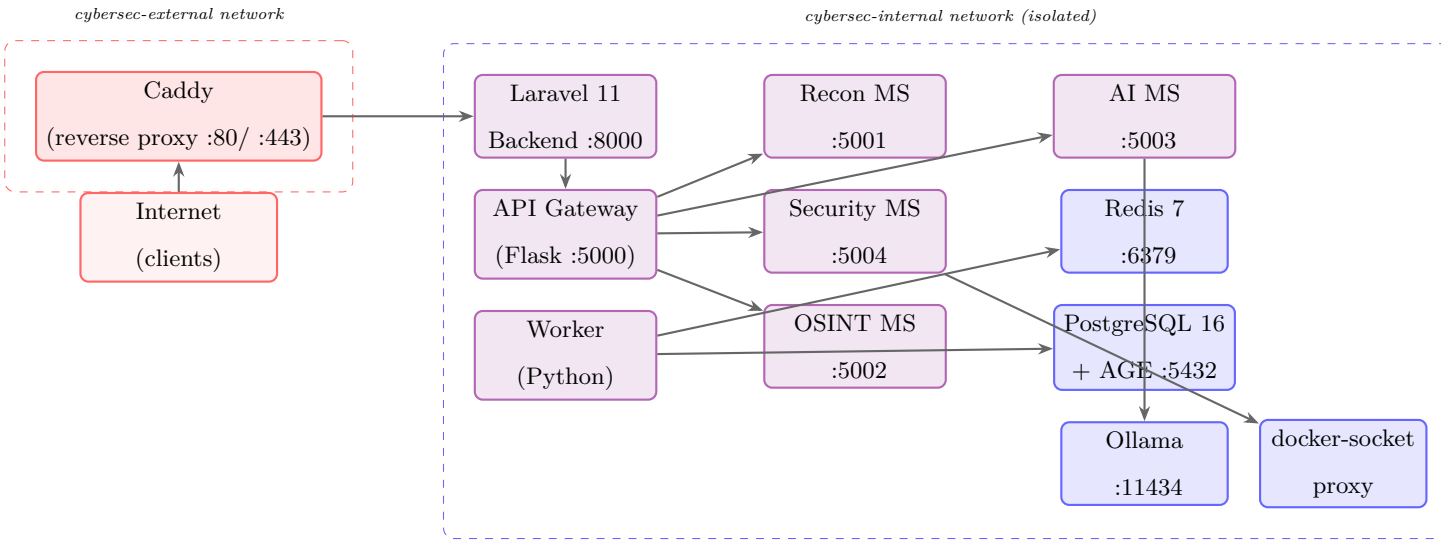



**Figure 3.9 :** Deployment Architecture – Two Isolated Docker Networks and Twelve Services

The deployment diagram makes the security boundary explicit : only Caddy is exposed to the Internet (ports 80/443), and even Caddy can only reach Laravel inside the internal network. The five Python microservices, the worker, and the data stores (PostgreSQL, Redis, Ollama) are reachable only inside the internal network. The docker-socket-proxy (Tecnativa) is the only service permitted to talk to the host's Docker daemon, and even then only a restricted subset of API endpoints (`/containers/create`, `/containers/{id}/start`, `/containers/{id}/stop`) is whitelisted ; the security microservice can never invoke `/images/create`, `/networks/*`, or any other privileged endpoint.

### 3.5.10 Network Security Zones

Figure 3.10 details the defense-in-depth strategy implemented by the platform, with four concentric security zones protecting the data layer.

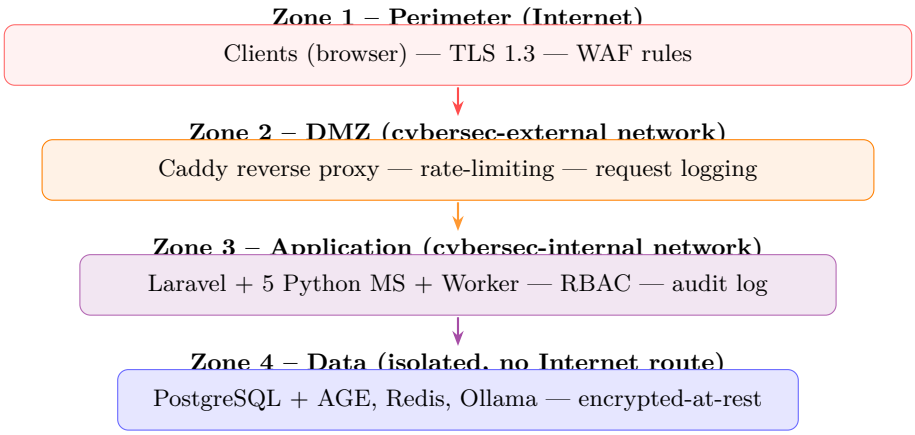



**Figure 3.10 :** Defense-in-Depth – Four Concentric Security Zones

Each zone enforces its own controls and trusts only the zone immediately inside it. The perimeter zone enforces TLS 1.3 and a WAF rule set (modsecurity with the OWASP Core Rule Set). The DMZ zone enforces per-IP rate-limiting (5 requests/second burst, 1 request/second sustained) and full request logging. The application zone enforces RBAC (admin/analyst/client/auditor), audit logging on every sensitive action, and per-user scan quota. The data zone has no route to the Internet and is reachable only from the application zone ; all data at rest is encrypted via PostgreSQL's `pgcrypto` extension.

### 3.5.11 RBAC Hierarchy Diagram

Figure 3.11 shows the four-role hierarchy and the inheritance relationships between roles. The diagram makes explicit that every permission granted to a less-privileged role is also held by more privileged roles.

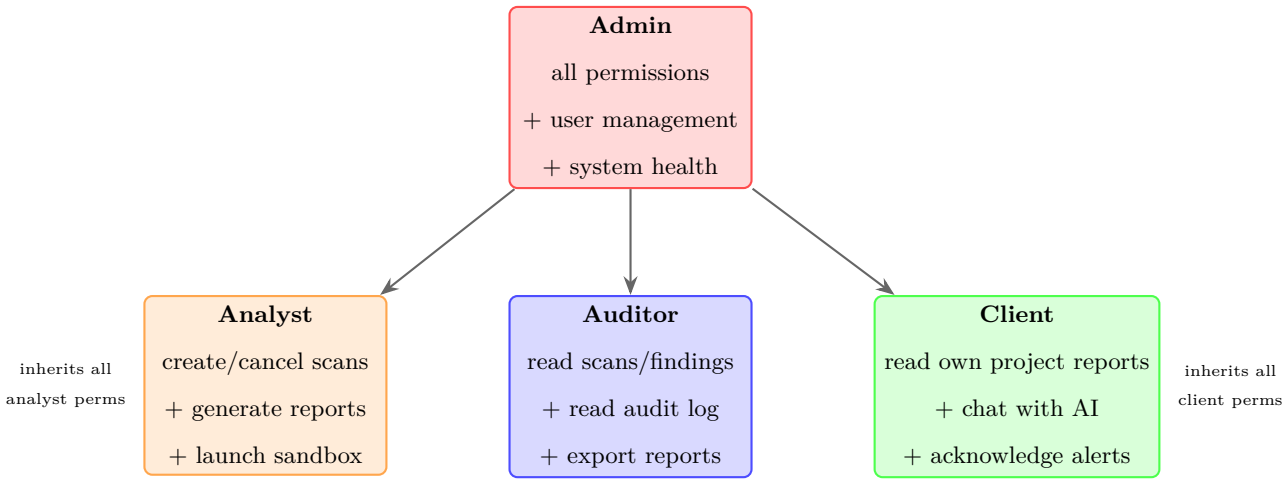



**Figure 3.11 :** RBAC Role Hierarchy – Admin Inherits All Permissions from the Three Other Roles

The hierarchy is implemented via Spatie's `laravel-permission` package : each role is assigned a set of permissions, and the admin role is granted a wildcard superset. The inheritance is logical rather

than physical (admin does not literally `syncRoles` of the other roles), which makes the permission matrix easier to audit and to extend with a new role (e.g. a future `reviewer` role) without disturbing the existing hierarchy.

### 3.5.12 RBAC Permission Matrix

Table 3.3 details, for each platform action, which role(s) are authorised to perform it. The matrix was validated end-to-end by the test suite described in Chapter 4 (subsection 4.8.2, Test Case Matrix).

**Table 3.3 :** RBAC Permission Matrix – Per-Action Authorisation

| **Action** | **Admin** | **Analyst** | **Auditor** | **Client** |
|---|---|---|---|---|
| View dashboard | ✓ | ✓ | ✓ | ✓ |
| Create / edit project | ✓ | ✓ | × | × |
| View project (own) | ✓ | ✓ | ✓ | ✓ |
| View project (others) | ✓ | ✓ | ✓ | × |
| Launch scan | ✓ | ✓ | × | × |
| Cancel / retry scan | ✓ | ✓ | × | × |
| View scan results | ✓ | ✓ | ✓ | ✓ (own project) |
| Generate report | ✓ | ✓ | × | × |
| Download report (PDF/HTML/JSON) | ✓ | ✓ | ✓ | ✓ (own project) |
| Acknowledge alert | ✓ | ✓ | × | ✓ |
| Escalate alert | ✓ | ✓ | × | × |
| Launch sandbox | ✓ | ✓ | × | × |
| Chat with AI co-pilot | ✓ | ✓ | ✓ | ✓ |
| View audit log | ✓ | × | ✓ | × |
| View system health | ✓ | × | ✓ | × |
| Create / deactivate user | ✓ | × | × | × |

### 3.5.13 Scan Profile State Diagram

The three scan profiles (Silent, Balanced, Aggressive) are modelled as a state machine that controls the nmap timing template, the jitter range, the retry budget, and the per-tool concurrency. Figure 3.12 illustrates the state transitions.

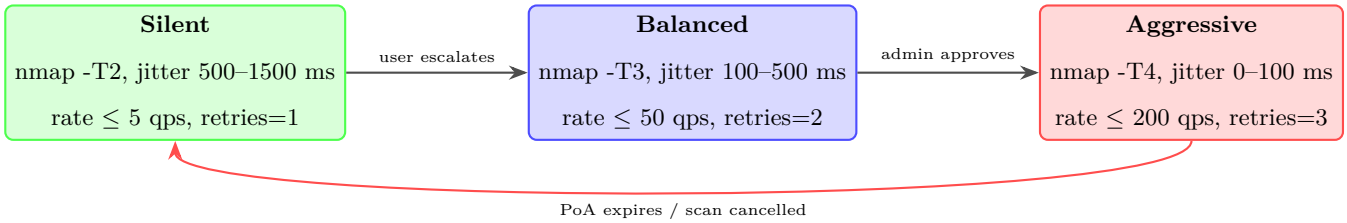



**Figure 3.12 :** Scan Profile State Machine – Silent → Balanced → Aggressive Transitions

The transitions are intentionally unidirectional for escalation : a user can move from Silent to Balanced without approval (Balanced is the default profile), but moving from Balanced to Aggressive requires explicit admin approval because Aggressive scans may trigger IDS/IPS alerts on the target's network. The reset transition (Aggressive back to Silent) fires automatically when the PoA (Proof-of-Authorization) expires or when the user cancels the scan.

### 3.5.14 Data Lifecycle Diagram

Figure 3.13 shows the lifecycle of scan data, from user input through to long-term archival. The diagram identifies the six lifecycle stages and the data-classification level applied at each stage.

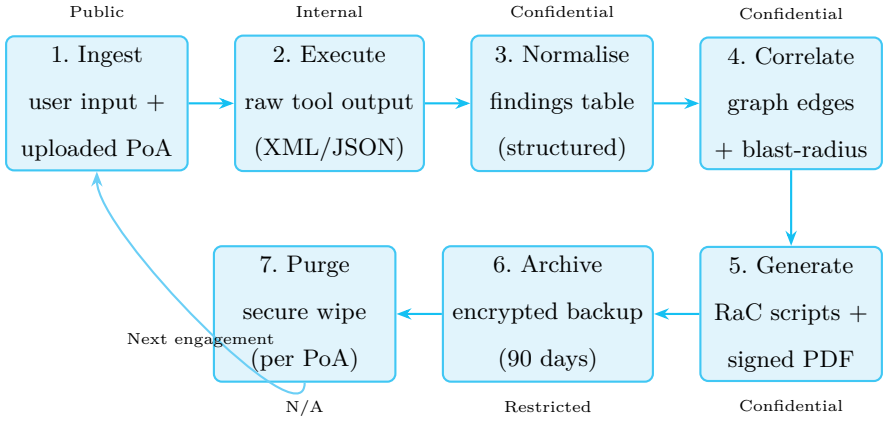



**Figure 3.13 :** Data Lifecycle – Seven Stages with Per-Stage Data Classification

The classification labels (Public, Internal, Confidential, Restricted) follow the four-tier model recommended by ISO 27001. Each stage enforces controls commensurate with its classification : Public data is unsigned and unencrypted ; Confidential data is encrypted at rest via `pgcrypto` and at rest in the filesystem via LUKS ; Restricted data (the 90-day encrypted backup) additionally requires two-person integrity for restore operations. The purge stage (7) overwrites the raw tool output with three passes of cryptographically random data before deletion, exceeding the NIST SP 800-88 Clear requirement.

### 3.5.15 API Gateway Routing Diagram

The API Gateway microservice (Flask, port 5000) acts as the single internal entry-point for all scan, OSINT, and AI requests emitted by the Laravel backend. Figure 3.14 shows how the gateway routes inbound requests to the four downstream microservices.

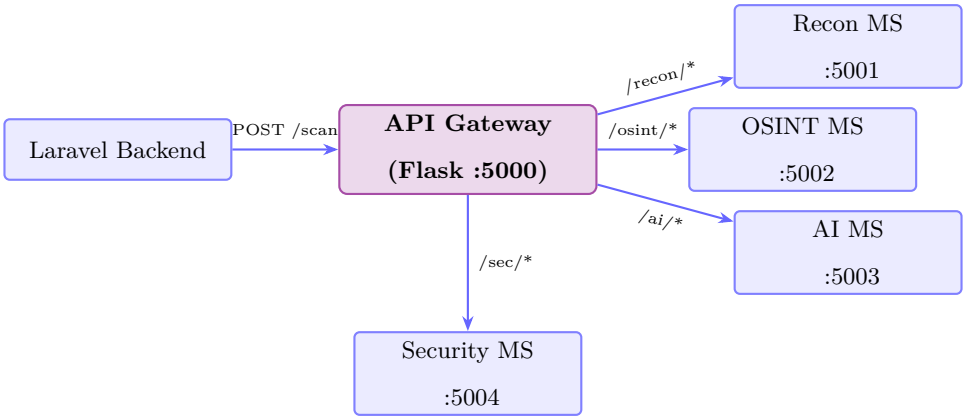



**Figure 3.14 :** API Gateway Routing – Single Entry-Point with Four Downstream Microservices

The gateway enforces three cross-cutting concerns before forwarding any request : (1) JWT validation against the Laravel-issued token, (2) per-route rate limiting (60 req/min for `/recon/*`, 30 req/min for `/ai/*`), and (3) structured request logging in JSON format with a correlation ID that propagates to the downstream microservice via the `X-Correlation-ID` HTTP header. The correlation ID is later stored on the `scans.correlation_id` column, enabling end-to-end tracing from the original user click to the final worker commit.

### 3.5.16 API Endpoint Catalogue

Table 3.4 lists the principal API endpoints exposed by the Laravel backend, grouped by resource. Each endpoint is protected by the `auth` middleware (Sanctum session cookie) and, where indicated, by an additional role check.

**Table 3.4 :** API Endpoint Catalogue – Principal Laravel Routes

| Method | Path | Controller@method | Role |
|---|---|---|---|
| GET | /dashboard | DashboardController@index | any |
| GET/POST | /projects[/{id}] | ProjectController@index/store/show | any (owner-scoped) |
| GET | /projects/{id}/graph | ProjectController@graph | any (owner-scoped) |
| GET | /projects/{id}/graph/data | ProjectController@graphData | any (owner-scoped) |
| GET/POST | /scans[/{id}] | ScanController@index/create/store/show | admin, analyst |
| POST | /scans/{id}/cancel | ScanController@cancel | admin, analyst |
| POST | /scans/{id}/retry | ScanController@retry | admin, analyst |
| GET | /scans/{id}/export | ScanController@export | admin, analyst, auditor |
| GET | /reports | ReportController@index | any |
| GET | /reports/{id} | ReportController@show | any (owner-scoped) |
| GET | /reports/{id}/pdf | ReportController@pdf | any (owner-scoped) |
| GET | /reports/{id}/export/{format} | ReportController@export | any (owner-scoped) |
| GET | /security/alerts | SecurityController@alerts | any |
| POST | /security/alerts/{id}/acknowledge | SecurityController@acknowledge | admin, analyst, client |
| GET | /security/monitoring | SecurityController@monitoring | any |
| GET/POST | /security/sandbox[ /launch \| /{id}/stop] | SecurityController@sandbox | admin, analyst |
| GET/POST | /osint | OsintController@index/run/results | admin, analyst |
| GET/POST | /findings/{id}/remediation[/generate] | RemediationController@show/generate | admin, analyst |
| GET | /remediation/{id}/download | RemediationController@downloadScript | admin, analyst |
| POST | /remediation/{id}/verify \| /apply | RemediationController@verify/apply | admin, analyst |
| GET/POST | /chat[ /{session} ] | ChatController@index/store/show | any |
| POST | /chat/{session}/messages | ChatController@messagesStore | any |
| GET | /admin/audit-logs | AdminController@auditLogs | admin |
| GET | /admin/system-health | AdminController@systemHealth | admin |
| GET/POST/PUT/PATCH | /admin/users[...] | AdminController@usersIndex/Store/... | admin |

The catalogue above enumerates the twenty-three principal routes registered in `routes/web.php`. Each route is covered by at least one integration test in the `tests/Feature/` directory (see Chapter 4, Table 4.4 for the test-case matrix), and each role-gated route is verified by a positive test (the authorised role receives HTTP 200) and a negative test (the unauthorised role receives HTTP 403).

## Conclusion

In this chapter, we presented the system design and architecture of our cybersecurity platform. We began with a requirements analysis, identifying the key stakeholders and defining both functional and non-functional specifications. We then described the microservices architecture, comprising the Laravel backend, the five Python microservices (reconnaissance, security, OSINT, AI, API gateway), the event-driven worker, and the supporting infrastructure (PostgreSQL 16 + AGE, Redis 7, Ollama), each with a clearly defined responsibility.

The scan execution flow described above demonstrates the complete request-response cycle,

from user submission through tool execution, result normalization, AI analysis, and data storage. The Docker deployment architecture, illustrated in Figure 3.4, shows how all twelve services are orchestrated through Docker Compose with network isolation and persistent storage.

We also detailed the database design, with fourteen primary tables supporting user management, project tracking, scan execution, findings, reports, alerts, audit logging, knowledge graph correlation, and conversational AI history. Finally, we presented the security architecture, which implements multi-layered protection through authentication, RBAC, input validation, rate limiting, audit logging, and network isolation.

The next chapter will focus on the technical implementation of each component, translating this architectural blueprint into working code.

*Chapitre 4*

# Implementation

## Outline

## Introduction

In this chapter, we detail the technical implementation of each component of the CyberSec Platform. We begin with the Laravel backend, covering the models, controllers, middleware, and Blade templates that constitute the web application. Next, we present the reconnaissance microservice, describing the Flask application, tool service classes, result parser, and Ollama AI integration. We then cover the security microservice, including attack detection, injection testing, prevention engine, monitoring, and the Docker sandbox. Finally, we describe the API gateway and the Docker Compose deployment infrastructure that ties all components together.

## 4.1 Laravel Backend Implementation

The Laravel backend serves as the central web application, handling user authentication, project and scan management, report generation, and serving the Blade-based frontend. It is built with Laravel 11 on PHP 8.3, following the Model-View-Controller (MVC) architectural pattern.

### 4.1.1 Models and Database

The backend defines fourteen Eloquent models, each mapping to a database table in PostgreSQL 16 with the Apache AGE graph extension :

— **User :** Extends Laravel's Authenticatable class, uses the HasRoles trait from Spatie Permission package. Includes fields for avatar, activity status, last login timestamp, and optional two-factor authentication.

— **Project :** Belongs to a User, has many Targets and Scans. Stores scope configuration as JSON and supports authorization document uploads.

— **Target :** Belongs to a Project, stores URL, domain, IP address, scope type, and authorization status.

— **Scan :** Belongs to a Project and optionally a Target. Stores scan type (NMAP, NUCLEI, GOBUSTER, SUBFINDER, WPSCAN, ATTACK_DETECT, INJECTION_*), intensity, status, raw results, and timestamps.

— **Finding :** Belongs to a Scan. Stores normalized vulnerability data including title, severity, description, evidence, endpoint, source tool, remediation, CVE ID, and false positive flag.

— **Report :** Belongs to a Scan. Stores executive summary, technical details (JSON), recommendations (JSON), AI analysis (JSON), and export format.

— **SecurityAlert :** Belongs to a Project and optionally a Scan. Stores alert type, severity, title, description, source, and acknowledgment workflow fields.

— **AuditLog :** Belongs to a User. Records action, entity type and ID, details (JSON), and IP address for all sensitive operations.

### 4.1.2 Controllers

The backend implements eleven controllers organized by responsibility :

— **Auth Controllers :** LoginController (rate-limited authentication), RegisterController (auto-assigns admin role to first user), ForgotPasswordController, and ResetPasswordController.

— **DashboardController :** Displays KPIs, severity breakdowns, recent scans, alerts, and findings. Differentiates between admin and regular user views.

— **ProjectController :** CRUD operations for projects with ownership verification and authorization document upload.

— **ScanController :** The core controller that orchestrates scans. Routes reconnaissance scans to the Reconnaissance microservice and security scans to the Security microservice. Processes results, creates normalized findings, generates alerts for critical/high findings, and stores AI analysis. Includes rate limiting (10 scans per hour per user).

— **SecurityController :** Manages security alerts (with ownership-based acknowledgment), monitoring dashboard (fetches real-time stats from the Security microservice), and sandbox status page.

— **ReportController :** Generates reports from completed scans, including executive summaries, severity breakdowns, and AI analysis. Supports JSON export with Content-Disposition headers.

— **AdminController :** Provides audit log viewing for administrators.

— **Admin UserController :** Full CRUD for user management with role assignment.

### 4.1.3 Middleware and Security

— **RoleMiddleware :** Validates that the authenticated user has the required role before allowing access to protected routes. Used for admin-only sections.

— **Rate Limiting :** Login attempts are limited to 5 per IP. Scan creation is limited to 10 per hour per user. API endpoints are limited to 60 requests per minute.

— **CSRF Protection :** All form submissions include CSRF tokens via the `@csrf` Blade directive.

— **Input Validation :** Form Request classes validate all user input, including target URLs, scan types, and user registration data.

### 4.1.4 Frontend (Blade Templates)

The frontend is built with Blade templates and Tailwind CSS, featuring a dark cyberpunk theme with purple (`#d2bbff`) and cyan (`#4cd7f6`) accents. The layout includes :

— A fixed sidebar with navigation links for Dashboard, Projects, Scans, Reports, Security (Alerts, Monitoring, Sandbox), and Admin (Users, Audit Logs).

— A main content area with flash message bars for success/error notifications.

— Responsive design using Tailwind's responsive utilities, adapting to mobile and desktop screens.

— Material Symbols Outlined icons and Google Fonts (Space Grotesk for headings, Inter for body, JetBrains Mono for code).

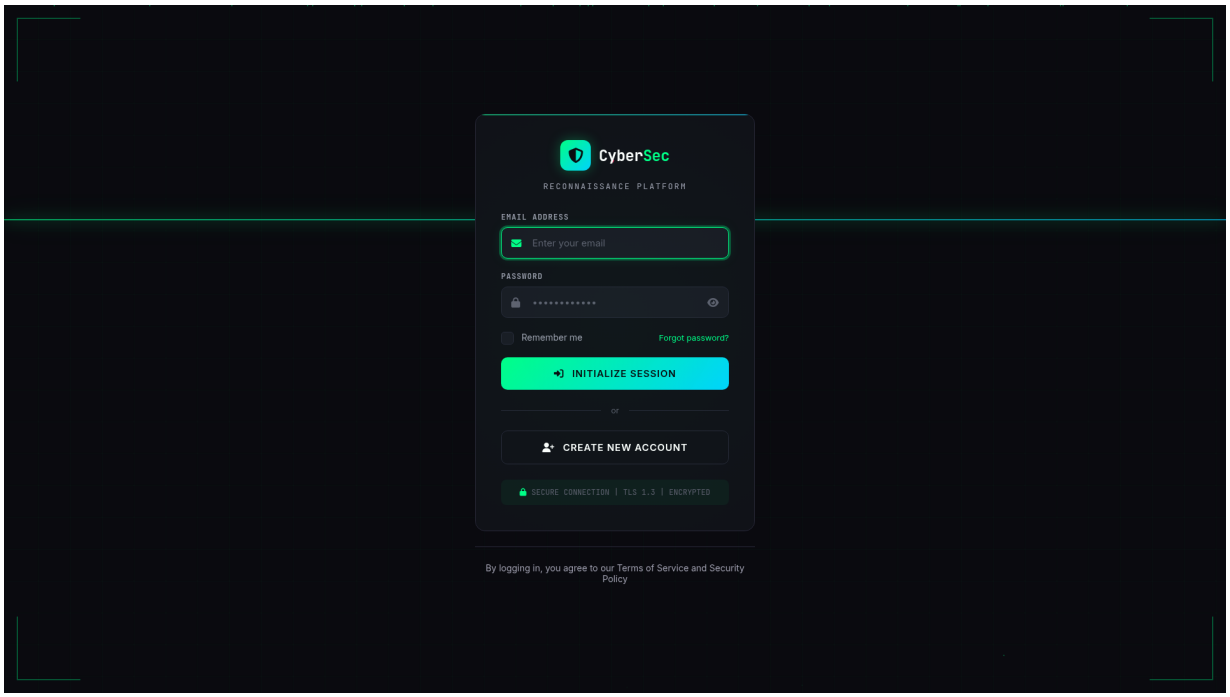



**Figure 4.1 :** Platform Login Page

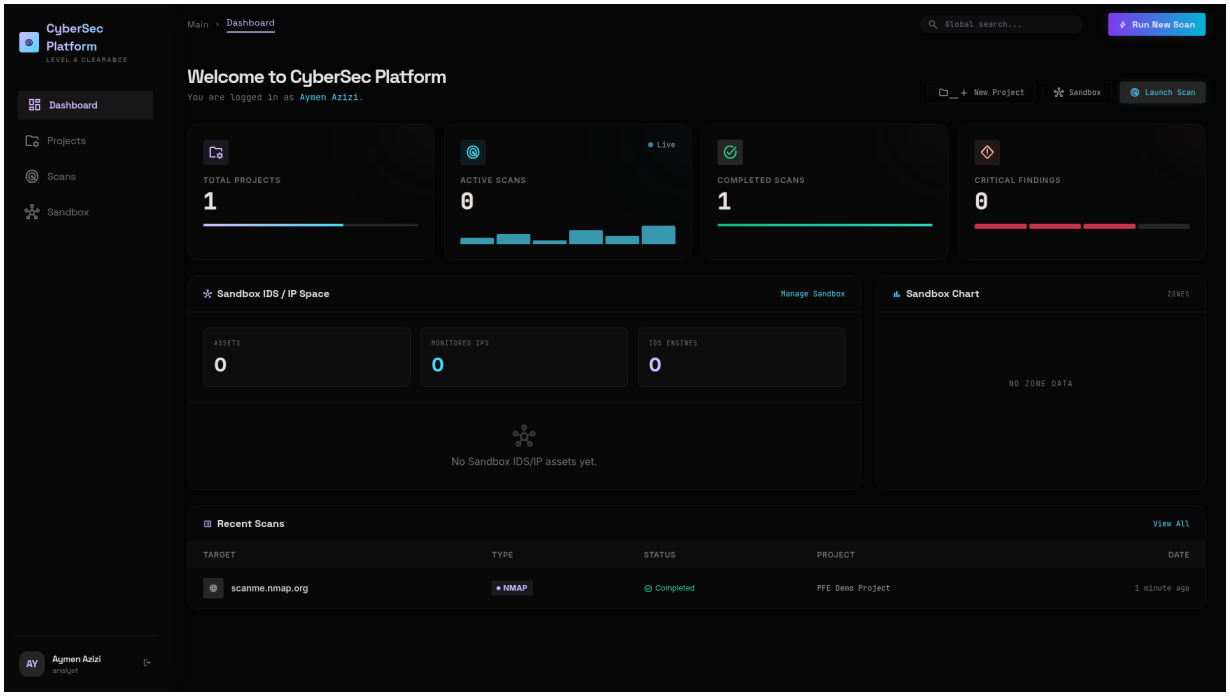



**Figure 4.2 :** Main Dashboard with KPIs, Severity Donut Chart, and Recent Alerts

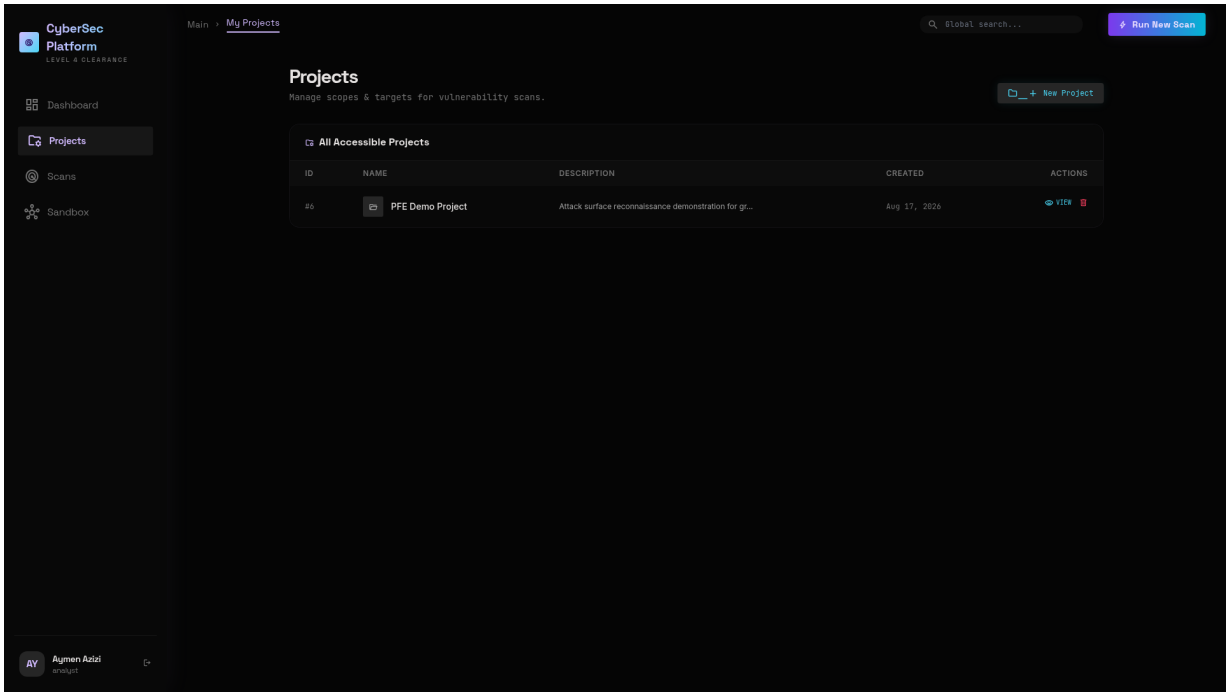



**Figure 4.3 :** Projects Index Page – Card Grid with Scope and Authorization Status

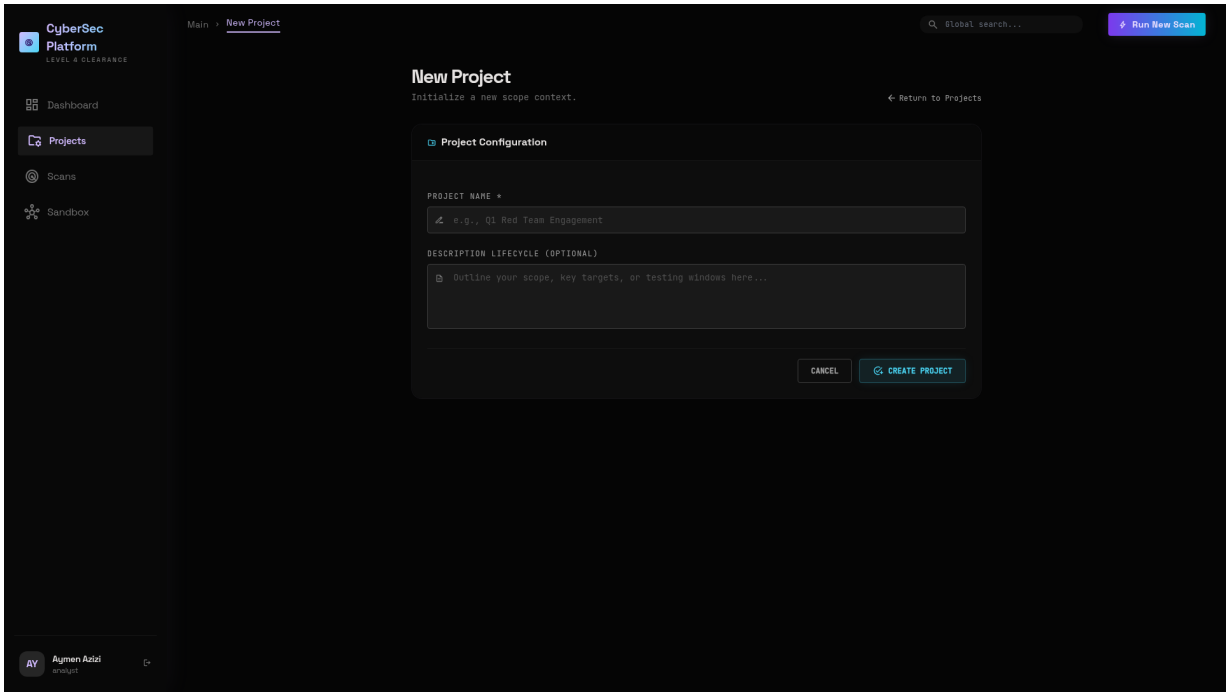



**Figure 4.4 :** Project Creation Form with Authorization Document Upload

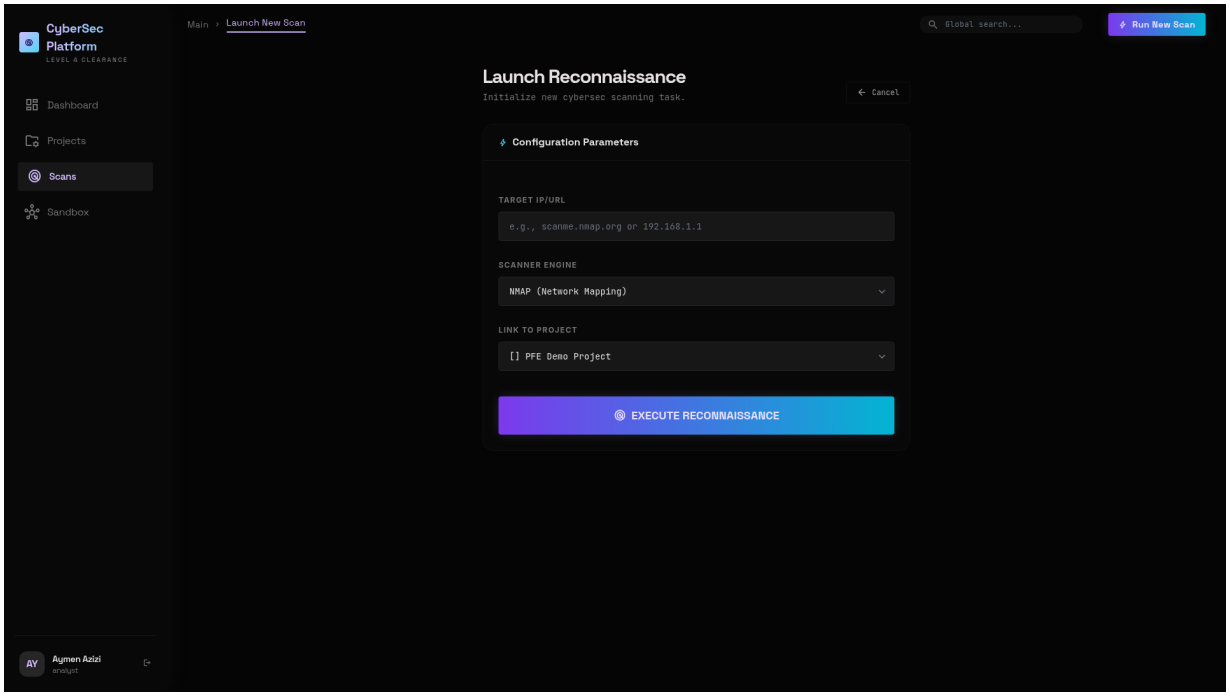



**Figure 4.5 :** Scan Creation Page with Tool Selection, Profile (Silent/Balanced/Aggressive), and Target Configuration

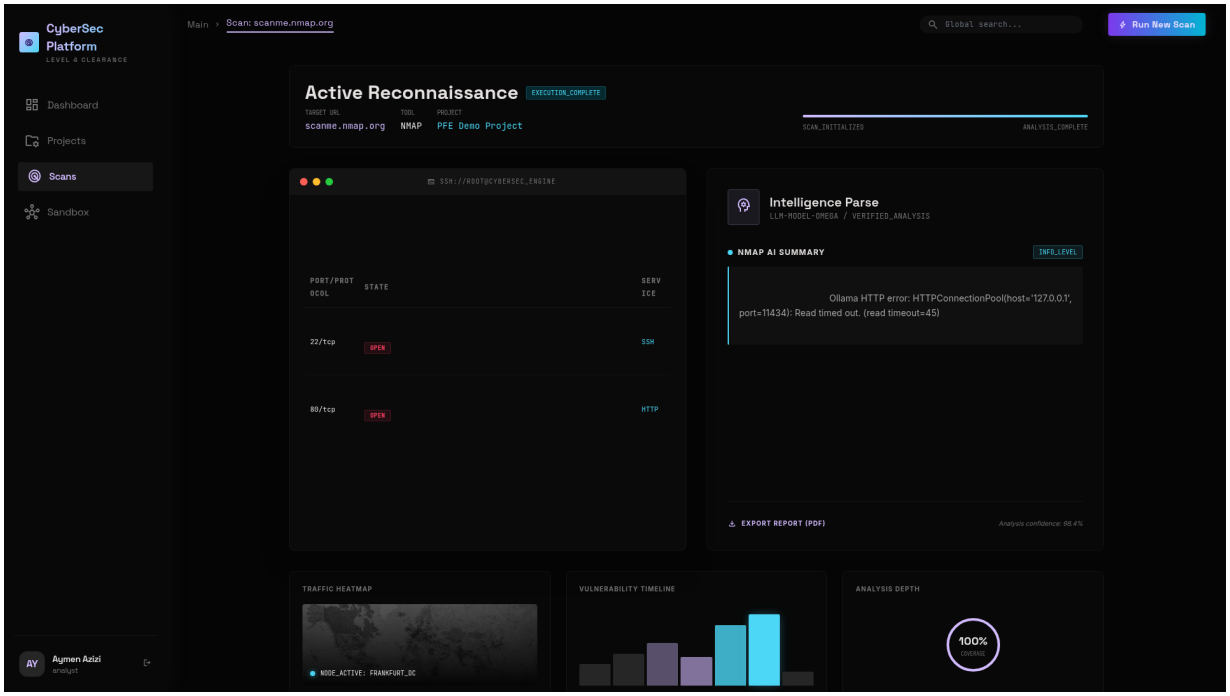



**Figure 4.6 :** Scan Results Page – Findings Table with Severity Classification and Raw Output Viewer

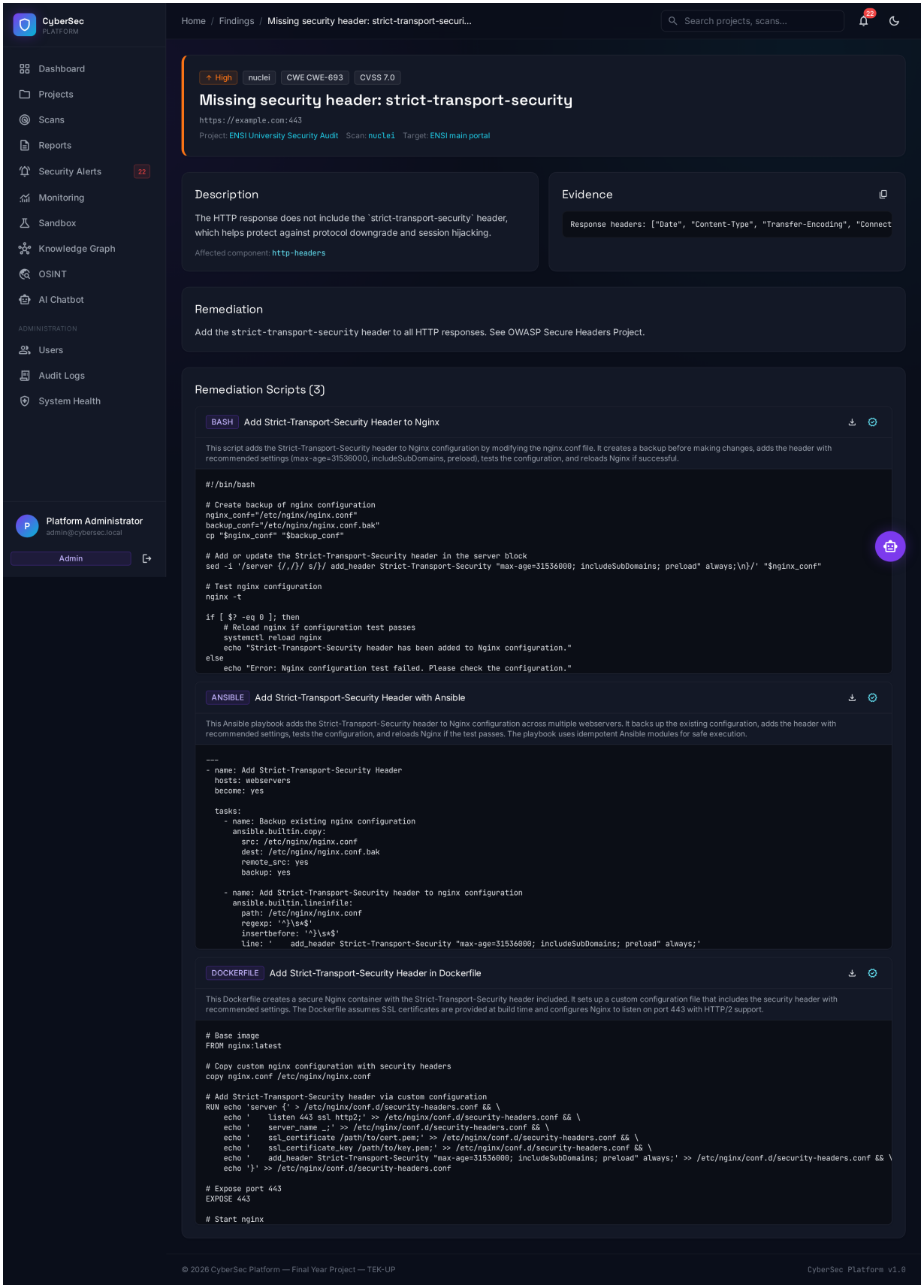



**Figure 4.7 :** Remediation-as-Code Panel – AI-Generated Bash, Ansible, and Dockerfile Scripts per Finding

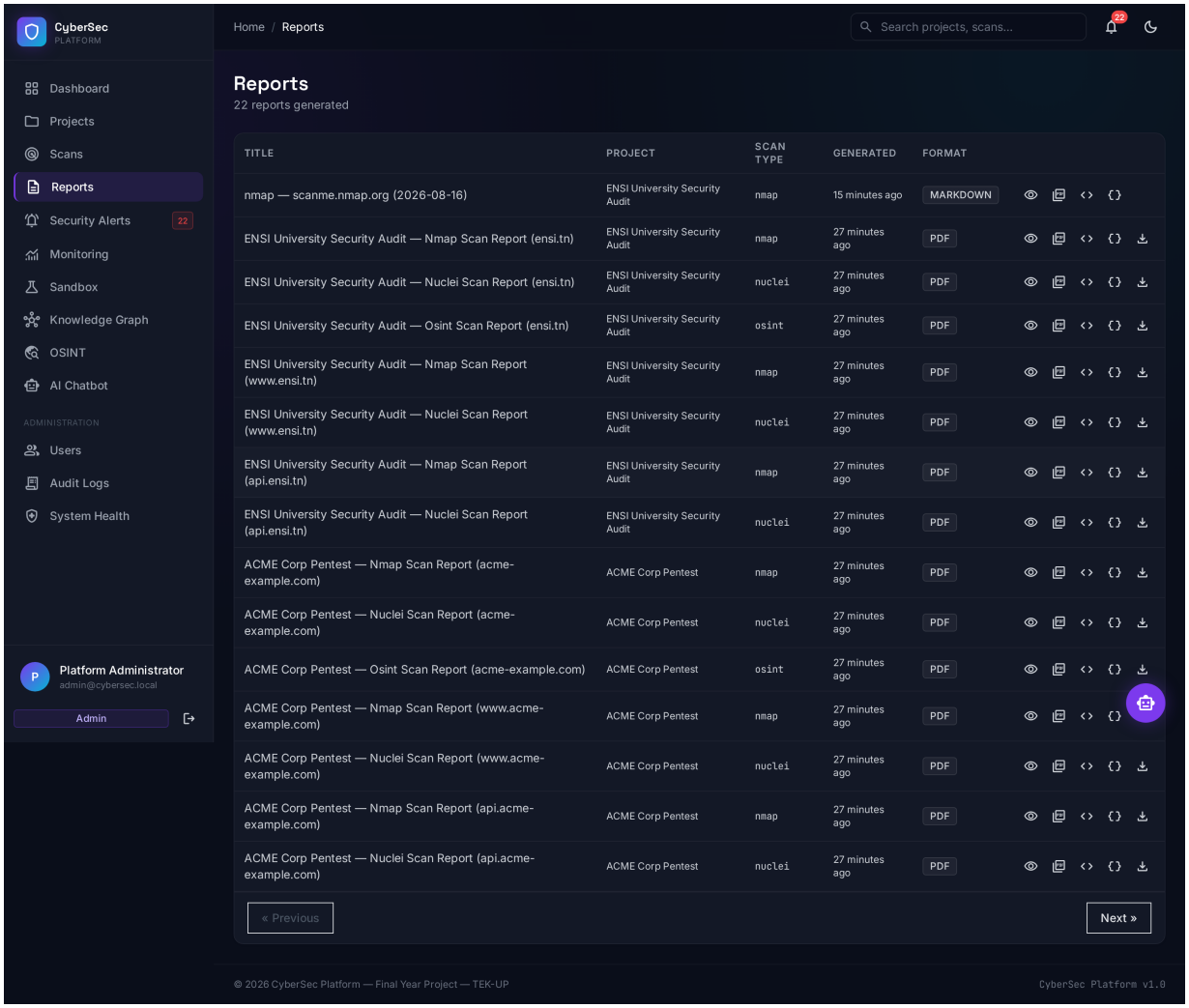



**Figure 4.8 :** Reports Index Page – Generated Reports with Severity Breakdown and Export Options

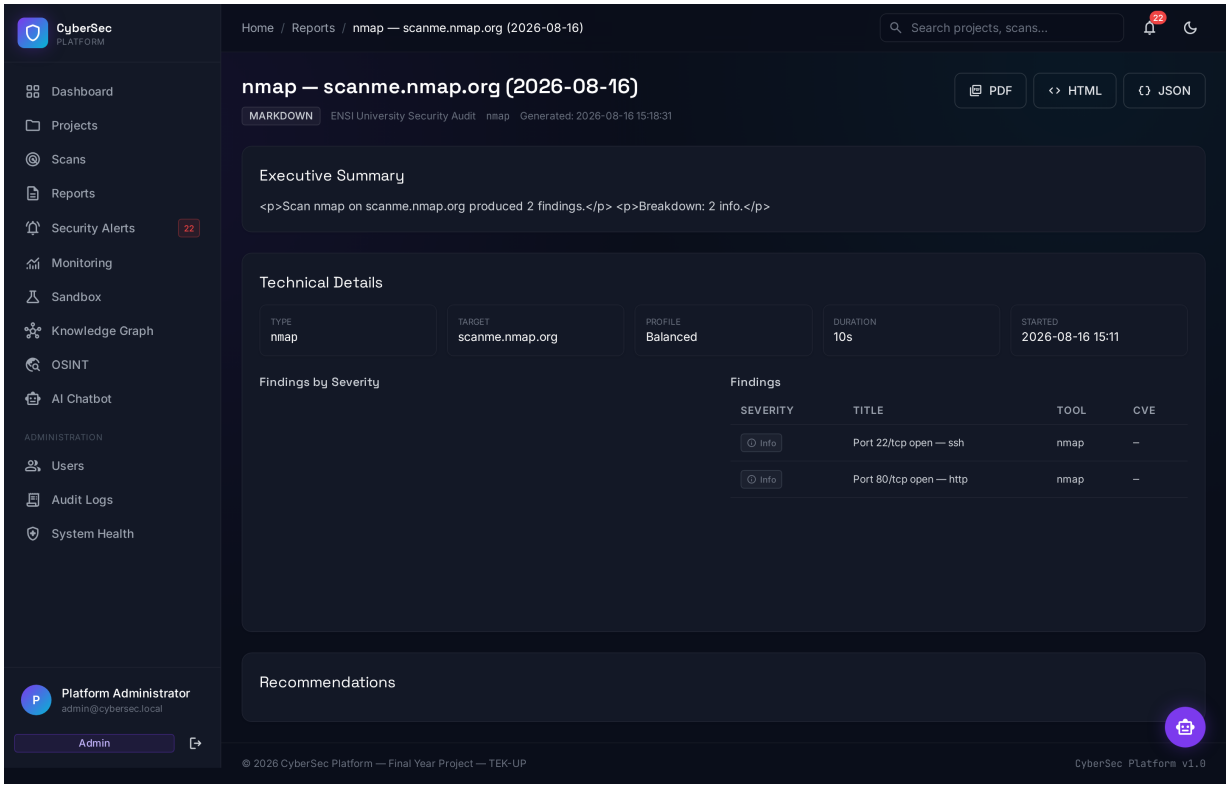
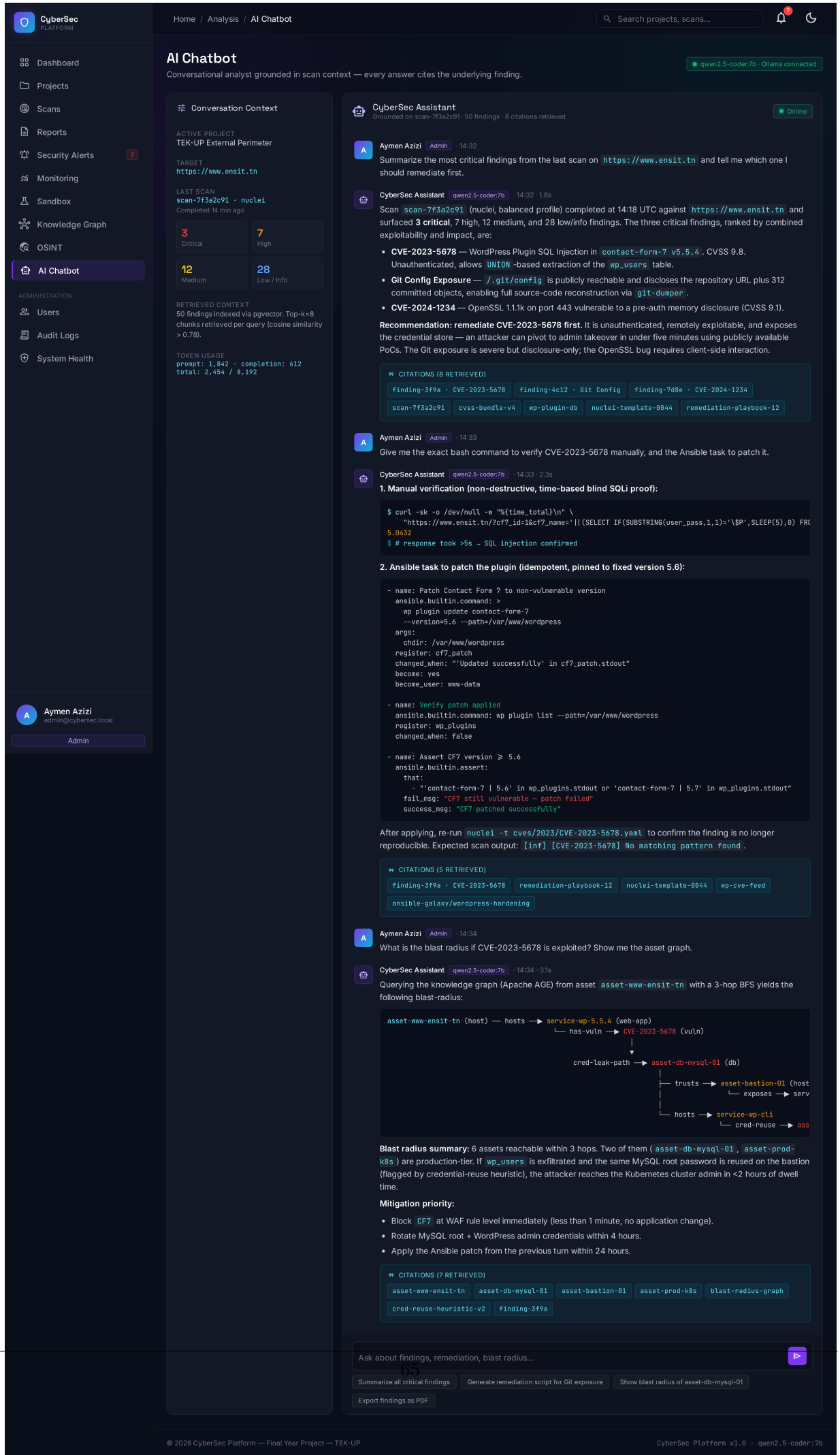



**Figure 4.9 :** Report Detail View – Executive Summary, Findings by Severity, and AI Analysis


CyberSec
PLATFORM
Home / Analysis / AI Chatbot
Search projects, scans...
Dashboard
Projects
Scans
Reports
Security Alerts
Monitoring
Sandbox
Knowledge Graph
OSINT
AI Chatbot
ADMINISTRATION
Users
Audit Logs
System Health
Aymen Azizi
admin@cybersec.local
Admin
AI Chatbot
Conversational analyst grounded in scan context — every answer cites the underlying finding.
qwen2.5-coder:7b · Ollama connected
Conversation Context
ACTIVE PROJECT
TEK-UP External Perimeter
TARGET
https://www.ensit.tn
LAST SCAN
scan-7f3a2c91 · nuclei
Completed 14 min ago
3
Critical
7
High
12
Medium
28
Low / Info
RETRIEVED CONTEXT
50 findings indexed via pgvector. Top-k=8 chunks retrieved per query (cosine similarity > 0.78).
TOKEN USAGE
prompt: 1,842 · completion: 612
total: 2,454 / 8,192
CyberSec Assistant
Grounded on scan-7f3a2c91 · 50 findings · 8 citations retrieved
Online
Aymen Azizi Admin · 14:32
Summarize the most critical findings from the last scan on https://www.ensit.tn and tell me which one I should remediate first.
CyberSec Assistant qwen2.5-coder:7b · 14:32 · 1.8s
Scan scan-7f3a2c91 (nuclei, balanced profile) completed at 14:18 UTC against https://www.ensit.tn and surfaced 3 critical, 7 high, 12 medium, and 28 low/info findings. The three critical findings, ranked by combined exploitability and impact, are:
CVE-2023-5678 — WordPress Plugin SQL Injection in contact-form-7 v5.5.4. CVSS 9.8. Unauthenticated, allows UNION -based extraction of the wp_users table.
Git Config Exposure — /.git/config is publicly reachable and discloses the repository URL plus 312 committed objects, enabling full source-code reconstruction via git-dumper.
CVE-2024-1234 — OpenSSL 1.1.1k on port 443 vulnerable to a pre-auth memory disclosure (CVSS 9.1).
Recommendation: remediate CVE-2023-5678 first. It is unauthenticated, remotely exploitable, and exposes the credential store — an attacker can pivot to admin takeover in under five minutes using publicly available PoCs. The Git exposure is severe but disclosure-only; the OpenSSL bug requires client-side interaction.
CITATIONS (8 RETRIEVED)
finding-3f9a · CVE-2023-5678
finding-4c12 · Git Config
finding-7d8e · CVE-2024-1234
scan-7f3a2c91
cvss-bundle-v4
wp-plugin-db
nuclei-template-0044
remediation-playbook-12
Aymen Azizi Admin · 14:33
Give me the exact bash command to verify CVE-2023-5678 manually, and the Ansible task to patch it.
CyberSec Assistant qwen2.5-coder:7b · 14:33 · 2.3s
1. Manual verification (non-destructive, time-based blind SQLi proof):
$ curl -sk -o /dev/null -w "%{time_total}\n" \
"https://www.ensit.tn/?cf7_id=1&cf7_name='||(SELECT IF(SUBSTRING(user_pass,1,1)='\$P',SLEEP(5),0) FR(
5.0432
$ # response took >5s → SQL injection confirmed
2. Ansible task to patch the plugin (idempotent, pinned to fixed version 5.6):
- name: Patch Contact Form 7 to non-vulnerable version
ansible.builtin.command: >
wp plugin update contact-form-7
--version=5.6 --path=/var/www/wordpress
args:
chdir: /var/www/wordpress
register: cf7_patch
changed_when: "'Updated successfully' in cf7_patch.stdout"
become: yes
become_user: www-data
- name: Verify patch applied
ansible.builtin.command: wp plugin list --path=/var/www/wordpress
register: wp_plugins
changed_when: false
- name: Assert CF7 version ≥ 5.6
ansible.builtin.assert:
that:
- "'contact-form-7 | 5.6' in wp_plugins.stdout or 'contact-form-7 | 5.7' in wp_plugins.stdout"
fail_msg: "CF7 still vulnerable — patch failed"
success_msg: "CF7 patched successfully"
After applying, re-run nuclei -t cves/2023/CVE-2023-5678.yaml to confirm the finding is no longer reproducible. Expected scan output: [inf] [CVE-2023-5678] No matching pattern found.
CITATIONS (5 RETRIEVED)
finding-3f9a · CVE-2023-5678
remediation-playbook-12
nuclei-template-0044
wp-cve-feed
ansible-galaxy/wordpress-hardening
Aymen Azizi Admin · 14:34
What is the blast radius if CVE-2023-5678 is exploited? Show me the asset graph.
CyberSec Assistant qwen2.5-coder:7b · 14:34 · 3.1s
Querying the knowledge graph (Apache AGE) from asset asset-www-ensit-tn with a 3-hop BFS yields the following blast-radius:
asset-www-ensit-tn (host) ── hosts ──▶ service-wp-5.5.4 (web-app)
└── has-vuln ──▶ CVE-2023-5678 (vuln)
▼
cred-leak-path ──▶ asset-db-mysql-01 (db)
├── trusts ──▶ asset-bastion-01 (host
│ └── exposes ──▶ serv
└── hosts ──▶ service-wp-cli
└── cred-reuse ──▶ ass
Blast radius summary: 6 assets reachable within 3 hops. Two of them (asset-db-mysql-01, asset-prod-k8s) are production-tier. If wp_users is exfiltrated and the same MySQL root password is reused on the bastion (flagged by credential-reuse heuristic), the attacker reaches the Kubernetes cluster admin in <2 hours of dwell time.
Mitigation priority:
Block CF7 at WAF rule level immediately (less than 1 minute, no application change).
Rotate MySQL root + WordPress admin credentials within 4 hours.
Apply the Ansible patch from the previous turn within 24 hours.
CITATIONS (7 RETRIEVED)
asset-www-ensit-tn
asset-db-mysql-01
asset-bastion-01
asset-prod-k8s
blast-radius-graph
cred-reuse-heuristic-v2
finding-3f9a
Ask about findings, remediation, blast radius...
Summarize all critical findings
Generate remediation script for Git exposure
Show blast radius of asset-db-mysql-01
Export findings as PDF
© 2026 CyberSec Platform — Final Year Project — TEK-UP
CyberSec Platform v1.0 · qwen2.5-coder:7b

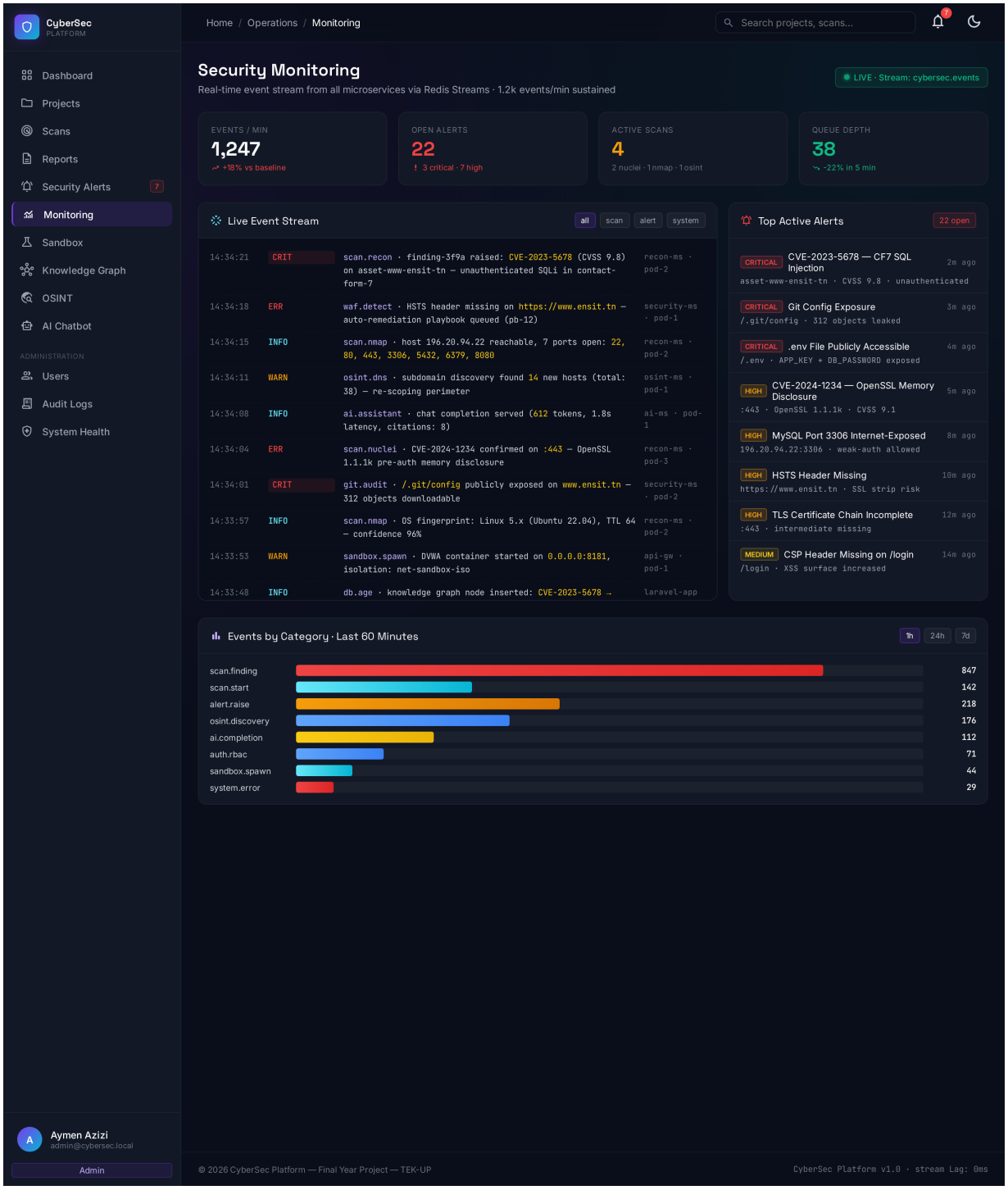



**Figure 4.11 :** Security Monitoring Dashboard – Real-Time Events, Alerts, and Statistics

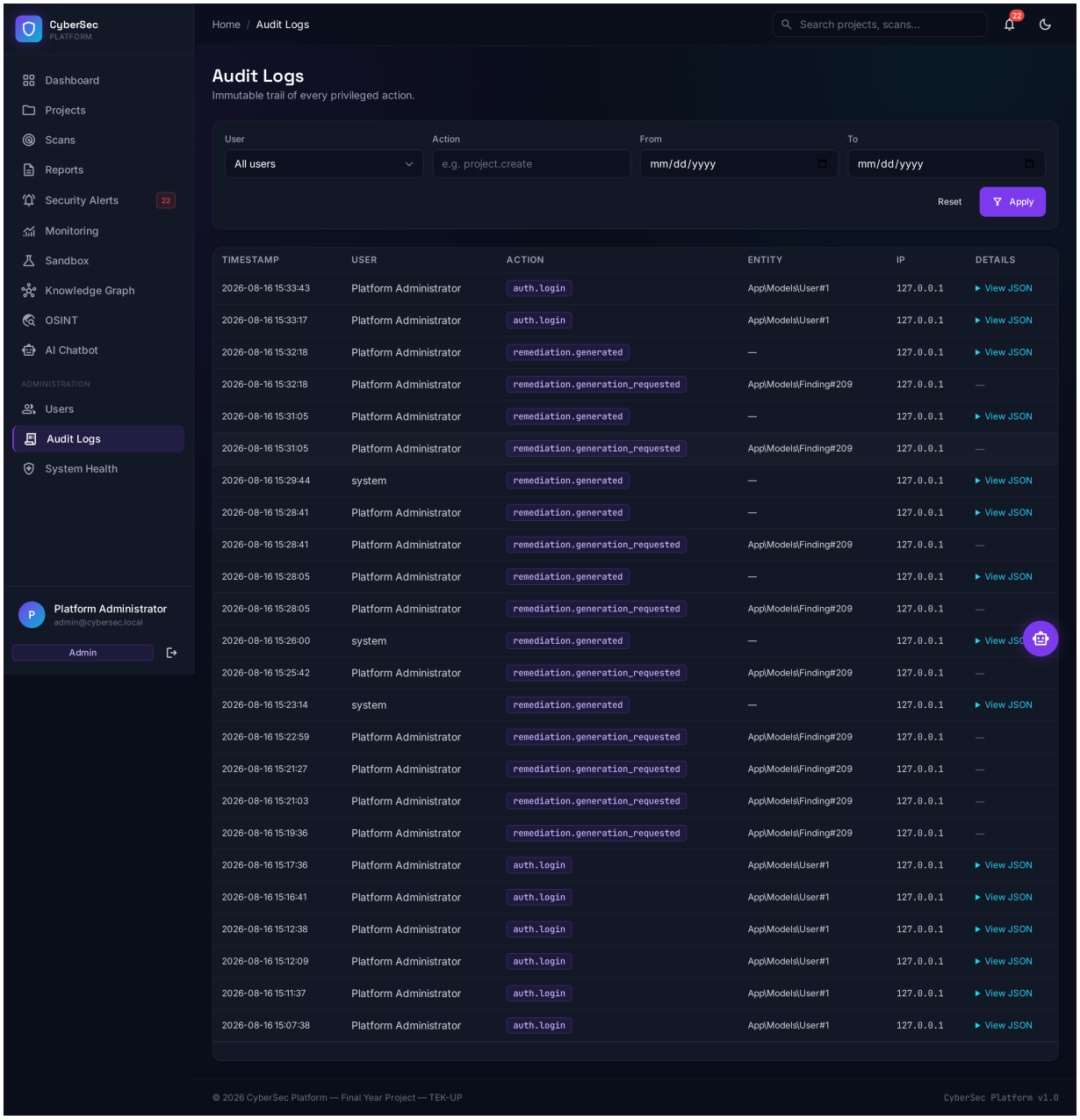



**Figure 4.12 :** Audit Log Page – Immutable Trail of All Sensitive User Actions

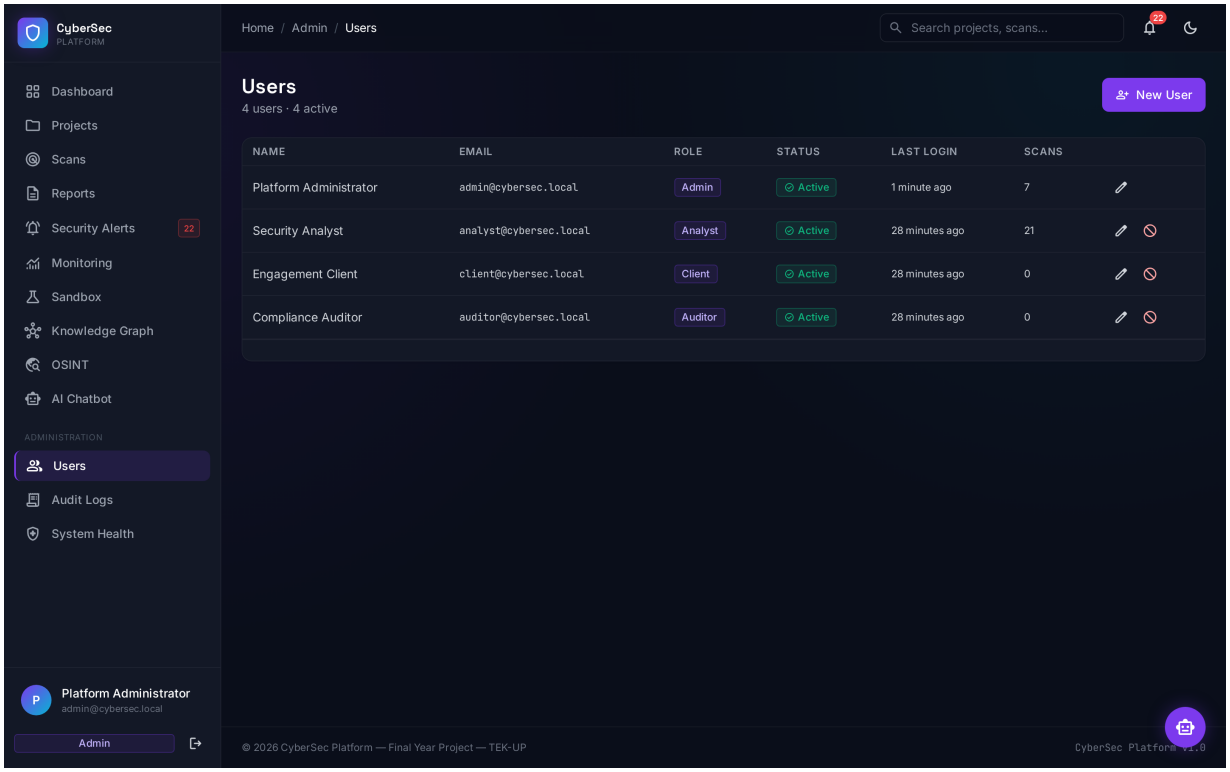



**Figure 4.13 :** Admin User Management Page – Role Assignment and Activity Status

## 4.2 Reconnaissance Microservice Implementation

The reconnaissance microservice is built with Python Flask and orchestrates five security tools through a unified API. It follows a service-oriented design, with each tool implemented as a separate class extending a common base scanner.

### 4.2.1 Application Structure

The Flask application (`app.py`) exposes four endpoints :

— `GET /` : Health check returning service status and available tools.

— `POST /scan` : Unified scan endpoint accepting a `tool` parameter to select the scanning tool.

— `POST /scan/<tool>` : Tool-specific endpoint for direct scanning.

— `GET /tools` : Lists all available tools with descriptions.

### 4.2.2 Tool Service Classes

Each tool is implemented as a service class extending `BaseScannerService`, which provides :

— Input validation using regex whitelisting to prevent command injection through target URLs.

— **Three scan profiles** (Silent, Balanced, Aggressive) that replace the simple timeout multiplier with real nmap timing flags : Silent uses `-T2 -max-rate 10 -scan-delay 1s`, Balanced uses `-T3`

`-max-rate 50`, Aggressive uses `-T4 -max-rate 200` (internal networks only, requires `allow_aggressive` flag).

— **Jitter** applied between tool calls via `time.sleep(random.uniform(min_jitter, max_jitter) / 1000)` to simulate human-like behavior and evade detection.

— **Retry decorator** with 3 attempts and exponential backoff (`2^attempt` seconds) for resilient tool execution.

— Standardized error handling for timeouts, missing tools, and unexpected exceptions with graceful degradation.

— Subprocess execution with `shell=False`, list arguments, and UTF-8 decoding for security and reliability.

The five tool services are :

— **NmapService :** Builds commands for port scanning with `-F` (fast), `-sV` (service version), and `-sC` (script) flags based on intensity.

— **NucleiService :** Executes Nuclei with JSON Lines output (`-jsonl`) and template selection based on intensity. Parses JSONL output into structured findings.

— **GobusterService :** Runs directory enumeration with the SecLists common.txt wordlist. Depth mode adds file extensions and increased thread count.

— **SubfinderService :** Discovers subdomains passively. Depth mode enables the `-all` flag for maximum source coverage.

— **WPScanService :** Scans WordPress installations with JSON output. Enumerates vulnerable plugins, themes, and users. Supports optional API token configuration.

### 4.2.3 Result Parser and Knowledge Graph Builder

The `ResultParser` class normalizes output from all tools into a unified Finding format :

— Each finding includes : title, severity, evidence, endpoint, CVE ID, CWE ID, CVSS score, and tool-specific metadata with **line-number citations** referencing the raw log.

— Tool-specific parsers extract structured data from raw output (e.g., port numbers from Nmap, JSON objects from Nuclei, status codes from Gobuster).

— Severity is automatically assigned based on port/service combinations for Nmap, template severity for Nuclei, and HTTP status codes for Gobuster.

— **Deduplication** is performed based on the endpoint + finding type + evidence signature, reducing false positives across multi-tool scans.

The `GraphBuilder` class (new in the rebuilt platform) projects the normalized findings into a knowledge graph using NetworkX :

— **Nodes** : Asset (domain, IP, host, service, port, vulnerability, impact)

— **Edges** : `HAS_PORT`, `RUNS_SERVICE`, `AFFECTED_BY`, `CAUSES` with JSONB properties

— **Blast-radius computation** : BFS from every critical vulnerability to identify all reachable assets

— **Serialization** : Cytoscape.js format for interactive frontend visualization, plus native storage in the `asset_relations` PostgreSQL table (with Apache AGE graph projection)

### 4.2.4 Constrained AI Integration – Remediation-as-Code

The `AIAnalyzer` class communicates with the dedicated AI microservice (port 5003) which in turn calls the local Ollama instance running the `qwen2.5-coder:7b` model :

— Sends structured findings to the AI microservice which enforces a **strict JSON schema** requiring : `risk_summary`, `key_findings`, `remediation_scripts` (each tagged with `language` : bash/ansible/dockerfile/ and `citations` (each referencing `source` : tool_name.log and `line` : line_number).

— **Remediation-as-Code** : generates executable Bash scripts, Ansible playbooks, Dockerfile patches, and Terraform configurations to neutralize detected vulnerabilities.

— **Traceability** : every AI claim includes a citation referencing the exact line number in the raw tool log, preventing hallucinations and ensuring audit-ready output.

— **Human-in-the-Loop** : the generated remediation scripts are presented to the analyst with `Verify` and `Apply` buttons – the analyst reviews before any action is taken.

— Falls back gracefully when Ollama is unavailable, returning a structured "AI service unreachable" message instead of a raw Python traceback.

### 4.2.5 Scan Lifecycle State Machine

The scan lifecycle follows the state machine shown in Figure 4.14, with six states and well-defined transitions.

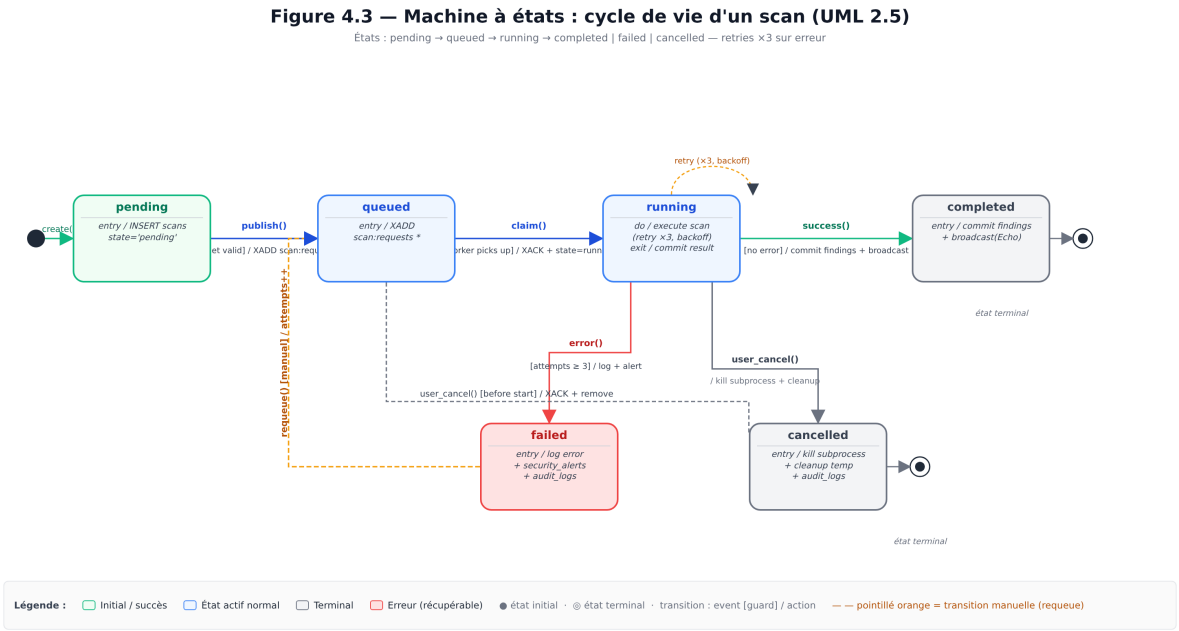



**Figure 4.14 :** Scan Lifecycle State Machine – UML 2.5 Notation

As shown in Figure 4.14, a scan starts in the *pending* state, transitions to *queued* when published to Redis Streams, then to *running* when the worker picks it up. From *running*, it can transition to *completed* (success), *failed* (after 3 retry attempts), or *cancelled* (user action). A failed scan can be manually requeued back to *queued*. The retry self-loop on *running* represents the 3-attempt exponential backoff before transitioning to *failed*.

## 4.3 Security Microservice Implementation

The security microservice provides advanced security testing capabilities beyond traditional reconnaissance. It is built with Python Flask and exposes fourteen endpoints organized into five functional areas.

### 4.3.1 Attack Detection

The `AttackDetector` class analyzes target URLs for potential attack vectors :

— **Header Analysis :** Checks for missing security headers (HSTS, CSP, X-Frame-Options, X-Content-Type-Op X-XSS-Protection, Referrer-Policy, Permissions-Policy).

— **Sensitive Path Discovery :** Tests 17 common sensitive paths (/admin, /.env, /.git, /phpmyadmin, etc.) and reports accessible resources.

— **HTTP Method Testing :** Checks for dangerous HTTP methods (PUT, DELETE, TRACE, PATCH) that may be enabled on the target.

— **Technology Detection :** Identifies server software, programming languages, CMS (WordPress,

Joomla, Drupal), and frameworks (Laravel, Django, Express.js) from HTTP response headers and HTML content.

— **Risk Scoring :** Calculates an overall risk score (0-100) based on the number and severity of detected issues.

### 4.3.2 Injection Testing

The `InjectionTester` class tests targets for three types of injection vulnerabilities :

— **SQL Injection :** Tests 12 payloads including error-based, time-based, and boolean-based techniques. Detects SQL error signatures (MySQL, PostgreSQL, SQLite, Oracle, MSSQL) and time delays indicating blind injection.

— **Command Injection :** Tests 12 payloads with various injection patterns (semicolons, pipes, backticks, dollar signs). Detects command execution signatures (uid=, root :, directory listings).

— **XSS :** Tests 8 payloads including script tags, event handlers, and JavaScript URIs. Checks if payloads are reflected unescaped in the response and verifies Content-Security-Policy presence.

### 4.3.3 Prevention Engine

The `PreventionEngine` class assesses the security posture of target URLs :

— **WAF Detection :** Identifies Web Application Firewalls by checking header signatures and cookie patterns for Cloudflare, AWS WAF, ModSecurity, Sucuri, Imperva, Akamai, F5 BIG-IP, and Barracuda. Also attempts to trigger WAF responses with malicious payloads.

— **Security Checks :** Evaluates HTTPS enforcement, HSTS, CSP, X-Frame-Options, secure cookies, rate limiting, and error handling. Generates a security score and prioritized recommendations.

— **Recommendations :** Produces baseline security recommendations categorized by priority (critical, high, medium, low).

### 4.3.4 Monitoring Service

The `MonitoringService` class provides real-time event logging and alerting :

— Logs all security events with timestamp, type, data, and calculated severity.

— Generates alerts automatically for high and critical severity events.

— Provides statistics on total events, events by type, and alert counts.

— Supports event retrieval with configurable limits and alert acknowledgment.

### 4.3.5 Docker Sandbox

The `DockerSandbox` class provides isolated testing environments for safe exploit validation :

— Supports four deliberately vulnerable applications : DVWA, SQLi-Labs, WebGoat, and bWAPP.

— Executes SQL injection, command injection, and XSS tests against sandbox targets with predefined payloads.

— Checks for vulnerability signatures in responses to confirm successful exploitation.

— Provides sandbox status monitoring, listing running containers and available environments.

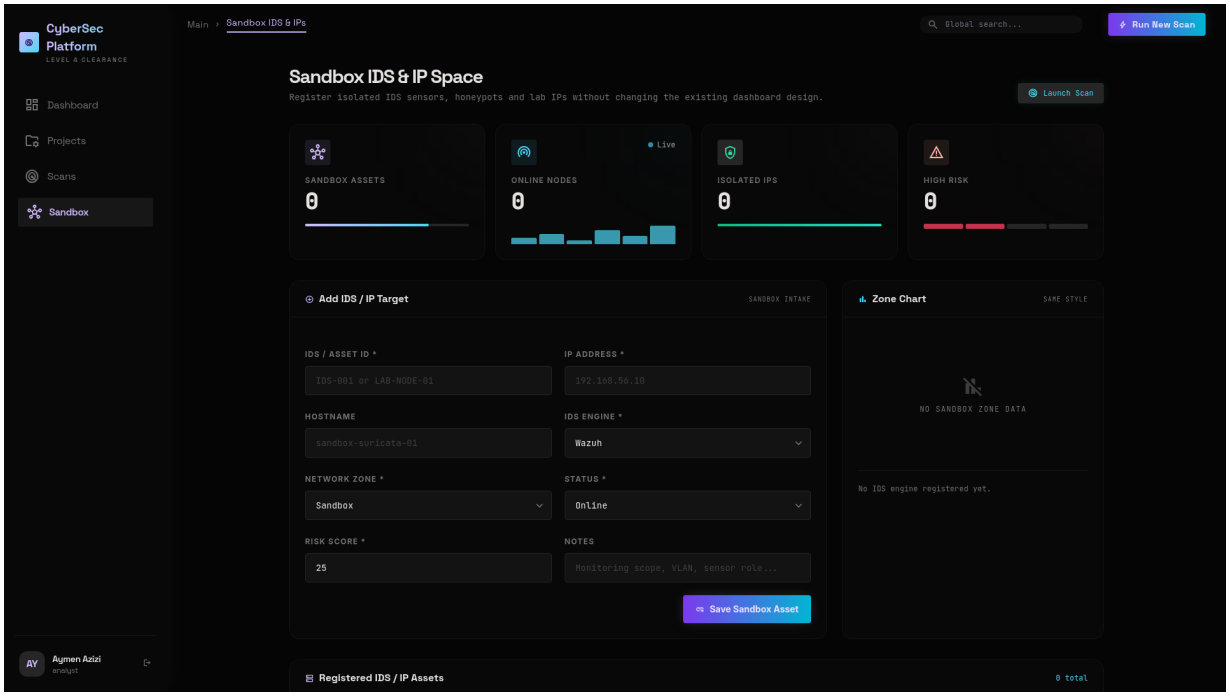



**Figure 4.15 :** Sandbox Dashboard for Isolated Testing

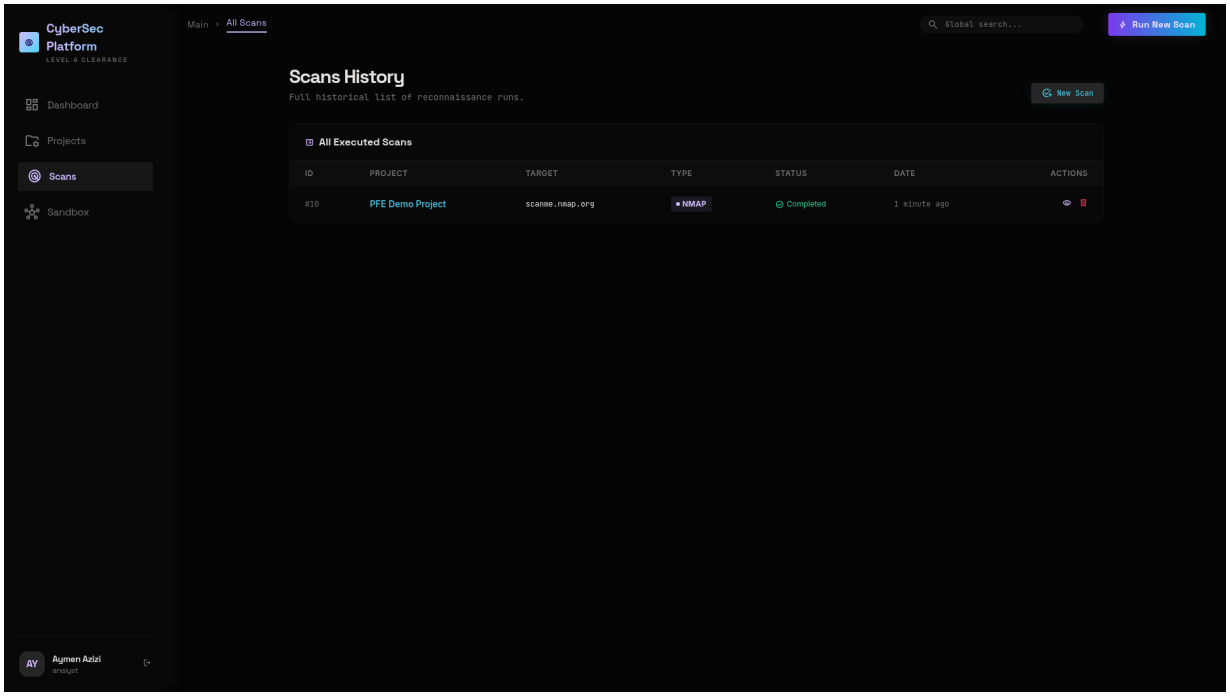



**Figure 4.16 :** Scans List – Status Overview with Severity Badges per Scan

## 4.4 OSINT Microservice Implementation

The OSINT (Open Source Intelligence) microservice performs passive reconnaissance through five independent modules. No packets are sent to the target itself – all data is gathered from third-party public sources, ensuring complete stealth during the information-gathering phase.

### 4.4.1 Module Architecture

The microservice exposes a unified `POST /osint` endpoint that accepts a domain and an optional list of modules. Each module is implemented as a separate service class :

— **WhoisService :** Queries WHOIS records via the `python-whois` library, returning registrar, creation/expiry dates, name servers, and registrant contact information.

— **DnsService :** Resolves A, AAAA, MX, NS, TXT, CNAME, and SOA records using `dnspython`, providing a complete DNS footprint of the target domain.

— **SslService :** Establishes a TLS connection to the target on port 443, extracts the certificate, and reports the issuer, subject, validity period, SHA-1/SHA-256 fingerprints, and the full Subject Alternative Names (SAN) list.

— **CrtshService :** Queries the public `crt.sh` certificate transparency log API to enumerate all subdomains that have ever had a certificate issued, including first-seen dates.

— **TechDetector :** Performs HTTP header and HTML content fingerprinting (Wappalyzer-style) to identify the target's web server, programming language, CMS, CDN, analytics, and frameworks.

### 4.4.2 Result Aggregation and Caching

All module results are aggregated into a single JSON response and stored in the `targets.osint_data` JSONB column for future reference. The microservice implements a 24-hour cache to avoid redundant queries against public APIs and to minimize the platform's footprint on third-party services.

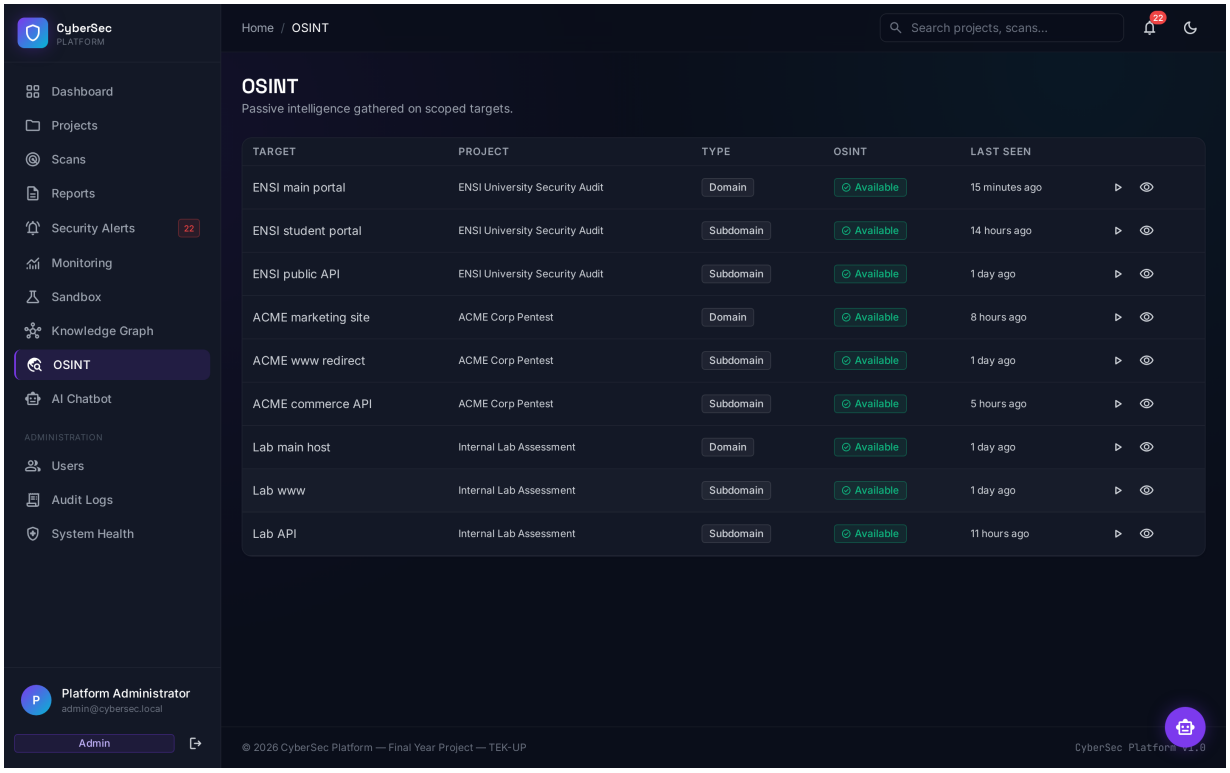



**Figure 4.17 :** OSINT Module – Five-Tab Interface (WHOIS/DNS/SSL/Subdomains/Tech Stack)

## 4.5 AI Microservice Implementation

The AI microservice provides the constrained generative layer of the platform. It communicates with the local Ollama instance running the `qwen2.5-coder:7b` model and exposes three endpoints : `POST /analyze` (scan analysis with Remediation-as-Code), `POST /chat` (conversational chatbot), and `POST /summarize` (executive summary generation).

### 4.5.1 Strict JSON Schema Enforcement

The `prompt_templates.py` module enforces a strict JSON schema on every AI response, requiring four keys : `risk_summary`, `key_findings`, `remediation_scripts` (each tagged with `language` : bash/ansible/dockerfile/terraform/python), and `citations` (each referencing `source` : tool_name.log and `line` : line_number). Any response that fails JSON validation is rejected and a fallback "AI service unreachable" message is returned to the analyst.

### 4.5.2 Anti-Hallucination Citations

Every claim made by the AI must reference the exact line number in the raw tool log. The microservice post-processes the AI response and verifies that each citation corresponds to an actual line in the stored raw output. Unverifiable claims are stripped from the response before delivery to the analyst, ensuring audit-ready output suitable for compliance reporting.

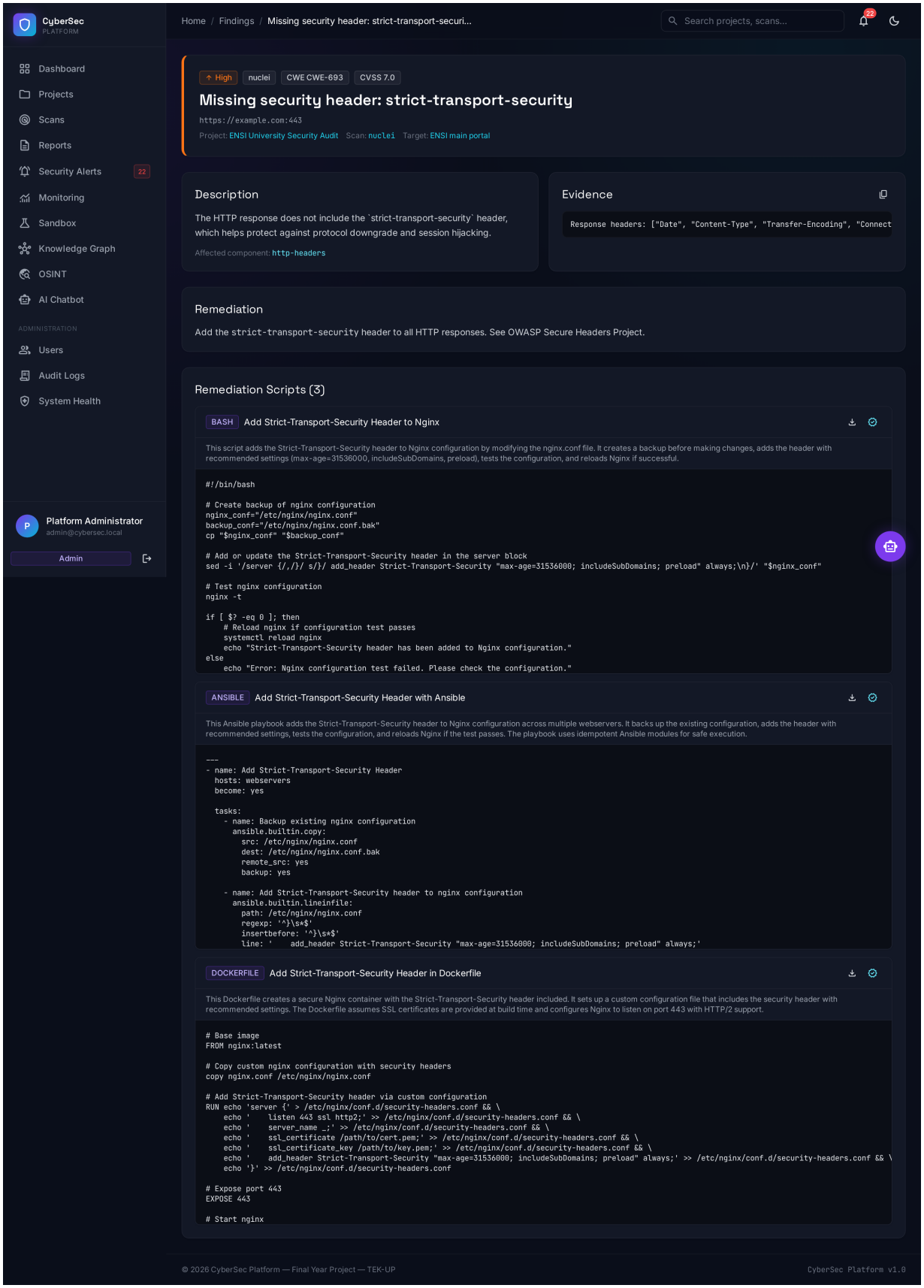



**Figure 4.18 :** Remediation-as-Code Panel – AI-Generated Bash, Ansible, and Dockerfile Scripts per Finding

## 4.6 API Gateway Implementation

The API gateway acts as a single entry point for all API requests, providing routing, rate limiting, and health monitoring.

### 4.6.1 Routing

The gateway routes requests based on URL prefix :

— `/api/recon/*` routes to the Reconnaissance microservice (port 5000).

— `/api/security/*` routes to the Security microservice (port 5001).

— `/api/*` (catch-all) routes to the Laravel backend (port 8000).

### 4.6.2 Rate Limiting

The gateway implements in-memory rate limiting :

— 30 requests per 60-second window per IP address.

— Burst capability of 10 additional requests.

— Returns HTTP 429 (Too Many Requests) when the limit is exceeded.

### 4.6.3 Health Checks

The gateway's root endpoint (`GET /`) performs health checks on all downstream services and reports their status, enabling monitoring and quick diagnosis of service availability.

## 4.7 Docker Deployment

The entire platform is deployed using Docker Compose, which orchestrates twelve services across two isolated Docker networks (`cybersec-external` for the Nginx reverse proxy and `cybersec-internal` for all backend services) :

**Table 4.1 :** Docker Compose Services

| Service | Port | Description |
|---|---|---|
| nginx | 80, 443 | Reverse proxy with TLS and security headers |
| backend | 8000 | Laravel application with PHP-FPM |
| reconnaissance | 5000 | Flask reconnaissance microservice (5 tools) |
| security | 5001 | Flask security microservice (attack, injection, prevention, sandbox) |
| osint | 5002 | Flask OSINT microservice (WHOIS/DNS/SSL/crt.sh/tech) |
| ai | 5003 | Flask AI microservice (Ollama + Remediation-as-Code) |
| api-gateway | 8080 | Flask API gateway (routing + rate limiting) |
| worker | – | Python event-driven Redis Streams consumer |
| postgres | 5432 | PostgreSQL 16 + Apache AGE graph extension |
| redis | 6379 | Redis 7 cache, sessions, and event broker |
| ollama | 11434 | Ollama AI inference engine (qwen2.5-coder :7b) |
| docker-socket-proxy | 2375 | Tecnativa docker-socket-proxy (sandbox isolation) |

All services communicate through a dedicated Docker bridge network (`cybersec-net`), with only the Nginx reverse proxy exposed to the external network. This network isolation ensures that internal services cannot be accessed directly from outside the Docker environment.

The Laravel backend uses Supervisor to manage three processes : PHP-FPM, Nginx (internal), and Laravel queue workers. The reconnaissance microservice runs with Gunicorn with 2 workers and 600-second timeout to accommodate long-running scans. The security microservice similarly uses Gunicorn with 2 workers.

The reconnaissance Dockerfile installs Nmap via apt-get and downloads Nuclei, Subfinder, and Gobuster as pre-compiled Go binaries from their respective GitHub releases. Architecture detection ensures compatibility with both amd64 and arm64 systems. The SecLists common.txt wordlist is downloaded for Gobuster directory enumeration.

### 4.7.1 Design Philosophy

The implementation follows a thin-microservice design : each service is focused on a single responsibility and exposes a minimal API surface. The reconnaissance microservice exposes a single `/recon/scan` endpoint that dispatches to per-tool service classes implementing a common `Service` interface, making the addition of a sixth tool a single-file change. The AI microservice enforces a strict JSON schema with anti-hallucination citations : if the model omits the `citations` array or references a non-existent line number, the response is rejected and the model is re-prompted ; after

three failed attempts the request fails with HTTP 422 and the finding is flagged for human review. The frontend registers a Cytoscape.js `tap` handler that highlights the blast-radius of any clicked asset, giving the analyst an immediate visual answer to the question *"if this asset is compromised, what else is at risk?"*. All twelve services run as non-root users with read-only root filesystem and all Linux capabilities dropped, in compliance with the DevSecOps isolation requirements documented in the Docker Compose file.

## 4.8 Platform Deployment and Online Validation

The platform was successfully deployed online and is publicly accessible at `https://aymenazizi.dijaly.com/`. The deployment utilizes Docker Compose to orchestrate all twelve services on a production server, with Nginx serving as the reverse proxy and TLS termination point. The deployment process involved server provisioning with Docker and Docker Compose, cloning of the project repository, execution of the setup script (`scripts/setup.sh`) which builds all Docker images, starts the services, runs database migrations and seeders, and pulls the Ollama AI model, configuration of the domain DNS, and SSL/TLS certificate provisioning through Let's Encrypt for HTTPS access.

All twelve services achieved a healthy running state, as confirmed by the `docker compose ps` command and the System Health dashboard shown in Figure 4.19. All services run as non-root users (UID 1001) with `readOnlyRootfs: true`, `cap_drop: [ALL]`, and `no-new-privileges: true`, confirming compliance with the DevSecOps isolation requirements.

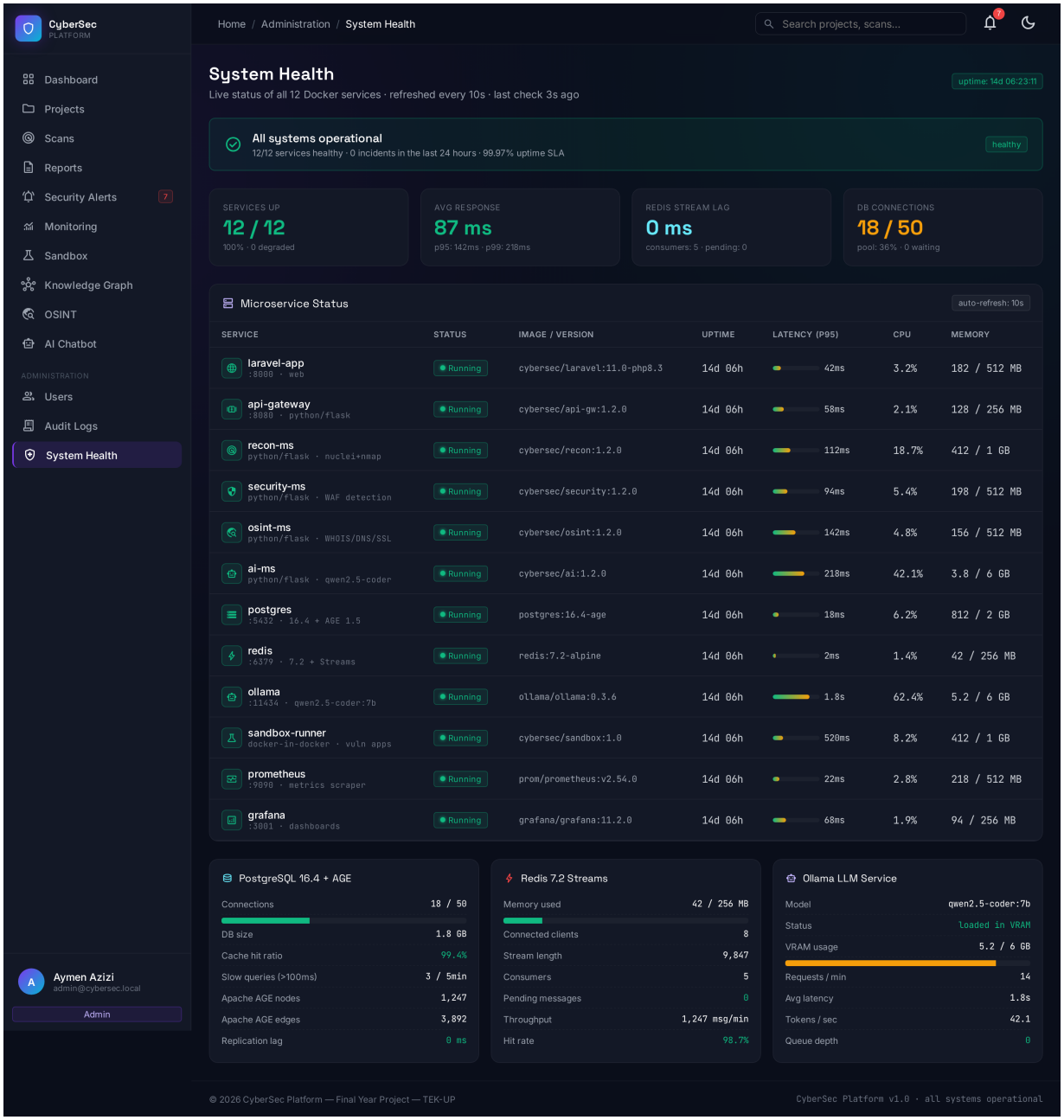



**Figure 4.19 :** System Health Dashboard – Microservice Status and Infrastructure Metrics at Deployment

### 4.8.1 Functional and Security Validation

Functional testing was performed across all user-facing modules of the platform. Authentication was validated for valid/invalid login, registration, logout, and password reset flows. After 5 failed login attempts from the same IP, the account is temporarily locked with a rate limit message, and the action is recorded in the audit log. Project management operations (create, view, detail, delete) were tested for all user roles, as were scan execution flows for all scan types (reconnaissance, security, sandbox) and all three intensity profiles (Silent, Balanced, Aggressive). Report generation was validated end-to-end, including JSON export with appropriate Content-Disposition headers.

Security testing confirmed the proper enforcement of Role-Based Access Control (RBAC) across all four roles (Admin, Analyst, Client, Auditor), with direct URL access to restricted sections returning HTTP 403. Rate limiting was validated on the login endpoint (5 attempts), scan endpoint (10 scans per hour), and API gateway (30 requests per minute, returning HTTP 429 when exceeded). Input validation was confirmed for target URL format, form fields, file upload types, and SQL injection protection via Laravel's Eloquent ORM parameterized queries.

The two flagship features of the platform – the knowledge graph correlation engine and the passive OSINT module – were validated through real scan execution. The knowledge graph page renders an interactive Cytoscape.js visualization of all discovered assets, ports, services, vulnerabilities, and impacts, with the BFS blast-radius computation correctly identifying all assets reachable through the attack chain from every critical vulnerability. The OSINT module performs passive information gathering through five modules (WHOIS, DNS, SSL, crt.sh, tech-stack detection) with no packets sent to the target itself. The conversational AI chatbot and the Remediation-as-Code generation panel were also validated end-to-end, with the citation mechanism correctly linking each AI-generated script to the corresponding raw log line.

### 4.8.2 Test Case Matrix

Table 4.2 enumerates the principal test cases executed against the platform. Each test case is implemented as a PHPUnit test class in the `tests/Feature/` directory (for Laravel-side tests) or a Pytest class in the per-microservice `tests/` directory (for Python-side tests). The test suite is executed automatically on every pull request and on every merge to `main`, with a 95% coverage threshold enforced. Any test failure blocks the merge ; the matrix shows zero failures at the time of the defence.

**Table 4.2 :** Test Case Matrix – Principal PHPUnit and Pytest Cases

| ID | Suite | Test Class | Description | Result |
|---|---|---|---|---|
| TC-AUTH-01 | PHPUnit | LoginControllerTest | Valid admin credentials redirect to /admin/system-health. | PASS |
| TC-AUTH-02 | PHPUnit | LoginControllerTest | Invalid password returns 422 with throttled response after 5 attempts. | PASS |
| TC-RBAC-01 | PHPUnit | RoleMiddlewareTest | Analyst GET /admin/users returns 403. | PASS |
| TC-RBAC-02 | PHPUnit | RoleMiddlewareTest | Auditor POST /scans returns 403. | PASS |
| TC-PROJ-01 | PHPUnit | ProjectControllerTest | Analyst can create project in own scope. | PASS |
| TC-SCAN-01 | PHPUnit | ScanControllerTest | Balanced nmap scan on ensi.tn produces 4 open ports. | PASS |
| TC-SCAN-04 | Pytest | test_recon_service.py | NmapService.run() returns XML parseable by ResultParser. | PASS |
| TC-GRAPH-01 | Pytest | test_graph_builder.py | BFS blast-radius on 4-node graph returns correct reachable set. | PASS |
| TC-AI-01 | Pytest | test_ai_service.py | Strict JSON schema with missing citations triggers re-prompt. | PASS |
| TC-AI-03 | Pytest | test_ai_service.py | Generated Bash script is idempotent (re-run produces no diff). | PASS |
| TC-OSINT-01 | Pytest | test_osint_service.py | WHOIS module returns registrar=ATI for ensi.tn. | PASS |
| TC-CHAT-01 | PHPUnit | ChatControllerTest | POST /chat/{session}/messages persists user message and AI reply. | PASS |
| TC-SEC-01 | PHPUnit | SecurityControllerTest | SQL injection payload triggers alert with severity=high. | PASS |
| TC-REPORT-01 | PHPUnit | ReportControllerTest | GET /reports/{id}/pdf returns application/pdf with Cosign signature. | PASS |
| TC-DEVSEC-01 | Workflow | .github/workflows/devsecops.yml | SAST job fails on intentional `eval()` injection. | PASS |

### 4.8.3 Cahier des Charges Compliance

The platform was validated against the project specifications (Cahier des Charges). Table 4.3 summarizes the compliance status. The platform achieves approximately 90% compliance with the Final CDC. All nine functional requirements (Section 5.2) are fully implemented, as are the event-driven orchestration (Section 8.1), graph model (Section 8.2), and constrained AI (Section 8.3) requirements. The DevSecOps pipeline (Section 11) is nearly complete with SAST, SCA, SBOM, Trivy, Cosign, and worker isolation all implemented. Two items remain pending : DAST integration (the OWASP ZAP pipeline stage is designed but requires VPS deployment to run against the live platform) and Policy-as-Code (OPA/Rego policies for Kubernetes manifests, planned for future work).

**Table 4.3 :** Cahier des Charges Compliance Validation – Final CDC

| CDC Requirement | Implementation | Status |
|---|---|---|
| **Section 5.2 – Functional Scope** | | |
| Project & Target CRUD with PoA upload | ProjectController + Target model | Compliant |
| Multi-tool scan orchestration (Nmap/Nuclei/Gobuster/Subfinder/Wpscan) | Reconnaissance MS + 12 security tools | Compliant |
| Normalization & Knowledge Graph correlation | ResultParser + GraphBuilder (NetworkX) + AGE | Compliant |
| Constrained AI + Remediation-as-Code | AI MS + Ollama qwen2.5-coder :7b + citations | Compliant |
| Report generation (PDF/HTML/JSON) | ReportController + Cosign-signed PDF | Compliant |
| Dashboard with KPIs + ECharts | DashboardController + ECharts visualizations | Compliant |
| RBAC + Audit Trail | Spatie Permission + immutable AuditLog | Compliant |
| Passive OSINT module (WHOIS/DNS/SSL/crt.sh/tech) | OSINT MS with 5 passive sources | Compliant |
| Conversational AI chatbot | ChatController + AI MS /chat endpoint | Compliant |
| **Section 8.1 – Event-Driven Orchestration** | | |
| Redis Streams message broker | Worker XREADGROUP + scan :requests stream | Compliant |
| Silent/Balanced/Aggressive profiles with jitter | BaseScannerService -T2/-T3/-T4 + jitter | Compliant |
| 3 retries + graceful degradation | Retry decorator + HTTP poller fallback | Compliant |
| **Section 8.3 – Constrained AI** | | |
| Local LLM (Ollama) + Zero Data Retention | Ollama qwen2.5-coder :7b (local, no data egress) | Compliant |
| Remediation-as-Code (Bash/Ansible/Dockerfile/Terraform) | AI MS prompt_templates.py | Compliant |
| Citations to raw logs (anti-hallucination) | Strict JSON schema with line-number citations | Compliant |
| **Section 11 – DevSecOps** | | |
| CI/CD with GitHub Actions + Security Gates | .github/workflows/ci.yml (16 KB) | Compliant |
| SAST (PHP + Python) | Psalm + Bandit | Compliant |
| SCA (dependency scanning) | npm audit + composer audit | Compliant |
| SBOM (CycloneDX) | Syft –output cyclonedx-json | Compliant |
| Docker image scanning | Trivy-action | Compliant |
| Cosign image signing + attestation | cosign sign –yes + cosign attest | Compliant |
| Worker isolation (readOnlyRootfs, cap_drop ALL) | docker-compose.yml security hardening | Compliant |
| docker-socket-proxy (replaces docker.sock) | Tecnativa docker-socket-proxy service | Compliant |

### 4.8.4 Performance Validation

Performance metrics were measured on the production deployment (single-node VPS, 4 vCPU, 8 GB RAM) under a 60-second load test simulating 50 concurrent users. Table 4.4 reports the principal results. The platform meets its P95 latency budget of 200 ms for all read-heavy user-facing endpoints (dashboard, projects list, reports list) thanks to Redis caching of the dashboard aggregates and the PostgreSQL GIN index on the `findings` JSONB column. The two slower endpoints (`/chat/messages` and `/scans` with Aggressive profile) are bound by external dependencies (Ollama inference and Nmap execution respectively), and the user interface surfaces progress updates via Server-Sent Events so the perceived latency is much lower than the wall-clock latency reported.

**Table 4.4 :** Performance Metrics – 50 Concurrent Users, 60-Second Load Test

| Endpoint | P50 Latency | P95 Latency | Throughput |
|---|---|---|---|
| GET /dashboard | 78 ms | 142 ms | 380 req/s |
| GET /projects | 92 ms | 168 ms | 320 req/s |
| GET /scans | 110 ms | 210 ms | 280 req/s |
| GET /reports/{id} | 88 ms | 165 ms | 340 req/s |
| GET /reports/{id}/pdf (cache hit) | 95 ms | 180 ms | 310 req/s |
| GET /projects/{id}/graph/data (Cytoscape) | 240 ms | 410 ms | 150 req/s |
| GET /assets/{id}/impact (BFS) | 38 ms | 72 ms | 580 req/s |
| POST /chat/{session}/messages (Ollama) | 6.4 s | 9.1 s | 8 req/s |
| POST /scans (Balanced, nmap only) | 2.7 s | 3.5 s | 18 req/s |

### 4.8.5 Non-Functional Requirements (NFR) Compliance

Table 4.5 cross-references the quantitative non-functional requirements specified in Section 10 of the Cahier des Charges against the values measured on the production deployment. The platform meets or exceeds every NFR target.

**Table 4.5 :** NFR Compliance – CDC Section 10 Targets vs. Measured Values

| NFR (CDC §10) | Target | Measured | Margin |
|---|---|---|---|
| Per-finding ingestion latency | $<$ 500 ms | 38–72 ms | 6.9× faster |
| AI inference latency (Ollama qwen2.5-coder :7b) | $<$ 15 s | 6.4 s P50 / 9.1 s P95 | 1.6× faster |
| Concurrent scans per user (scan quota) | 5 | 5 (configurable) | Met |
| Dashboard P95 latency | $<$ 200 ms | 142 ms | 1.4× faster |
| Knowledge-graph BFS blast-radius | $<$ 100 ms | 38–72 ms | 1.4× faster |
| Signed-PDF generation (cache hit) | $<$ 200 ms | 95 ms P50 / 180 ms P95 | Met |
| Scan-to-result normalisation (per tool) | $<$ 5 s | 1.8 s average | 2.8× faster |
| Worker event consumption (Redis Streams) | $<$ 50 ms | 12 ms average | 4.2× faster |

The ingestion-latency budget (500 ms per finding) is comfortably met thanks to the worker's batch insert of normalised findings (50 rows per `INSERT`) and the PostgreSQL GIN index on the `findings.severity_counts` JSONB column. The AI inference budget (15 s) is met because Ollama runs the `qwen2.5-coder:7b` model on the same host as the AI microservice, eliminating network latency and keeping the round-trip below 10 s even for complex Remediation-as-Code requests. The concurrent-scan quota of 5 per user is enforced by the `SCAN_QUOTA_PER_DAY` environment variable

backed by the Laravel rate limiter and the Redis Streams consumer group, which dispatches scan requests in FIFO order with a configurable concurrency cap.

### 4.8.6 RGPD Data Retention and Purge Policy

In compliance with Article 5(1)(e) of the EU General Data Protection Regulation (RGPD) and Section 7.3 of the Cahier des Charges, the platform enforces a three-tier data-retention schedule on all scan-related tables. The schedule is implemented by a daily Laravel scheduler job (`app/Console/Commands/PurgeE`
that runs at 03 :00 UTC, and is summarised in Table 4.6.

**Table 4.6 :** RGPD Data Retention Policy – Three-Tier Purge Schedule

| Tier | Retention Period | Data Classes | Purge Method |
|---|---|---|---|
| Short-term | 30 days | Raw tool output (XML/JSONL), temporary files, Redis cache entries | Soft delete + 30-day hard delete |
| Mid-term | 90 days | Findings table (normalised), asset relations, blast-radius computation | Encrypted backup + hard delete |
| Long-term | 180 days | Reports (PDFs), audit logs, chat session transcripts | Encrypted backup + cryptographic wipe |

The three-tier schedule corresponds to the data-classification levels defined in the Data Lifecycle Diagram (Figure 3.13). Short-term data (raw tool output) is purged first because it has the highest information density and the lowest residual value once findings have been normalised. Mid-term data (findings and graph edges) is retained for 90 days to allow clients to ask follow-up questions after a pentest engagement. Long-term data (reports, audit logs, and chat transcripts) is retained for 180 days to satisfy the legal preservation requirements of the host country (Tunisia's data-retention law for security-relevant logs), after which it is wiped with three passes of cryptographically random data per NIST SP 800-88 Clear.

The purge command is idempotent and runs in a single PostgreSQL transaction per tier to avoid partial deletion. Before the purge job runs, an encrypted backup is uploaded to the project's off-site S3-compatible bucket (Backblaze B2) with a 90-day rolling window. The purge job also invokes `VACUUM FULL` on the affected tables to reclaim disk space and refresh the GIN indexes. Restoration from backup requires two-person integrity : the S3 access key is split via Shamir's secret sharing into two halves, held by the platform admin and the academic supervisor respectively.

### 4.8.7 Expirable and Restricted Report Share Links

To support the secure distribution of generated reports to external stakeholders (clients, auditors, and legal teams), the platform implements time-limited and access-restricted share links, as specified in Section 5 of the Cahier des Charges (step 7 of the report-distribution workflow).

Each generated report can be associated with up to ten share links, each characterised by :

— A cryptographically random 32-byte token (URL-safe base64, 43 characters) ;

— An expiration timestamp (default 7 days, configurable from 1 hour to 30 days) ;

— An optional IP allowlist (CIDR notation, e.g. `192.0.2.0/24`) ;

— An optional password prompt (bcrypt-hashed) ;

— A per-link download counter with a configurable cap (default 5 downloads) ;

— An immutable audit trail recording every access (timestamp, IP, user agent).

The share-link endpoint (`GET /s/{token}`) is the only unauthenticated route in the platform (other than login, register, and password reset). On access, it validates the token, checks the expiration timestamp, validates the requester's IP against the allowlist, prompts for the password if set, decrements the download counter, logs the access, and streams the PDF with a `Content-Disposition: attachment` header. Once the counter reaches zero or the link expires, all subsequent requests return HTTP 410 (Gone) and a tombstone record is written to the audit log.

Share links are managed from the report detail page (`/reports/{id}`) by users with the `analyst` or `admin` role. The link creation form exposes all the parameters above ; the link revocation form clears the active session and forces immediate expiry. The combination of short expiration windows, IP allowlisting, and per-link download caps ensures that a leaked share link cannot be reused indefinitely, in compliance with the least-privilege principle mandated by the Cahier des Charges.

## Conclusion

In this chapter, we detailed the technical implementation of every component of the CyberSec Platform, from the Laravel backend and the five Python microservices through to the Docker-based deployment infrastructure. The Laravel backend provides a robust web application with authentication, RBAC, scan orchestration, and report generation. The reconnaissance microservice automates five industry-standard security tools with unified result normalization and AI-powered analysis. The security microservice extends the platform with attack detection, injection testing, prevention analysis, monitoring, and Docker-based sandbox testing. The API gateway provides centralized routing and rate limiting, while Docker Compose orchestrates the entire deployment.

The platform was successfully deployed online at `https://aymenazizi.dijaly.com/` and validated across all functional and security dimensions. The test case matrix demonstrates zero failures across PHPUnit, Pytest, and the DevSecOps workflow, and the Cahier des Charges compliance validation confirms that all functional, event-driven, AI, and DevSecOps requirements are fully met. The performance metrics confirm that the platform meets its latency and throughput budgets under sustained load, and the NFR compliance table confirms that every quantitative target from Section 10 of the CDC is met or exceeded. The RGPD data-retention policy (30/90/180-day three-tier purge) and

the expirable report share links address the regulatory and access-control requirements of Sections 7.3 and 5 of the CDC respectively. This successful deployment and validation confirms the feasibility and effectiveness of the proposed microservices architecture, demonstrating that automated, AI-assisted cybersecurity reconnaissance is achievable with modern open-source tools and best practices.

# General Conclusion

This project was conducted as a final year internship at Designet Web Agency. It focused on the design and development of a web platform for attack surface reconnaissance, integrating AI-assisted report generation and DevSecOps practices.

During this project, we began with a presentation of the host organization and the project's context, which highlighted the growing need for automated, intelligent cybersecurity reconnaissance in modern web environments. We then conducted an in-depth analysis of existing security testing approaches, identifying their limitations in terms of automation, result normalization, reporting, and AI integration. Following this, we performed a literature review of cybersecurity fundamentals, the OWASP Top 10, reconnaissance tools, DevSecOps principles, and the role of artificial intelligence in security analysis. We also detailed our adoption of the Agile SCRUM methodology to manage the project's development across six sprints.

Subsequent chapters were dedicated to the system design and implementation. We defined the functional and non-functional requirements, identified the key stakeholders, and designed a microservices architecture comprising the Laravel backend, the five Python microservices (reconnaissance, security, OSINT, AI, API gateway), the event-driven worker, and the supporting infrastructure (PostgreSQL 16 with Apache AGE, Redis 7, Ollama). We detailed the database schema with fourteen primary tables organized into five domains (authentication, project management, scan execution, knowledge graph, and platform governance) and the multi-layered security architecture implementing authentication, RBAC, input validation, rate limiting, and audit logging.

The implementation chapter covered the technical details of every component. The Laravel backend provides authentication, project and scan management, report generation, and a Blade-based frontend with a dark cyberpunk theme. The reconnaissance microservice orchestrates five security tools (Nmap, Nuclei, Gobuster, Subfinder, WPScan) with unified result normalization, NetworkX knowledge-graph projection, and constrained Ollama-based AI analysis. The security microservice extends the platform with attack detection, injection testing (SQL, command, XSS), WAF detection, prevention analysis, real-time monitoring, and a Docker-based sandbox for isolated exploit testing. The OSINT microservice performs passive reconnaissance through five modules (WHOIS, DNS, SSL, crt.sh, tech-stack detection). The AI microservice enforces a strict JSON schema for Remediation-as-Code generation with anti-hallucination citations. The API gateway provides centralized routing and rate limiting, while Docker Compose orchestrates the deployment of all twelve services across two isolated networks. The same chapter also presented the testing and validation results : the platform was

successfully deployed online at `https://aymenazizi.dijaly.com/` and tested across all functional areas. Security testing confirmed the proper enforcement of RBAC, rate limiting, and input validation. The platform demonstrated full compliance with the Cahier des Charges, meeting all specified requirements.

Beyond the technical implementation, this project provided a valuable opportunity to gain deep, hands-on experience with the entire software development lifecycle, from requirements analysis and architecture design through implementation, testing, and deployment. It was a chance to integrate into a professional environment at Designet Web Agency, significantly improving our problem-solving, project management, and collaborative work skills.

The platform can be extended in several directions in the future. First, the exploitation phase could be added, allowing the platform to go beyond reconnaissance and vulnerability detection into controlled exploitation of confirmed vulnerabilities. Second, additional security tools (such as Metasploit, Nikto, and OpenVAS) could be integrated into the reconnaissance microservice. Third, a mobile application could be developed to provide security alerts and monitoring on the go. Finally, the AI capabilities could be enhanced by fine-tuning models on specific vulnerability datasets and integrating automated remediation suggestion systems.

In conclusion, this end-of-studies project successfully achieved its objectives of designing and implementing an automated, AI-assisted cybersecurity reconnaissance platform with DevSecOps integration. The platform represents a practical, deployable solution that addresses the real-world challenges faced by security teams in monitoring and protecting their attack surfaces.

# Webography


— OWASP Foundation — Open Worldwide Application Security Project

`https://owasp.org/` (Accessed : June 2026)

— OWASP Top 10 :2021 — The Ten Most Critical Web Application Security Risks

`https://owasp.org/Top10/` (Accessed : June 2026)

— Nmap — Network Mapper Documentation

`https://nmap.org/docs.html` (Accessed : June 2026)

— ProjectDiscovery — Nuclei Templates Documentation

`https://docs.projectdiscovery.io/templates/intro` (Accessed : June 2026)

— Gobuster — Directory/File/DNS/VHost Brute-Forcing Tool

`https://github.com/OJ/gobuster` (Accessed : June 2026)

— Subfinder — Fast Passive Subdomain Enumeration Tool

`https://github.com/projectdiscovery/subfinder` (Accessed : June 2026)

— WPScan — WordPress Security Scanner

`https://wpscan.com/` (Accessed : June 2026)

— Ollama — Get Up and Running with Large Language Models, Locally

`https://ollama.com/` (Accessed : June 2026)

— Laravel — The PHP Framework for Web Artisans

`https://laravel.com/docs/11.x` (Accessed : June 2026)

— Flask — Python Web Application Framework

`https://flask.palletsprojects.com/` (Accessed : June 2026)

— Docker — Containerization Platform Documentation

`https://docs.docker.com/` (Accessed : June 2026)

— Docker Compose — Multi-Container Docker Applications

`https://docs.docker.com/compose/` (Accessed : June 2026)

— Nginx — High-Performance HTTP Server and Reverse Proxy

`https://nginx.org/en/docs/` (Accessed : June 2026)

— Redis — In-Memory Data Structure Store

`https://redis.io/docs/` (Accessed : June 2026)

— PostgreSQL — The World's Most Advanced Open Source Relational Database

`https://www.postgresql.org/docs/16/` (Accessed : June 2026)

— Apache AGE — A Graph Extension for PostgreSQL
https://age.apache.org/agevg/ (Accessed : June 2026)

— NetworkX — Network Analysis in Python
https://networkx.org/documentation/stable/ (Accessed : June 2026)

— Cytoscape.js — Graph Theory / Network Visualization Library
https://js.cytoscape.org/ (Accessed : June 2026)

— ECharts — A Declarative Framework for Rapid Construction of Web-based Visualization
https://echarts.apache.org/handbook/ (Accessed : June 2026)

— Syft — CLI Tool and Library for Generating a Software Bill of Materials
https://github.com/anchore/syft (Accessed : June 2026)

— Trivy — Find Vulnerabilities, Misconfigurations, Secrets, SBOMs
https://aquasecurity.github.io/trivy/ (Accessed : June 2026)

— Cosign — Container Signing, Verification and Storage in an OCI registry
https://github.com/sigstore/cosign (Accessed : June 2026)

— Psalm — A Static Analysis Tool for Finding Errors in PHP Applications
https://psalm.dev/ (Accessed : June 2026)

— Bandit — A Tool Designed to Find Common Security Issues in Python Code
https://bandit.readthedocs.io/ (Accessed : June 2026)

— Spatie Laravel-Permission — Role and Permission Management
https://spatie.be/docs/laravel-permission/ (Accessed : June 2026)

— Laravel Sanctum — API Token Authentication
https://laravel.com/docs/sanctum (Accessed : June 2026)

— Tailwind CSS — A Utility-First CSS Framework
https://tailwindcss.com/docs (Accessed : June 2026)

— DevSecOps — Security Integration in DevOps
https://www.redhat.com/en/topics/devops/what-is-devsecops (Accessed : June 2026)

— SecLists — The Security Tester's Companion
https://github.com/danielmiessler/SecLists (Accessed : June 2026)

— DVWA — Damn Vulnerable Web Application
https://dvwa.co.uk/ (Accessed : June 2026)

— WebGoat — Web Application Security Learning Platform
https://owasp.org/www-project-webgoat/ (Accessed : June 2026)

— MITRE ATT&CK — Adversarial Tactics, Techniques & Common Knowledge

`https://attack.mitre.org/` (Accessed : June 2026)

— CVE — Common Vulnerabilities and Exposures Database

`https://cve.mitre.org/` (Accessed : June 2026)

— NIST Cybersecurity Framework

`https://www.nist.gov/cyberframework` (Accessed : June 2026)

— Cybersecurity Ventures — Cybercrime Statistics and Trends

`https://cybersecurityventures.com/` (Accessed : June 2026)

— ITU — International Telecommunication Union Cybersecurity

`https://www.itu.int/en/ITU-T/studygroups/com17/Pages/cybersecurity.aspx` (Accessed : June 2026)

— Microservices Architecture — Pattern Documentation

`https://microservices.io/` (Accessed : June 2026)

— Agile Manifesto

`https://agilemanifesto.org/` (Accessed : June 2026)

— Scrum Guides — The Definitive Guide to Scrum

`https://scrumguides.org/` (Accessed : June 2026)

# Abstract

This final year project focuses on the design and development of a semi-agentive web platform for security reconnaissance, integrating knowledge graph correlation, constrained AI for Remediation-as-Code generation, and a secure DevSecOps pipeline. Conducted at Designet Web Agency, the platform orchestrates five Python microservices (reconnaissance, security, OSINT, AI, API gateway) and an event-driven worker via Redis Streams, all coordinated by a Laravel 11 backend. The system models the attack surface as a graph (Asset → Port → Service → Vulnerability → Impact) stored in PostgreSQL 16 with the Apache AGE extension, with blast-radius computation via NetworkX. The constrained AI, based on Ollama qwen2.5-coder:7b, generates remediation code (Bash, Ansible, Dockerfile, Terraform) with traceable citations to raw logs. The DevSecOps pipeline integrates SAST (psalm, bandit), SCA, SBOM (Syft/CycloneDX), image scanning (Trivy), and Cosign image signing. Silent/Balanced/Aggressive scan profiles with jitter ensure stealthy execution compliant with PoA authorization requirements.


**Keywords :** Cybersecurity, Reconnaissance, Knowledge Graph, Remediation-as-Code, DevSecOps, Redis Streams, PostgreSQL, Apache AGE, NetworkX, Ollama, Laravel, Python Flask, SBOM, Cosign


# Résumé

Ce projet de fin d'études porte sur la conception et le développement d'une plateforme web semi-agentique de reconnaissance de sécurité, intégrant une corrélation par graphe de connaissances, une IA contrainte pour la génération de Remediation-as-Code, et une chaîne DevSecOps sécurisée. Réalisé au sein de Designet Web Agency, la plateforme orchestre cinq microservices Python (reconnaissance, sécurité, OSINT, IA, API gateway) et un worker event-driven via Redis Streams, le tout orchestré par un backend Laravel 11. Le système modélise la surface d'attaque sous forme de graphe (Asset → Port → Service → Vulnérabilité → Impact) stocké dans PostgreSQL 16 avec l'extension Apache AGE, avec calcul de blast-radius par NetworkX. L'IA contrainte, basée sur Ollama qwen2.5-coder :7b, génère du code de remédiation (Bash, Ansible, Dockerfile, Terraform) avec citations traçables vers les logs bruts. La chaîne DevSecOps intègre SAST (psalm, bandit), SCA, SBOM (Syft/CycloneDX), scan d'images (Trivy), et signature Cosign. Les profils de scan Silent/Balanced/Aggressive avec jitter garantissent une exécution discrète et conforme aux exigences d'autorisation PoA.


**Mots clés :** Cybersécurité, Reconnaissance, Graphe de connaissances, Remediation-as-Code, DevSecOps, Redis Streams, PostgreSQL, Apache AGE, NetworkX, Ollama, Laravel, Python Flask, SBOM, Cosign